



E-KİTAP

# YAZILIMCILARIN BİLMESİ GEREKEN ALGORİTMALAR

## İŞ DÜNYASINA YÖNELİK

ENES TOY

# İÇİNDEKİLER

## 1. Sıralama Algoritmaları

- 1.1 Bubble Sort
- 1.2 Selection Sort
- 1.3 Insertion Sort
- 1.4 Merge Sort
- 1.5 Quick Sort
- 1.6 Heap Sort
- 1.7 Radix Sort
- 1.8 Counting Sort
- 1.9 Bucket Sort

## 2. Arama Algoritmaları

- 2.1 Linear Search
- 2.2 Binary Search
- 2.3 Jump Search
- 2.4 Exponential Search

## 3. Veri Yapıları Algoritmaları

- 3.1 Linked List: Tekli, Çiftli, Döngüsel
- 3.2 Stack (Yığın)
- 3.3 Queue (Kuyruk)
- 3.4 Deque (Çift Yönlü Kuyruk)
- 3.5 Hashing ve Hash Tabloları
- 3.6 Binary Search Tree (BST)
- 3.7 AVL Tree (Self-balancing Binary Search Tree)
- 3.8 Red-Black Tree
- 3.9 Trie
- 3.10 Heap (Min-Heap, Max-Heap)
- 3.11 Graph Veri Yapıları (Adjacency List, Matrix)

## 4. Graf Algoritmaları

- 4.1 Depth-First Search (DFS)
- 4.2 Breadth-First Search (BFS)
- 4.3 Dijkstra's Algorithm (Kısa Yol Algoritması)
- 4.4 Bellman-Ford Algorithm
- 4.5 Floyd-Warshall Algorithm (Tüm Çiftler Arası En Kısa Yol)
- 4.6 Kruskal's Algorithm (MST - Minimum Spanning Tree)
- 4.7 Prim's Algorithm (MST)
- 4.8 Topological Sorting
- 4.9 Ford-Fulkerson Algorithm (Max Flow)

# İÇİNDEKİLER

## 5. Dinamik Programlama Algoritmaları

- 5.1 Fibonacci Sequence
- 5.2 Knapsack Problem
- 5.3 Longest Common Subsequence (LCS)
- 5.4 Longest Increasing Subsequence (LIS)
- 5.5 Coin Change Problem
- 5.6 Matrix Chain Multiplication
- 5.7 Rod Cutting Problem
- 5.8 Edit Distance (Levenshtein Distance)

## 6. Böl ve Fethet (Divide and Conquer) Algoritmaları

- 6.1 Merge Sort
- 6.2 Quick Sort
- 6.3 Binary Search
- 6.4 Strassen's Matrix Multiplication

## 7. Geri İzleme (Backtracking) Algoritmaları

- 7.1 N-Queens Problem
- 7.2 Subset Sum Problem
- 7.3 Sudoku Solver
- 7.4 Graph Coloring
- 7.5 Hamiltonian Path

## 8. Greedy Algoritmalar

- 8.1 Fractional Knapsack Problem
- 8.2 Huffman Coding
- 8.3 Activity Selection Problem
- 8.4 Job Scheduling Problem
- 8.5 Prim's Algorithm (MST)

## 9. İleri Seviye Algoritmalar

- 9.1 Floyd's Cycle-Finding Algorithm (Tortoise and Hare)
- 9.2 Topological Sort
- 9.3 Sieve of Eratosthenes
- 9.4 Union-Find (Disjoint Set)
- 9.5 Turing Machines

# İÇİNDEKİLER

## 10. Matematiksel Algoritmalar

10.1 Greatest Common Divisor (GCD) - Euclidean Algorithm

10.2 Sieve of Eratosthenes (Prime Number Generation)

10.3 Fast Exponentiation

10.4 Fermat's Little Theorem

10.5 Modular Arithmetic Algorithms

10.6 Chinese Remainder Theorem

## 11. String Matching ve Manipülasyon Algoritmaları

11.1 Rabin-Karp Algorithm

11.2 KMP (Knuth-Morris-Pratt) Algorithm

11.3 Boyer-Moore Algorithm

11.4 Z-Algorithm

## 12. Çok Değişkenli (Multi-Variable) Optimizasyon

12.1 Simulated Annealing

12.2 Genetic Algorithms

12.3 Ant Colony Optimization

## 13. Benzerlik ve Kümeleme Algoritmaları

13.1 K-Means Clustering

13.2 Hierarchical Clustering

13.3 Nearest Neighbors

13.4 DBSCAN

## 14. Diğer Önemli Algoritmalar

14.1 Bloom Filter

14.2 Reservoir Sampling

14.3 Karnaugh Map (Boolean Minimization)

14.4 Consensus Algorithms (Paxos, Raft, etc.)

14.5 MapReduce

14.6 Monte Carlo Methods

## 15.Uygulamalı Örnekler ve Mülakat Soruları Rehberi

## 16.Teşekkürler

# ÖNSÖZ

Sevgili Okuyucu,

**Yazılım geliştirme**, yalnızca kod yazmaktan ibaret değil; **akıllı ve verimli algoritmalar, modern iş dünyasında** süreçleri optimize etmenin ve **rekabet avantajı** sağlamanın anahtarıdır. İster **finans, sağlık**, ister **e-ticaret** sektöründe olun, güçlü **algoritmalar** sayesinde karmaşık problemleri çözebilir ve daha etkin çözümler üretebilirsiniz.

Bu e-kitap, yazılım geliştiricilerin iş dünyasında ihtiyaç duyduğu **temel ve ileri düzey algoritmaları** kapsamlı ve anlaşılır bir şekilde ele alıyor. Amacım, yalnızca teorik bilgiler vermek değil, aynı zamanda bu algoritmaların gerçek dünya senaryolarında nasıl uygulandığını somut örneklerle göstermek. Kitap boyunca **sıralama, arama, veri yapıları, graf algoritmaları** gibi temel konuların yanı sıra, **dinamik programlama, böl ve fethet, geri izleme ve greedy algoritmalar** gibi ileri seviye teknikleri de detaylıca inceleyeceğiz.

Ayrıca, öğrendiklerinizi doğrudan pratiğe dökebilmeniz için teknik **mülakatlarda sıkça karşılaşılan** algoritma problemlerini, **GitHub** üzerinden ulaşabileceğiniz detaylı çözümlerle birlikte derledim. Kitabın sonunda, bu çözümlerle birlikte **PHP diliyle** hazırlanmış **uygulamalı örneklere** de yer verdiğim bağlantılar sayesinde, konuları pekiştirme imkânı bulacaksınız.

İster algoritmalar konusunda yeni olun ister deneyimli bir yazılımcı, bu kitap **stratejik düşünme yeteneğinizi geliştirerek** yazılım dünyasında fark yaratmanıza yardımcı olacak.

Başarı dolu bir öğrenme yolculuğu dilerim!

**İyi çalışmalar!**

**Enes Toy**



# Sıralama Algoritmaları

## Bubble Sort Algoritması

**Bubble Sort**, sıralama algoritmalarından biridir ve oldukça basit bir mantığa dayanır. Bu algoritma, listedeki her öğeyi birbiriyle karşılaştırır ve sırasıyla yer değiştirir. Bir dizi içinde sıralama yaparken, her geçişte en büyük (veya en küçük) öğe sona "baloncuk" gibi yükselir. Bu işlem, tüm öğeler sıralanana kadar devam eder.

### Çalışma Prensibi

1. Listedeki öğeleri birbiriyle karşılaştırır.
2. Eğer bir öğe, bir sonraki öğeden daha büyükse, bu iki öğe yer değiştirir.
3. Bu işlem tüm liste sıralanana kadar devam eder.

### Örnek

Varsayalım ki sıralamak istediğimiz liste şu şekilde olsun:  
[5, 3, 8, 4, 2]

### Bubble Sort Algoritması Adımları

#### 1. Adım (İlk Geçiş)

- 5 ve 3 karşılaştırılır,  $5 > 3$  olduğu için yer değiştirir: [3, 5, 8, 4, 2]
- 5 ve 8 karşılaştırılır,  $5 < 8$  olduğu için değişiklik yapılmaz: [3, 5, 8, 4, 2]
- 8 ve 4 karşılaştırılır,  $8 > 4$  olduğu için yer değiştirir: [3, 5, 4, 8, 2]
- 8 ve 2 karşılaştırılır,  $8 > 2$  olduğu için yer değiştirir: [3, 5, 4, 2, 8]

#### 2. Adım (İkinci Geçiş)

- 3 ve 5 karşılaştırılır,  $3 < 5$  olduğu için değişiklik yapılmaz: [3, 5, 4, 2, 8]
- 5 ve 4 karşılaştırılır,  $5 > 4$  olduğu için yer değiştirir: [3, 4, 5, 2, 8]
- 5 ve 2 karşılaştırılır,  $5 > 2$  olduğu için yer değiştirir: [3, 4, 2, 5, 8]

#### 3. Adım (Üçüncü Geçiş)

- 3 ve 4 karşılaştırılır,  $3 < 4$  olduğu için değişiklik yapılmaz: [3, 4, 2, 5, 8]
- 4 ve 2 karşılaştırılır,  $4 > 2$  olduğu için yer değiştirir: [3, 2, 4, 5, 8]

#### 4. Adım (Dördüncü Geçiş)

- 3 ve 2 karşılaştırılır,  $3 > 2$  olduğu için yer değiştirir: [2, 3, 4, 5, 8]

Bu işlem tamamlandığında liste sıralanmış olur.

### Zaman Karmaşıklığı

- En kötü ve ortalama durum:  $O(n^2)$
- En iyi durum:  $O(n)$  (Eğer liste zaten sıralıysa)
- Küçük veri setleri için uygundur, ancak büyük veri setlerinde verimli değildir.

### Avantajları

- ✓ **Basit ve anlaşılır:** Uygulaması ve kavranması kolaydır.
- ✓ **Yerinde sıralama:** Ekstra belleğe ihtiyaç duymaz.
- ✓ **Neredeyse sıralı dizilerde hızlı olabilir:** En iyi durumda  $O(n)$ .

### Dezavantajları

- ✗ **Verimsiz:** Büyük veri kümelerinde yavaş çalışır ( $O(n^2)$ ).
- ✗ **Profesyonel uygulamalarda tercih edilmez:** Daha hızlı sıralama algoritmaları vardır (Quick Sort, Merge Sort gibi).



# Sıralama Algoritmaları

## Selection Sort Algoritması

**Selection Sort (Seçmeli Sıralama)**, sıralama algoritmalarından biridir ve temel olarak dizinin her turda en küçük (veya en büyük) elemanını seçerek sıralama işlemini gerçekleştirir.

### Çalışma Prensipleri

1. Dizinin en küçük elemanını bul.
2. Bulduğun en küçük elemanı, dizinin başındaki ile yer değiştir.
3. Sıralanmış kısmı genişletip kalan kısmı tekrar işle.
4. Tüm elemanlar sıralanana kadar bu işlemi devam ettir.

### Örnek

Varsayalım ki sıralamak istediğimiz liste şu şekilde olsun:  
[5, 3, 8, 4, 2]

#### 1. Adım

- En küçük eleman 2.
- 2 ile 5 yer değiştirir → [2, 3, 8, 4, 5]

#### 2. Adım

- 3 zaten doğru yerde → [2, 3, 8, 4, 5]

#### 3. Adım

- 4 en küçük eleman → 4 ile 8 yer değiştirir → [2, 3, 4, 8, 5]

#### 4. Adım

- 5 en küçük eleman → 5 ile 8 yer değiştirir → [2, 3, 4, 5, 8]

Bu işlem tamamlandığında liste sıralanmış olur.

### Zaman Karmaşıklığı

En kötü, en iyi ve ortalama durum:  $O(n^2)$

✓ **Avantajı:** Yer değiştirme sayısı azdır.

✗ **Dezavantajı:** Büyük listelerde verimli değildir.

## Selection sort

8, 5, 1, 3, 7

1, 5, 8, 3, 7

1, 3, 8, 5, 7

1, 3, 5, 8, 7

1, 3, 5, 7, 8



# Sıralama Algoritmaları

## Insertion Sort Algoritması

**Insertion Sort**, diziyi sıralamak için kullanılan basit bir sıralama algoritmasıdır. Bu algoritma, her bir öğeyi doğru yerine yerleştirerek diziyi sıralar.

### Algoritmanın Çalışma Prensibi

- Başlangıç Durumu:** Algoritma, dizinin ikinci elemanından başlar (ilk eleman zaten sıralı kabul edilir).
- İç İçe Döngü:** Seçilen öğeyi (şu anki öge) sıralı kısmın içine doğru karşılaştırarak yerleştirir. Bunu, öğeyi doğru pozisyona yerleştirmek için yapar.
- Yerleştirme:** Eğer şu anki öge, sıralı kısımdaki öğeden küçükse, sıralı kısımdaki öge bir adım sağa kaydırılır ve şu anki öge doğru pozisyona yerleştirilir.
- Tekrar:** Algoritma, dizinin sonuna kadar devam eder ve her öğeyi doğru pozisyona yerleştirir.

### Adım Adım Örnek

Örnek dizi: [5, 2, 9, 1, 5, 6]

#### 1. Adım

- İlk iki eleman: [5, 2]
- 2, 5'ten küçük olduğu için 5 sağa kaydırılır ve 2 sağındaki pozisyona yerleşir.
- Yeni dizi: [2, 5, 9, 1, 5, 6]

#### 2. Adım

- Üçüncü eleman: [2, 5, 9]
- 9, zaten doğru pozisyonda olduğu için hiçbir değişiklik yapılmaz.
- Yeni dizi: [2, 5, 9, 1, 5, 6]

#### 3. Adım

- Dördüncü eleman: [2, 5, 9, 1]
- 1, sıralı kısımdan küçük olduğu için sıralı kısımdaki tüm öğeler sağa kaydırılır ve 1 en başa yerleşir.
- Yeni dizi: [1, 2, 5, 9, 5, 6]

#### 4. Adım

- Beşinci eleman: [1, 2, 5, 9, 5]
- 5, sıralı kısımdaki öğelerle karşılaştırılır ve doğru pozisyona yerleşir.
- Yeni dizi: [1, 2, 5, 5, 9, 6]

#### 5. Adım

- Son eleman: [1, 2, 5, 5, 9, 6]
- 6, sıralı kısımdaki öğelerle karşılaştırılır ve doğru pozisyona yerleşir.
- Yeni dizi: [1, 2, 5, 5, 6, 9]

### Zaman Karmaşıklığı

- En İyi Durum (Dizi zaten sıralı):  $O(n)$
- Ortalama ve En Kötü Durum (Dizi ters sıralı):  $O(n^2)$

### Avantajları

- ✓ Basit ve anlaşılır bir algoritmadır.
- ✓ Küçük diziler için oldukça verimlidir.

### Dezavantajları

- ✗ Büyük dizilerde verimsizdir ( $O(n^2)$ ).



# Sıralama Algoritmaları

## Merge Sort Algoritması

**Merge Sort**, böl ve fethet (divide and conquer) yaklaşımına dayalı bir sıralama algoritmasıdır. Bu algoritma, büyük bir problemi daha küçük parçalara ayırarak çözmeyi hedefler. Önce diziyi daha küçük alt dizilere böler, ardından bu alt dizileri sıralar ve nihayetinde sıralı alt dizileri birleştirir (merge eder).

### Algoritmanın Çalışma Prensipleri

#### 1. Bölme (Divide):

- Dizi ortadan ikiye bölünür.
- Bu işlem, dizi tek elemanlı parçalara ayrılana kadar devam eder.

#### 2. Sıralama (Conquer):

- Bölünen her bir parça tek elemanlı hale gelene kadar devam edilir.
- Tek elemanlı diziler zaten sıralıdır, bu yüzden sıradaki adım, bu dizileri birleştirerek sıralı hale getirmektir.

#### 3. Birleştirme (Merge):

- İki sıralı dizi, her iki dizinin en küçük elemanlarını karşılaştırarak sıralı bir şekilde birleştirilir.
- Bu işlem, tüm alt diziler birleşene kadar devam eder.

### Adım Adım Örnek

Örnek dizi: [38, 27, 43, 3, 9, 82, 10]

#### 1. Bölme:

- İlk olarak diziyi iki parçaya ayırırız: [38, 27, 43] ve [3, 9, 82, 10].
- Bu parçalar daha da bölünür: [38], [27, 43] ve [3, 9], [82, 10] gibi.

#### 2. Sıralama ve Birleştirme:

- Ardından her iki küçük dizi sıralanır ve birleştirilir:
  - [27, 43] sıralanır ve birleşir: [27, 43]
  - [3, 9] sıralanır ve birleşir: [3, 9]
  - [82, 10] sıralanır ve birleşir: [10, 82]
- Son olarak, bu sıralı diziler birleştirilir: [3, 9, 10, 27, 38, 43, 82]

| Durum          | Zaman Karmaşıklığı |
|----------------|--------------------|
| En İyi Durum   | $O(n \log n)$      |
| Ortalama Durum | $O(n \log n)$      |
| En Kötü Durum  | $O(n \log n)$      |

**Merge Sort**, her durumda  $O(n \log n)$  zaman karmaşıklığına sahiptir. Bu nedenle, büyük veri kümeleri için oldukça verimlidir.

### Avantajları

- ✓ Kararlı bir sıralama algoritmasıdır (eşit elemanlar sıralandıktan sonra aynı sıradadır).
- ✓ Büyük dizilerde hızlıdır ve  $O(n \log n)$  karmaşıklığı ile etkili çalışır.

### Dezavantajları

- ✗ Ekstra bellek kullanır: Diziyi bölerken ve birleştirirken ek bellek gereksinimi vardır.



# Sıralama Algoritmaları

## Quick Sort Algoritması

**Quick Sort**, hızlı bir sıralama algoritmasıdır ve böl ve fethet (divide and conquer) yaklaşımını kullanır. Hedefi, diziyi hızlı bir şekilde sıralamaktır. Algoritma, büyük bir diziyi daha küçük parçalara ayırarak çözüm üretir. Bu, genellikle ortalama  $O(n \log n)$  zaman karmaşıklığına sahiptir.

### Algoritmanın Çalışma Prensipleri

#### 1. Pivot Seçimi (Partitioning):

- Öncelikle, diziden bir "pivot" seçilir. Pivot, sıralamayı yapmak için dizinin bir elemanıdır.
- Bu pivot etrafında tüm diziyi iki alt diziyeye böleriz:
  - Pivot'tan küçük olanlar bir diziyeye, büyük olanlar diğer diziyeye yerleştirilir.
  - Pivot, sıralandıktan sonra doğru yerinde olur.

#### 2. Alt Dizilerin Sıralanması:

- Daha sonra, pivot'un her iki tarafındaki alt diziler için aynı işlemler (pivot seçme ve bölme) tekrarlanır.
- Bu işlem, her bir alt dizi tek elemanlı hale gelene kadar devam eder.

#### 3. Birleştirme:

- Alt diziler sıralandıktan sonra, tek tek birleştirilir ve nihai sıralı dizi elde edilir.

### Adım Adım Örnek

Örnek dizi: [10, 80, 30, 90, 40, 50, 70]

#### 1. Pivot Seçimi ve Bölme:

- İlk adımda, pivot olarak 70 seçilir.
- 70'ten küçük olan elemanlar: [10, 30, 40, 50]
- 70'ten büyük olan elemanlar: [80, 90]
- Pivot yerinde: [10, 30, 40, 50, 70, 80, 90]

#### 2. Alt Dizilerin Sıralanması:

- Sol taraf: [10, 30, 40, 50] → Pivot 50 seçilir, sıralama yapılır.
- Sağ taraf: [80, 90] → Pivot 90 seçilir, sıralama yapılır.

#### 3. Sonuç:

- Nihai sıralı dizi: [10, 30, 40, 50, 70, 80, 90]

**Quick Sort**, en kötü durumda  $O(n^2)$  zaman karmaşıklığına sahip olabilir ancak genellikle  $O(n \log n)$  performans gösterir.

### Avantajları

- ✓ **Hızlıdır:** Ortalama durumda  $O(n \log n)$  zaman karmaşıklığıyla oldukça hızlıdır.
- ✓ **Yerinde sıralama yapar:** Ekstra bellek kullanmaz.

### Dezavantajları

- ✗ **Kötü durumda performans düşebilir:** Eğer pivot seçimi kötü yapılırsa,  $O(n^2)$  zaman karmaşıklığına düşebilir.
- ✗ **İstikrarsız sıralama algoritmasıdır:** Aynı değere sahip elemanların sırası değişebilir.



# Sıralama Algoritmaları

## Heap Sort

**Heap Sort**, sıralama yapmak için heap veri yapısını kullanan bir sıralama algoritmasıdır. Heap, özel bir tür ikili ağaçtır ve her bir düğümün değeri, çocuk düğümlerinin değerlerinden büyüktür (maksimum heap) ya da küçük olmalıdır (minimum heap). Heap Sort, büyük bir diziyi sıralamak için bu yapıyı kullanarak verimli bir sıralama sağlar.

### Algoritmanın Çalışma Prensipleri

#### 1. Heap Yapısını Kurma:

- İlk adımda, verilen dizi bir heap yapısına dönüştürülür. Bu adımda, diziyi bir maksimum heap veya minimum heap yapısına dönüştürürüz.
- Maksimum heap: Her düğümün değeri, çocuklarının değerlerinden büyük ya da eşit olmalıdır.
- Minimum heap: Her düğümün değeri, çocuklarının değerlerinden küçük ya da eşit olmalıdır.

#### 2. Heap'ten Eleman Çıkarma:

- Heap yapısına dönüştürülmüş diziden en büyük (veya en küçük) eleman çıkarılır.
- En büyük eleman heap'in kökünde bulunur, bu eleman dizinin sonuna yerleştirilir.

#### 3. Heap Yeniden Düzenleme:

- Heap yapısındaki bozulmayı gidermek için, kalan elemanlar yeniden heap düzenine sokulur.

#### 4. Tekrarlama:

- Bu işlem, tüm elemanlar sıralanana kadar devam eder.

### Adım Adım Örnek

Dizi: [4, 10, 3, 5, 1]

#### 1. Heap Yapısına Dönüştürme (Maksimum Heap):

- İlk olarak, dizi bir maksimum heap yapısına dönüştürülür. Dizi şu şekilde olur: [10, 5, 3, 4, 1]

#### 2. En Büyük Elemanı Çıkarma:

- En büyük eleman (10) kökten çıkarılır ve dizinin sonuna yerleştirilir: [1, 5, 3, 4]
- Heap düzeni yeniden sağlanır: [5, 4, 3, 1]

#### 3. Tekrarlama:

- Yeniden en büyük eleman çıkarılır, dizinin sonuna yerleştirilir ve heap yapısı tekrar sağlanır: [4, 1, 3]
- Son adımda, sıralı dizi elde edilir: [1, 3, 4, 5, 10]

**Heap Sort**, her durumda  $O(n \log n)$  zaman karmaşıklığına sahiptir. Bu nedenle, özellikle yavaş bellek alanlarında ve sıralama sırasında ek bellek kullanımının düşük olmasının önemli olduğu durumlarda oldukça verimlidir.

### Avantajları

- ✓ **Zaman karmaşıklığı:** Heap Sort her durumda  $O(n \log n)$  performans gösterir.
- ✓ **Yerinde sıralama:** Ekstra bellek kullanmaz. Bellek karmaşıklığı  $O(1)$ 'dir.
- ✓ **Stabil sıralama değildir:** Aynı elemanlar sıralandıktan sonra sırası değişebilir.

### Dezavantajları

- ✗ **İstikrarsız sıralama:** Heap Sort, kararlı (stable) bir sıralama algoritması değildir, yani aynı değere sahip elemanların sırası değişebilir.
- ✗ **Hızlı değil:** Diğer sıralama algoritmaları (örneğin Quick Sort) genellikle daha hızlıdır, çünkü heap yapılarını kurma ve yeniden düzenleme işlemleri zaman alabilir.



# Sıralama Algoritmaları

## Radix Sort

**Radix Sort**, sıralama algoritmaları arasında yer alan, sayıları rakamlarına göre sıralayarak çalışan bir algoritmadır. Bu algoritma, her bir sayının basamağını dikkate alarak sıralama işlemi yapar. Genellikle tamsayı dizilerini sıralamak için kullanılır ve özellikle sayılar büyükse veya belirli bir sayıya odaklanılmışsa oldukça etkilidir.

### Algoritmanın Çalışma Prensibi

Radix Sort, sıralama işlemini basamakasal (digit-by-digit) yapar ve bu işlem şu adımlarla gerçekleşir:

#### 1. En Düşük Basamaktan Başlamak:

- İlk olarak, dizinin her bir elemanını en düşük basamaktan (genellikle birler basamağından) başlayarak sıralarız.
- Örneğin, 123 ve 456 sayılarının birler basamağına bakılır (yani, 3 ve 6).

#### 2. Sonraki Basamaklara Geçiş:

- İlk basamaktan sıralama işlemi yapıldıktan sonra, sıradaki basamağa (onlar basamağı, yüzler basamağı vb.) geçilir ve her basamağa göre sıralama yapılır.

#### 3. Sıralama İşlemi:

- Bu işlem, sayının en yüksek basamağına kadar tekrarlanır.

#### 4. Diziyi Tamamlama:

- Sonuçta, tüm basamaklarda sıralama yapılmış ve dizinin tamamı sıralanmış olur.

### Adım Adım Örnek

Dizi: [170, 45, 75, 90, 802, 24, 2, 66]

#### • Birler Basamağına Göre Sıralama:

- [170, 90, 802, 2, 24, 45, 75, 66] (Birler basamağı: 0, 0, 2, 2, 4, 5, 5, 6)

#### • Onlar Basamağına Göre Sıralama:

- [170, 802, 2, 24, 45, 66, 75, 90] (Onlar basamağı: 7, 0, 0, 2, 4, 6, 7, 9)

#### • Yüzler Basamağına Göre Sıralama:

- [2, 24, 45, 66, 75, 90, 170, 802] (Yüzler basamağı: 0, 0, 0, 0, 0, 0, 1, 8)

#### • Sonuç:

- Sonuç olarak sıralı dizi: [2, 24, 45, 66, 75, 90, 170, 802]

### Zaman Karmaşıklığı

- En İyi Durum:  $O(nk)$
- Ortalama Durum:  $O(nk)$
- En Kötü Durum:  $O(nk)$

### Burada:

- n: Dizinin eleman sayısı
- k: Sayıların basamak sayısıdır.



# Sıralama Algoritmaları

## Avantajları

- ✓ **Hızlıdır:** Özellikle belirli bir basamağa göre sıralama yapılacağı için çok büyük sayıların sıralanmasında verimli olabilir.
- ✓ **Kararlı:** Aynı değere sahip elemanların sırası korunur.
- ✓ **Doğrudan Sıralama:** Sayıların basamaklarına göre sıralama yapıldığı için ekstra bir karşılaştırma gerekmez.

## Dezavantajları

- ✗ **Sadece Sayılar İçin Uygunudur:** Radix Sort genellikle sayılarla çalışır, bu yüzden diğer veri türleri için kullanılamaz.
- ✗ **Bellek Kullanımı:** Sıralama için ekstra bir bellek alanı gerekir, bu da çok büyük verilerle çalışırken problem yaratabilir.

## Kullanım Alanları

- **Dijital verilerin sıralanması:** Özellikle sayılar, tarihsel veriler gibi sıralanabilir büyük veri kümeleriyle çalışırken kullanılır.
- **Sayısal verilerin hızlı sıralanması:** Rakamlar üzerinde işlem yapan algoritmalar için uygundur.

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 0 | 0 | 1 |
| 4 | 3 | 2 |
| 0 | 2 | 3 |
| 5 | 6 | 4 |
| 0 | 4 | 5 |
| 7 | 8 | 8 |

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 2 | 1 |
| 0 | 2 | 3 |
| 4 | 3 | 2 |
| 0 | 4 | 5 |
| 5 | 6 | 4 |
| 7 | 8 | 8 |

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 2 | 3 |
| 0 | 4 | 5 |
| 1 | 2 | 1 |
| 4 | 3 | 2 |
| 5 | 6 | 4 |
| 7 | 8 | 8 |



# Sıralama Algoritmaları

## Counting Sort

**Counting Sort**, sayılar arasındaki frekanslara dayalı çalışan bir sıralama algoritmasıdır. Bu algoritma, dizideki her bir elemanın frekansını (kaç kere tekrar ettiğini) sayar ve bu bilgiyi kullanarak elemanları sıralar. Counting Sort, genellikle pozitif tam sayılar için verimli bir sıralama algoritmasıdır.

### Algoritmanın Çalışma Prensipleri

Counting Sort, genellikle şu adımlarla çalışır:

#### 1. Frekans Dizisini Oluşturma:

- İlk olarak, sıralanacak dizinin en küçük ve en büyük elemanlarını buluruz.
- Bu aralık içinde her bir sayının kaç kez tekrar ettiğini tutacak bir "frekans dizisi" oluştururuz.

#### 2. Frekansları Sayma:

- Dizideki her bir elemanın frekansını sayarız. Yani, her bir elemanın ne kadar tekrar ettiğini buluruz.

#### 3. Toplamları Hesaplama:

- Frekans dizisini, her bir sayının sıralanmış dizide hangi pozisyonda olması gerektiğini belirleyecek şekilde güncelleriz.

#### 4. Sonuç Dizisini Oluşturma:

- Son olarak, bu güncellenmiş frekans dizisini kullanarak sıralı diziyi oluştururuz.

### Adım Adım Örnek

Dizi: [4, 2, 2, 8, 3, 3, 1]

#### 1. Frekans Dizisini Oluşturma:

- Frekans dizisini oluşturmak için dizinin her elemanını sayıyoruz:
  - 1: 1 kez
  - 2: 2 kez
  - 3: 2 kez
  - 4: 1 kez
  - 8: 1 kez

#### 2. Frekansları Sayma:

- Frekans dizisini oluşturuyoruz:
  - [0, 1, 2, 2, 1, 0, 0, 0, 1] (0'dan 8'e kadar)

#### 3. Toplamları Hesaplama:

- Bu diziyi güncelliyoruz. Her bir eleman, bu elemanın ve önceki elemanların toplamını tutacak şekilde güncellenir:
  - [0, 1, 3, 5, 6, 6, 6, 6, 7]

#### 4. Sonuç Dizisini Oluşturma:

- Bu güncellenmiş diziyi kullanarak sıralı dizi oluşturulacak:
  - Sonuç: [1, 2, 2, 3, 3, 4, 8]



# Sıralama Algoritmaları

## Zaman Karmaşıklığı

- En İyi Durum:  $O(n + k)$
- Ortalama Durum:  $O(n + k)$
- En Kötü Durum:  $O(n + k)$

## Burada:

- $n$ : Dizinin eleman sayısı
- $k$ : Dizideki en büyük elemanın değeri (yani sıralanacak sayılar arasındaki aralık)

## Avantajları

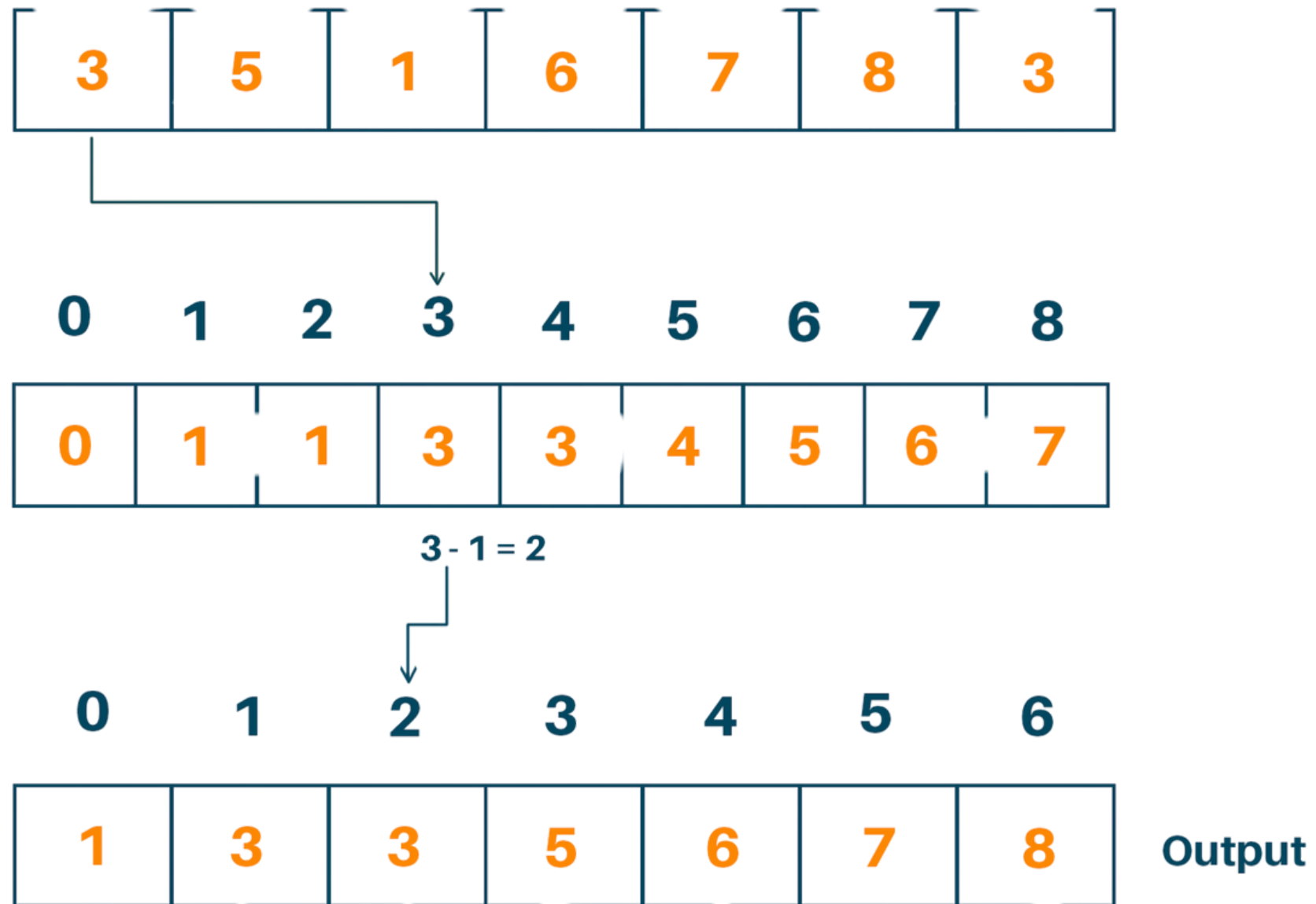
- ✓ **Hızlıdır:** Özellikle belirli bir aralıktaki sayılarla çalışırken çok hızlıdır. Eğer aralıktaki sayıların sayısı  $n$ 'den küçükse, çok verimlidir.
- ✓ **Kararlı:** Aynı değere sahip elemanların sırası korunur.
- ✓ **Karşılaştırmasız:** Sayılar doğrudan karşılaştırılmadan sıralanır.

## Dezavantajları

- ✗ **Sadece Pozitif Sayılar İçin Uygundur:** Counting Sort, negatif sayılar ve genel olarak karmaşık veri türleriyle çalışmaz.
- ✗ **Büyük Aralıklar:** Eğer dizideki elemanlar çok büyükse, frekans dizisinin boyutu çok büyük olabilir, bu da bellek kullanımını artırır.

## Kullanım Alanları

- **Sayılar Arasında Sıralama:** Pozitif tamsayılar arasında hızlı sıralama yapmayı gerektiren durumlarda kullanılır.
- **Dijital Veriler:** Belirli bir aralıktaki sayıları hızlıca sıralamak için kullanılır





# Sıralama Algoritmaları

## Bucket Sort

**Bucket Sort**, sıralama algoritmalarından biridir ve genellikle sayılar belirli bir aralıkta ve birbirine yakın olduğunda oldukça verimlidir. Bu algoritma, diziyi belirli aralıklara (veya "kovalara") ayırır, ardından her kova içindeki elemanları sıralar ve tüm kova elemanları birleştirilerek sıralı bir dizi elde edilir.

### Algoritmanın Çalışma Prensibi

#### 1. Kovaları (Buckets) Oluşturma:

- İlk olarak, sıralanacak elemanların değerlerine göre bir dizi kova oluşturulur.
- Bu kovalar, genellikle elemanların değeri belirli bir aralığa yayılacak şekilde tasarlanır. Kovaların sayısı, genellikle dizinin eleman sayısına eşit ya da biraz daha fazla olabilir.

#### 2. Elemanları Kovalarına Dağıtma:

- Her bir eleman, değeri doğrultusunda uygun kovanın içine yerleştirilir.

#### 3. Her Kovada Sıralama Yapma:

- Kovaların içine yerleştirilen elemanlar, genellikle hızlı sıralama algoritmaları (örneğin, Insertion Sort) ile sıralanır.

#### 4. Tüm Kovaları Birleştirme:

- Son olarak, sıralanan kovalar birleştirilir ve sıralı bir dizi elde edilir.

### Adım Adım Örnek

Dizi: [0.42, 0.32, 0.23, 0.51, 0.12, 0.33]

#### 1. Kovaları Oluşturma:

- Verilen aralık (0 ile 1 arasında) dikkate alınarak 5 tane kova oluşturulabilir.

#### 2. Elemanları Kovalarına Dağıtma:

- Elemanlar şu şekilde dağıtılabilir:
  - Kova 1: [0.12]
  - Kova 2: [0.23]
  - Kova 3: [0.32, 0.33]
  - Kova 4: [0.42]
  - Kova 5: [0.51]

#### 3. Her Kovada Sıralama:

- Her kova, kendi içinde sıralanır:
  - Kova 1: [0.12]
  - Kova 2: [0.23]
  - Kova 3: [0.32, 0.33]
  - Kova 4: [0.42]
  - Kova 5: [0.51]

#### 4. Kovaları Birleştirme:

- Son olarak, sıralı kovalar birleştirilir:
  - [0.12, 0.23, 0.32, 0.33, 0.42, 0.51]



# Sıralama Algoritmaları

## Zaman Karmaşıklığı

- En İyi Durum:  $O(n + k)$
- Ortalama Durum:  $O(n + k)$
- En Kötü Durum:  $O(n^2)$ , eğer her kova tek bir eleman içerirse ve kovalar arasında sıralama yapılıyorsa.

## Burada:

- $n$ : Dizinin eleman sayısı
- $k$ : Kovaların sayısı (veya dizinin aralığı)

## Avantajları

- ✓ **Verimli Aralıklar İçin:** Eğer dizinin elemanları belirli bir aralıkta yoğunlaşıyorsa, Bucket Sort çok verimli olabilir.
- ✓ **Sıralama Süresi:** Kovalar arasında sıralama yapılmadığı takdirde, algoritmanın çalışma süresi çok hızlı olabilir.

## Dezavantajları

- ✗ **Düşük Performans:** Eğer dizinin elemanları çok dağılmışsa, Bucket Sort çok verimli olmayabilir.
- ✗ **Bellek Kullanımı:** Kovaların saklanması için ek bellek kullanımı gerektirir.

## Kullanım Alanları

- **Sayılarla Dayalı Sıralamalar:** Özellikle belirli bir aralıktaki sayılar için uygundur.
- **Karmaşık Sayılarla Çalışma:** Eğer sayıların yoğunlukları belirli bir aralıktaysa, Bucket Sort algoritması kullanılabilir.





# Arama Algoritmaları

## Linear Search Algoritması

### Linear Search Nedir?

**Linear Search (Doğrusal Arama)**, en basit arama algoritmalarından biridir. Bir dizide (veya listede) bir elemanı bulmak için dizinin başından sonuna kadar tek tek elemanlar kontrol edilir. Eğer aranan eleman bulunursa, ilgili indeks döndürür. Bulunamazsa genellikle -1 değeri veya "Bulunamadı" mesajı döndürülür.

### Çalışma Mantığı

1. Dizinin ilk elemanından başlanır.
2. Aranan eleman, her bir elemanla karşılaştırılır.
3. Eğer eşleşme bulunursa, indeks döndürülür.
4. Tüm elemanlar kontrol edilmesine rağmen bulunamazsa, -1 döndürülür.

### Zaman Karmaşıklığı

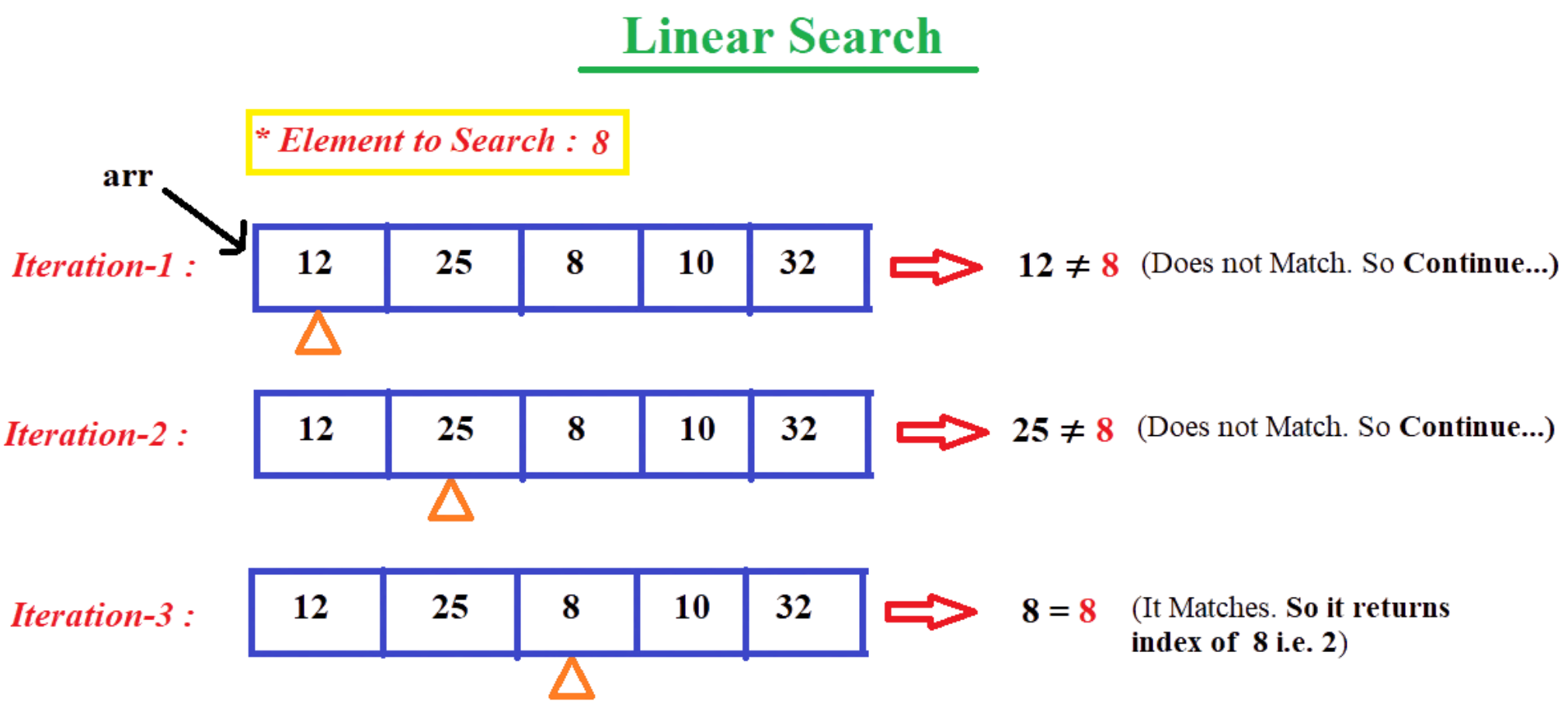
- En kötü durumda:  $O(n)$  → Eleman listenin sonunda ya da yoksa tüm liste taranır.
- En iyi durumda:  $O(1)$  → Eleman ilk sıradaysa hemen bulunur.
- Ortalama durumda:  $O(n)$  → Eleman rastgele bir yerdeyse genellikle yarıya kadar tarama yapılır.

### Avantajları

- ✓ Basit ve anlaşılırdır.
- ✓ Küçük listeler için idealdir.
- ✓ Sıralı olma zorunluluğu yoktur.

### Dezavantajları

- ✗ Büyük veri kümelerinde yavaştır.
- ✗ Daha verimli algoritmalar (Binary Search gibi) varken genellikle tercih edilmez.





# Arama Algoritmaları

## İkili Arama (Binary Search) Algoritması

### Binary Search Nedir?

**İkili Arama (Binary Search)**, sıralı bir listede bir elemanı hızlı bir şekilde bulmak için kullanılan verimli bir arama algoritmasıdır. Doğrusal Arama (Linear Search) gibi tüm elemanları tek tek kontrol etmek yerine, böl ve fethet yaklaşımını kullanır.

### Çalışma Mantığı

1. Liste, küçükten büyüğe sıralı olmalıdır.
2. Listenin ortasındaki eleman bulunur.
3. Aranan eleman ile orta eleman karşılaştırılır:
  - Eğer eşitse, indeks döndürülür.
  - Eğer aranan eleman küçükse, sol yarıda arama yapılır.
  - Eğer aranan eleman büyükse, sağ yarıda arama yapılır.
4. Eleman bulunana veya arama aralığı boşalana kadar bu işlem tekrarlanır.

### Zaman Karmaşıklığı

- En kötü durumda:  $O(\log n)$  → Arama alanı her adımda yarıya iner.
- En iyi durumda:  $O(1)$  → Orta eleman doğrudan aranan eleman olabilir.
- Ortalama durumda:  $O(\log n)$  → Büyük veri kümeleri için oldukça verimlidir.

### Avantajları

- ✓ Büyük listelerde Doğrusal Arama'ya göre çok daha hızlıdır.
- ✓ Sıralı verilerle verimli çalışır.
- ✓ Karşılaştırma sayısını önemli ölçüde azaltır.

### Dezavantajları

- ✗ Arama yapmadan önce listenin sıralı olması gerekir.
- ✗ Küçük veri kümeleri için Doğrusal Arama'ya kıyasla daha az avantajlıdır.

### Kullanım Alanları

- **Veritabanı Aramaları:** Sıralı verilerde hızlı arama yapmak için kullanılır.
- **Arama Motorları:** Web aramalarında sıralı veriler üzerinde hızlı sonuçlar elde edilir.
- **Binary Search Tree (BST):** İkili arama ağaçlarında verileri sıralı tutarak hızlı arama yapılır.
- **Bilgisayar Grafikleri:** Sıralı piksellerde hızlı arama işlemleri için kullanılır.
- **Oyunlar:** Oyuncu seviyeleri sıralandığında eşleşme yaparken hızlı arama sağlar.
- **Zaman Serisi Verileri:** Sıralı zaman dilimlerinde veri aramaları yapılır.
- **Dosya Sistemleri:** Sıralı dosya listelerinde hızlı dosya aramaları yapılır.
- **Yapay Zeka:** Hiperparametre optimizasyonunda sıralı parametrelerde hızlı arama yapılır.



# Arama Algoritmaları

## Jump Search Algoritması

### Jump Search Nedir?

**Jump Search**, sıralı bir listede bir elemanı aramak için kullanılan bir algoritmadır. Bu algoritma, belirli bir adım büyüklüğü ile ilerleyerek, her adımda elemanın bulunup bulunmadığını kontrol eder. Eğer eleman adım içinde bulunmazsa, bir sonraki adıma geçilir ve işlem bu şekilde devam eder.

### Çalışma Mantığı

1. Liste sıralı olmalıdır.
2. Bir adım büyüklüğü  $k$  (genellikle listenin karekökü kadar) belirlenir.
3. Listenin başından başlayarak,  $k$  adım ilerlenir.
4. Hedef elemanın şu anki blokta olup olmadığı kontrol edilir.
5. Eğer eleman blok içinde bulunmazsa, bir sonraki bloğa geçmek için  $k$  adım daha ilerlenir.
6. Eğer eleman bulunduysa, o blokta doğrusal arama yapılır ve kesin pozisyon bulunur.

### Zaman Karmaşıklığı

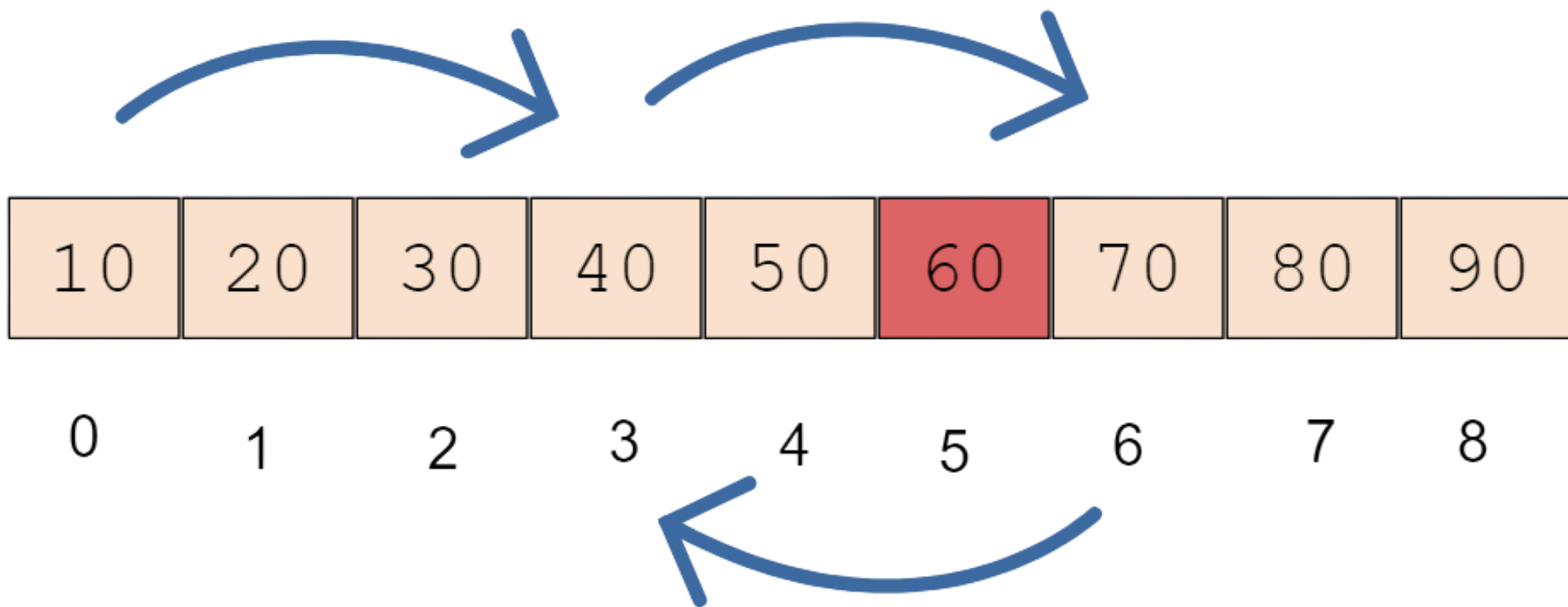
- En kötü durumda:  $O(\sqrt{n})$  → Algoritma adım adım ilerler, Linear Search'ten daha az karşılaştırma yapar.
- En iyi durumda:  $O(1)$  → Hedef eleman ilk adımdan hemen bulunabilir.
- Ortalama durumda:  $O(\sqrt{n})$  → Genellikle büyük veri kümelerinde Linear Search'ten daha hızlıdır.

### Avantajları

- ✓ Linear Search'e göre büyük veri kümelerinde daha hızlıdır.
- ✓ Sıralı veriyle verimli çalışır.
- ✓ Bloklar halinde ilerleyerek karşılaştırma sayısını azaltır.

### Dezavantajları

- ✗ Liste sıralı olmalıdır.
- ✗ Veri kümesindeki blok sayısı az ise Binary Search kadar verimli olmayabilir.





# Arama Algoritmaları

## Exponential Search Algoritması

**Exponential Search (Üssel Arama)**, sıralı bir dizide hızlı bir şekilde arama yapmak için kullanılan bir algoritmadır. Bu algoritma, klasik binary search (ikili arama) algoritmasına benzer, ancak bir farkı vardır. İlk olarak diziyi daha büyük aralıklarla tarar ve ardından ikili arama kullanarak arama işlemini tamamlar.

### Algoritmanın Mantığı

#### 1. Başlangıç Adımı:

- Dizinin ilk elemanını kontrol ederiz. Eğer aradığımız değer ilk elemanla eşleşiyorsa, direkt olarak bulmuş oluruz.

#### 2. Aralık Büyütme:

- Eğer ilk eleman aradığımız değeri karşılamıyorsa, dizinin arama alanını iki katına çıkararak aramaya devam ederiz. İlk olarak 1. elemandan başlarız, sonra 2. elemana, sonra 4. elemana ve bu şekilde devam ederiz. Aradığımız değeri içerebilecek bir aralık bulunana kadar bu şekilde ilerleriz.

#### 3. Binary Search (İkili Arama):

- Arama alanını büyüttükten sonra, bulduğumuz aralıkta klasik binary search (ikili arama) uygulanır. Binary search ile arama, arama alanını ikiye böler ve daha hızlı bir şekilde değeri bulmamızı sağlar.

### Örnek Akış

Diyelim ki elimizde şu sıralı dizi var:

[1, 3, 5, 7, 9, 11, 15, 19, 25, 30]

- İlk olarak, dizinin ilk elemanına bakarız: 1. Aradığımız değer 15 olduğunda, 1 ile eşleşmediği için arama devam eder.
- Daha sonra, 1'in iki katı kadar (2. eleman, 4. eleman...) gideriz. Bu şekilde arama alanı genişler.
- Arama alanı genişledikten sonra, aradığımız değeri bulduğumuz bir aralık belirlendiğinde, bu aralık içinde klasik binary search kullanarak daha hızlı bir şekilde arama yapılır.

### Avantajları

✅ **Daha Hızlı:** Özellikle çok büyük dizilerde daha hızlı sonuçlar verebilir çünkü ilk başta arama alanını hızla büyütür.

✅ **Binary Search Kullanımı:** Binary search uygulandığı için, arama sonunda verimli bir şekilde sonuç alınır.

### Zaman Karmaşıklığı

- Best Case:  $O(\log n)$
- Worst Case:  $O(\log n)$

Bu, **Exponential Search** algoritmasının temel mantığıdır. Kullanıldığında büyük dizilerde verimli sonuçlar verir.



# Veri Yapıları Algoritmaları

## Linked List Nedir?

**Linked List**, her öğesinin bir veri ve bir sonraki öğeyi gösteren işaretçiye sahip olduğu dinamik bir veri yapısıdır. Bu yapı, dizi (array) gibi statik yapılara alternatif olarak, belleği verimli kullanmak amacıyla geliştirilmiştir.

### Linked List Temel Elemanları

#### 1. Node (Düğüm):

- Veri (Data): Düğümün taşıdığı veri.
- Bağlantı (Next): Bir sonraki düğümü işaret eden gösterici.

#### 2. Head (Başlangıç):

- Listenin ilk düğümünü işaret eden gösterici.

#### 3. Tail (Son):

- Genellikle listenin sonundaki düğüm.

### Linked List Türleri

- **Singly Linked List (Tek Yönlü Bağlantılı Liste):** Her node yalnızca bir sonraki node'a işaret eder.
- **Doubly Linked List (Çift Yönlü Bağlantılı Liste):** Her node, hem bir sonraki hem de bir önceki node'a işaret eder.
- **Circular Linked List (Dairesel Bağlantılı Liste):** Son node, ilk node'a işaret eder.

### Linked List İşlemleri

#### 1. Ekleme (Insertion):

- Başta, ortada veya sonda yeni düğüm eklemek.

#### 2. Silme (Deletion):

- Belirli bir düğümü silmek.

#### 3. Arama (Search):

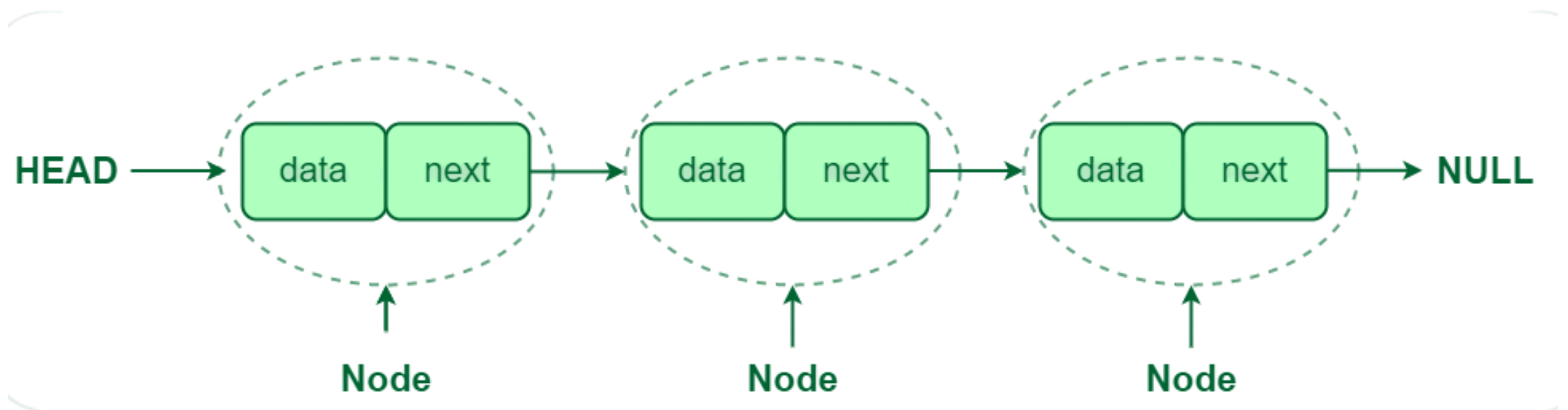
- Belirli bir değeri bulmak.

#### 4. Görüntüleme (Traversal):

- Listeyi gezmek.

### Sonuç

**Bağlantılı listeler**, dizilere kıyasla daha dinamik bir yapı sağlar. Listeye eleman eklemek, silmek ve listeyi gezmek gibi işlemler oldukça etkilidir. Ancak her düğümün işaretçi tutması gerektiği için bellek kullanımı daha yüksektir.





# Veri Yapıları Algoritmaları

## Stack (Yığın) Veri Yapısı

**Stack (Yığın)**, son giren ilk çıkar (LIFO - Last In, First Out) prensibiyle çalışan bir veri yapısıdır. Yani, en son eklenen öge, ilk olarak çıkarılır. Stack, tıpkı bir yığının üst kısmına eklenen veya çıkarılan öğeler gibi, yalnızca bir uçtan işlem yapılabilir.

### Stack'in Temel Elemanları

- 1.Push (Ekleme):**
  - Stack'e bir öge eklemek.
- 2.Pop (Çıkarmak):**
  - Stack'ten bir öge çıkarmak (en son eklenen ögeyi).
- 3.Peek (Görünüm):**
  - Stack'in en üstündeki ögeyi görmek, ancak çıkarmadan.
- 4.isEmpty (Boş Mu?):**
  - Stack'in boş olup olmadığını kontrol etmek.

### Kullanım Alanları

- Fonksiyonların geri dönüşü:** Programlama dillerinde fonksiyon çağrılarını ve geri dönüşleri stack veri yapısı ile yönetilir.
- Undo/Redo işlemleri:** Uygulamalarda yapılan işlemlerin geri alınıp yeniden yapılması işlemleri stack kullanılarak gerçekleştirilir.
- İfadelerin çözülmesi:** Matematiksel ifadelerin çözümünde (örneğin parantez denetimi) stack kullanılır.
- Web tarayıcı geçmişi:** Bir tarayıcıda gezinme geçmişi de stack veri yapısını kullanır.

### Stack'in Temel İşlemleri

- 1.Push:** Stack'e yeni bir öge ekler. Bu işlem, öge en üstte olacak şekilde yapılır.
- 2.Pop:** Stack'ten en üstteki ögeyi çıkarır ve bu öge geri döndürülür.
- 3.Peek:** Stack'in en üstündeki ögeyi döndürür ancak stack'ten çıkarmaz.
- 4.isEmpty:** Stack'in boş olup olmadığını kontrol eder.

### Sonuç

**Stack**, son giren ilk çıkar (LIFO) prensibine göre çalışan, çok kullanışlı bir veri yapısıdır. Fonksiyon çağrıları, undo/redo işlemleri gibi birçok alanda kullanılır. Stack işlemleri (push, pop, peek) oldukça hızlıdır ve bu veri yapısının dinamikliği sayesinde çeşitli problemleri çözmek kolaylaşır.



# Veri Yapıları Algoritmaları

## Queue (Kuyruk) Veri Yapısı

**Queue (Kuyruk)**, ilk giren ilk çıkar (FIFO - First In, First Out) prensibine göre çalışan bir veri yapısıdır. Yani, ilk eklenen öge, ilk olarak çıkarılır. Kuyruk, tıpkı bir kuyrukta sıraya giren insanların en önce sırada olan kişinin işlem görmesi gibi çalışır.

### Queue'in Temel Elemanları

- 1.Enqueue (Ekleme):**
  - Kuyruğa bir öge eklemek.
- 2.Dequeue (Çıkarmak):**
  - Kuyruktan bir öge çıkarmak (ilk eklenen öge çıkarılır).
- 3.Peek (Görünüm):**
  - Kuyruğun en önündeki ögeyi görmek, ancak çıkarmadan.
- 4.isEmpty (Boş Mu?):**
  - Kuyruğun boş olup olmadığını kontrol etmek.
- 5.Size (Boyut):**
  - Kuyruğun içindeki eleman sayısını döndürür.

### Kullanım Alanları

- **Yazıcı sıralamaları:** Yazıcıda belge basımı kuyruğa alınarak, ilk gelen belge ilk basılır.
- **İşlem sıralamaları:** Bilgisayar sistemlerinde işlemlerin sıralanması ve yönetilmesi.
- **Veri iletim sıraları:** Ağa bağlı sistemlerde veri iletim sıralamaları da kuyruğu kullanabilir.

### Queue'in Temel İşlemleri

- 1.Enqueue:** Kuyruğa yeni bir öge ekler. Bu işlem ögeyi kuyruğun sonuna ekler.
- 2.Dequeue:** Kuyruktan en öndeki ögeyi çıkarır ve bu öge geri döndürülür.
- 3.Peek:** Kuyruğun en önündeki ögeyi döndürür ancak kuyruktan çıkarmaz.
- 4.isEmpty:** Kuyruğun boş olup olmadığını kontrol eder.
- 5.Size:** Kuyruktaki eleman sayısını döndürür.

### Sonuç

**Queue**, ilk giren ilk çıkar (FIFO) prensibine dayalı olarak çalışan önemli bir veri yapısıdır. Kuyruk, verilerin sıralı bir şekilde işlendiği durumlarda ve çeşitli işlemler arasında düzen sağlamak için kullanılır. Kuyruk işlemleri (enqueue, dequeue, peek) genellikle hızlı ve etkilidir, bu nedenle birçok farklı uygulama ve sistemde yaygın olarak kullanılır.



# Veri Yapıları Algoritmaları

## Deque (Çift Yönlü Kuyruk) Veri Yapısı

**Deque (Double Ended Queue - Çift Yönlü Kuyruk)**, her iki uçtan da eleman ekleyip çıkarabileceğiniz bir veri yapısıdır. Yani, elemanlar hem kuyruğun önünden (front) hem de kuyruğun sonundan (back) eklenip çıkarılabilir.

**Deque, Queue ve Stack veri yapılarının birleşimi** gibi düşünülebilir. Queue'da yalnızca kuyruğun sonundan ekleme yapılırken, Deque'de hem baştan hem de sondan ekleme yapılabilir. Aynı şekilde, Stack gibi, Deque'de de elemanlar her iki uçtan da çıkarılabilir.

### Deque'in Temel Elemanları

- 1.Enqueue Front (Başından Ekleme):**
  - Deque'e kuyruğun önünden bir öge eklemek.
- 2.Enqueue Back (Sonundan Ekleme):**
  - Deque'e kuyruğun sonundan bir öge eklemek.
- 3.Dequeue Front (Başından Çıkarmak):**
  - Deque'den kuyruğun önündeki ögeyi çıkarmak.
- 4.Dequeue Back (Sonundan Çıkarmak):**
  - Deque'den kuyruğun sonundaki ögeyi çıkarmak.
- 5.Peek Front (Başındaki Ögeyi Görmek):**
  - Kuyruğun en önündeki ögeyi görmek.
- 6.Peek Back (Sonundaki Ögeyi Görmek):**
  - Kuyruğun en sonundaki ögeyi görmek.
- 7.isEmpty (Boş Mu?):**
  - Deque'in boş olup olmadığını kontrol etmek.

### Kullanım Alanları

- **Veri akış kontrolü:** Çift yönlü kuyruklar, veri akışının her iki yönde kontrol edilmesi gereken durumlarda kullanılır.
- **LIFO ve FIFO kombinasyonu:** Hem FIFO (ilk giren ilk çıkar) hem de LIFO (son giren ilk çıkar) özelliklerini bir arada bulundurması nedeniyle, birçok karmaşık veri işlemi için uygundur.
- **İç içe geçmiş menüler:** Çift yönlü kuyruklar, menülerde ileri ve geri navigasyon işlemlerinde de kullanılabilir.

### Deque'in Temel İşlemleri

- 1.Enqueue Front:** Deque'e kuyruğun önünden yeni bir öge ekler.
- 2.Enqueue Back:** Deque'e kuyruğun sonundan yeni bir öge ekler.
- 3.Dequeue Front:** Kuyruğun önündeki ögeyi çıkarır.
- 4.Dequeue Back:** Kuyruğun sonundaki ögeyi çıkarır.
- 5.Peek Front:** Kuyruğun en önündeki ögeyi görmenizi sağlar.
- 6.Peek Back:** Kuyruğun en sonundaki ögeyi görmenizi sağlar.
- 7.isEmpty:** Deque'in boş olup olmadığını kontrol eder.

### Sonuç

**Deque**, çift yönlü kuyruk olarak her iki uçtan da işlem yapabilmenizi sağlar. Bu özellik, çeşitli algoritmalarda daha esnek ve verimli çözümler üretmenizi mümkün kılar. FIFO ve LIFO özelliklerini aynı anda taşıması, Deque'in birçok farklı kullanım alanında etkili olmasını sağlar.



# Veri Yapıları Algoritmaları

## Hashing ve Hash Table Veri Yapıları

### Hashing Nedir?

**Hashing**, verileri bir anahtara (key) dönüştürmek için kullanılan bir tekniktir. Bu dönüşüm, genellikle bir hash fonksiyonu aracılığıyla yapılır. Hash fonksiyonu, bir veri ögesini (örneğin bir dize veya sayı) sabit boyutlu bir değere dönüştüren bir matematiksel işlemdir. Bu dönüşüm sonucu elde edilen değere "hash kodu" denir.

**Hashing**, verilerin hızlı bir şekilde depolanmasını ve erişilmesini sağlamak için kullanılır. Özellikle büyük veri setlerinde, belirli bir anahtara hızlıca erişim sağlamak amacıyla oldukça faydalıdır.

### Hashing Temel Kavramları

#### 1.Hash Fonksiyonu (Hash Function):

- Hash fonksiyonu, verileri bir anahtara dönüştüren matematiksel bir fonksiyondur. Bu fonksiyon, belirli bir veri girişi için her zaman aynı çıktıyı üretir.

#### 2.Hash Kodu (Hash Code):

- Hash fonksiyonundan elde edilen çıktıdır. Hash kodu, verinin tek bir temsilidir ve veri üzerinde yapılan işlemler sonucu her zaman aynı olmalıdır.

#### 3.Çakışma (Collision):

- Farklı veriler aynı hash kodunu üretirse, bu bir çakışma olarak kabul edilir. Çakışmalar hash tablosu tasarımında önemli bir problem oluşturur.

### Hash Table (Hash Tablosu) Nedir?

**Hash Table (Hash Tablosu)**, verilerin anahtar-değer çiftleri olarak saklandığı bir veri yapısıdır. Hash table, hashing tekniği kullanarak verileri hızlı bir şekilde depolar ve erişir. Anahtarlar, veriye erişim sağlamak için kullanılır. Bu anahtarlar, hash fonksiyonu aracılığıyla hash kodlarına dönüştürülür.

### Hash Table'in Temel Elemanları

#### 1.Anahtar (Key):

- Veriyi tanımlayan benzersiz bir değerdir. Anahtarlar hash fonksiyonu ile hash koduna dönüştürülür.

#### 2.Değer (Value):

- Anahtar ile ilişkilendirilmiş veridir. Anahtarın hash kodu kullanılarak değere hızlıca erişilebilir.

#### 3.Slot (Depolama Alanı):

- Hash tablosunda her anahtar-değer çiftinin depolandığı alandır. Hash fonksiyonu, anahtarı bir slot'a yerleştirir.



# Veri Yapıları Algoritmaları

## Hash Table Temel İşlemleri

- **Ekleme (Insert):**
  - Anahtar ve değeri hash tablosuna ekler. Anahtar hash fonksiyonundan geçirilerek uygun slot'a yerleştirilir.
- **Erişim (Search):**
  - Belirli bir anahtar ile ilişkili değeri hızlı bir şekilde arar ve bulur. Anahtar hash fonksiyonundan geçirilerek uygun slot'tan değere erişilir.
- **Silme (Delete):**
  - Belirli bir anahtar ile ilişkili değeri siler. Anahtarın hash kodu kullanılarak uygun slot'tan değer çıkarılır.
- **Çakışma Çözümü (Collision Resolution):**
  - Çakışma durumunda, birden fazla çözüm yaklaşımı vardır:
    - Zincirleme (Chaining): Çakışan anahtarlar için bir liste (veya başka bir veri yapısı) kullanılır.
    - Açık Adresleme (Open Addressing): Çakışma durumunda boş bir slot aranır ve veri o slot'a yerleştirilir.

## Hashing ve Hash Table Kullanım Alanları

- **Veritabanı Yönetim Sistemleri:** Veritabanlarında hızlı veri erişimi sağlamak için hash table kullanılır.
- **Dizinleme:** Arama motorlarında, web sayfalarını hızlıca indekslemek için hash table'lar kullanılabilir.
- **Şifreleme ve Kimlik Doğrulama:** Şifreler ve kullanıcı doğrulama işlemleri için hash fonksiyonları kullanılır.
- **Hızlı Veri Erişimi:** Uygulamalarda veri yapılarını hızlıca aramak ve güncellemek için yaygın olarak kullanılır.

## Sonuç

**Hashing ve hash table veri yapıları**, verileri hızlı ve verimli bir şekilde depolamak ve erişmek için oldukça güçlü araçlardır. Hashing, verilerin anahtarlar üzerinden hızlıca erişilmesini sağlar. Hash table, bu anahtar-değer çiftlerini depolayan yapıdır. Ancak, çakışmalar gibi problemlerle karşılaşıldığında doğru çakışma çözümü tekniklerini uygulamak önemlidir.



# Veri Yapıları Algoritmaları

## Binary Search Tree (BST) Veri Yapısı

### Binary Search Tree (İkili Arama Ağacı) Nedir?

Binary Search Tree (BST), her bir düğümde en fazla iki çocuk bulunan bir ikili ağaç yapısıdır. BST'nin en önemli özelliği, her düğümün sol alt ağacındaki tüm değerlerin o düğümde küçük, sağ alt ağacındaki tüm değerlerin ise o düğümde büyük olmasıdır. Bu özellik sayesinde BST, veriye hızlı bir şekilde erişim sağlayabilir.

### BST'nin Temel Özellikleri

- Her düğümün en fazla iki çocuğu vardır.**
  - Bir düğümde en fazla iki çocuk (sol ve sağ) bulunabilir.
- Sol alt ağaçtaki tüm elemanlar, kök düğümde daha küçük olmalıdır.**
  - Sol alt ağaçta bulunan her düğüm, kök düğümde küçük bir değere sahip olmalıdır.
- Sağ alt ağaçtaki tüm elemanlar, kök düğümde daha büyük olmalıdır.**
  - Sağ alt ağaçta bulunan her düğüm, kök düğümde büyük bir değere sahip olmalıdır.
- BST'de her düğümde yalnızca bir anahtar değeri bulunur.**
- Ağaç yapısı dengeli olabilir veya dengesiz olabilir.**
  - Eğer ağacın her seviyesinde yeterli denge varsa, arama, ekleme ve silme işlemleri oldukça hızlıdır. Ancak dengesiz bir ağaç, verimliliği düşürebilir.

### Binary Search Tree Temel İşlemleri

- Ekleme (Insert):**
  - Yeni bir düğüm eklerken, BST'nin özelliklerine göre doğru yeri bulmak için ağaç üzerinde gezinilir ve yeni düğüm uygun yere yerleştirilir.
- Arama (Search):**
  - Bir anahtar ararken, kök düğümde başlanarak, aranan değer küçük mü büyük mü olduğuna bakılır. Sol veya sağ alt ağaçlara ilerleyerek arama işlemi yapılır.
- Silme (Delete):**
  - Bir düğüm silinirken, üç durumla karşılaşılır:
    - Düğümün hiç çocuğu yoksa: Düğüm doğrudan silinir.
    - Düğümün bir çocuğu varsa: Düğüm, tek çocuğuyla değiştirilir.
    - Düğümün iki çocuğu varsa: Düğümün yerine, sağ alt ağacındaki en küçük değer (veya sol alt ağacındaki en büyük değer) getirilir.
- Inorder Traversal (Gezinme):**
  - BST'de sıralı bir gezinti yapmak için inorder traversal kullanılır. Bu gezinme yöntemi, önce sol alt ağaç, sonra kök düğüm, sonra sağ alt ağaç şeklinde yapılır ve bu şekilde BST'nin elemanları sıralı bir şekilde elde edilir.



# Veri Yapıları Algoritmaları

## Avantajları

- ✓ **Hızlı Arama:** BST, sıralı bir yapıya sahip olduğu için, veriye erişim çok hızlıdır. Ortalama durumda  $O(\log n)$  zaman kompleksitesine sahiptir.
- ✓ **Hızlı Ekleme ve Silme:** Verinin yerini bulduktan sonra ekleme ve silme işlemleri de hızlı bir şekilde yapılabilir.

## Dezavantajları

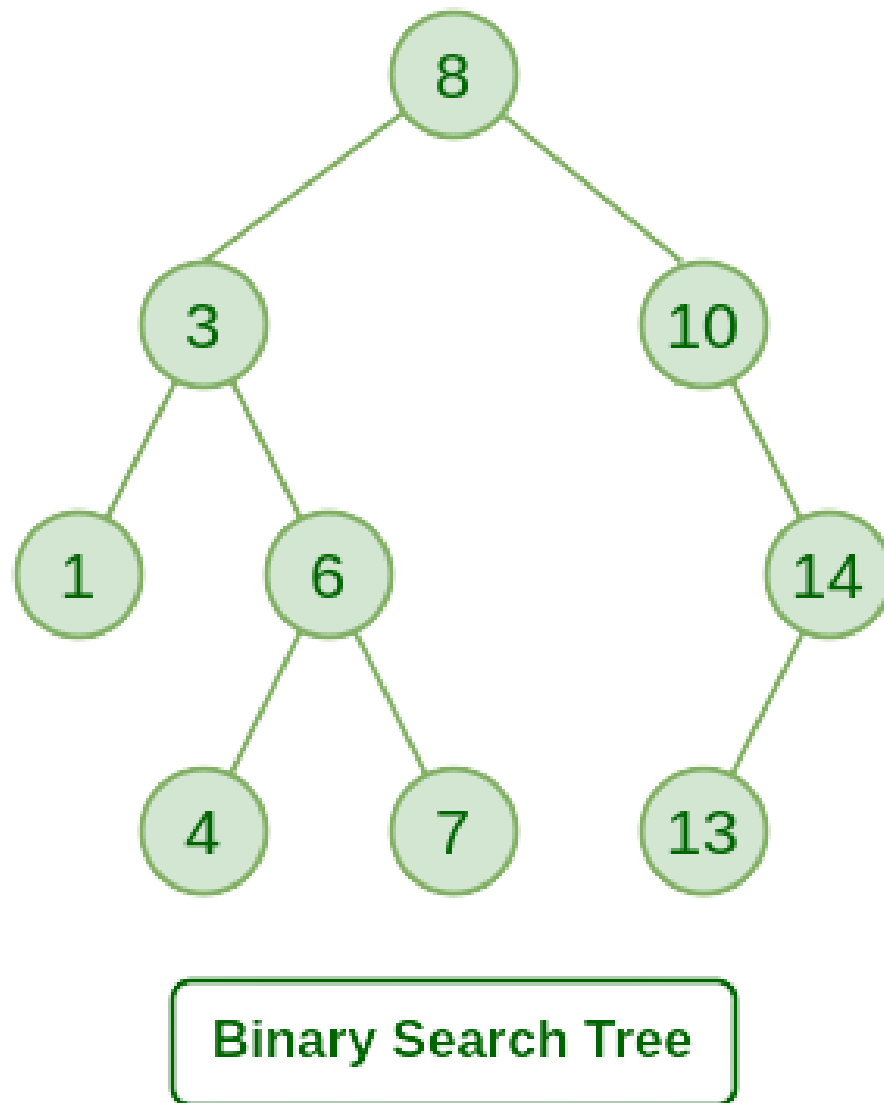
- ✗ **Dengesiz Ağaçlar:** Eğer ekleme işlemleri rastgele yapılırsa, BST dengesiz hale gelebilir ve bu durumda ağaç bir bağlı listeye dönüşebilir, böylece arama, ekleme ve silme işlemleri  $O(n)$  zaman kompleksitesine sahip olabilir.

## Kullanım Alanları

- **Veritabanı İndeksleme:** Veritabanları, verilerin hızlı bir şekilde aranması ve sıralanması için BST kullanabilir.
- **Dosya Sistemi Yönetimi:** Dosya sistemlerinde dizinleme ve arama işlemleri için BST yaygın olarak kullanılır.
- **Yapay Zeka ve Oyun Programlaması:** Bazı algoritmalar ve oyun programlarında karar ağaçları (decision trees) gibi yapılar kullanılır ve BST bu tür uygulamalarda da kullanılabilir.

## Sonuç

**Binary Search Tree (BST)**, verilerin sıralı bir yapıda tutulmasını sağlayan ve hızlı arama, ekleme ve silme işlemleri yapabilen güçlü bir veri yapısıdır. Ancak, ağacın dengeli olması önemlidir; aksi takdirde performans düşebilir. BST, birçok farklı alanda veri erişimi ve sıralama işlemleri için kullanışlıdır.





# Veri Yapıları Algoritmaları

## AVL Tree (Adelson-Velsky ve Landis Ağacı)

### AVL Tree Nedir?

**AVL Tree (Adelson-Velsky ve Landis Ağacı)**, self-balancing binary search tree (kendiliğinden dengelenen ikili arama ağacı) türlerinden biridir. AVL ağacında, her düğümün sol ve sağ alt ağaçlarının yükseklik farkı (denge faktörü) en fazla 1 olmalıdır. Bu, ağacın her zaman dengeli olmasını sağlar ve böylece arama, ekleme ve silme işlemlerinin zaman karmaşıklığı  $O(\log n)$  seviyesinde kalır.

### AVL Ağacının Temel Özellikleri

#### 1. İkili Arama Ağacı Özellikleri:

- Her düğümün sol alt ağacındaki tüm elemanlar o düğümden küçük, sağ alt ağacındaki tüm elemanlar ise o düğümden büyüktür.

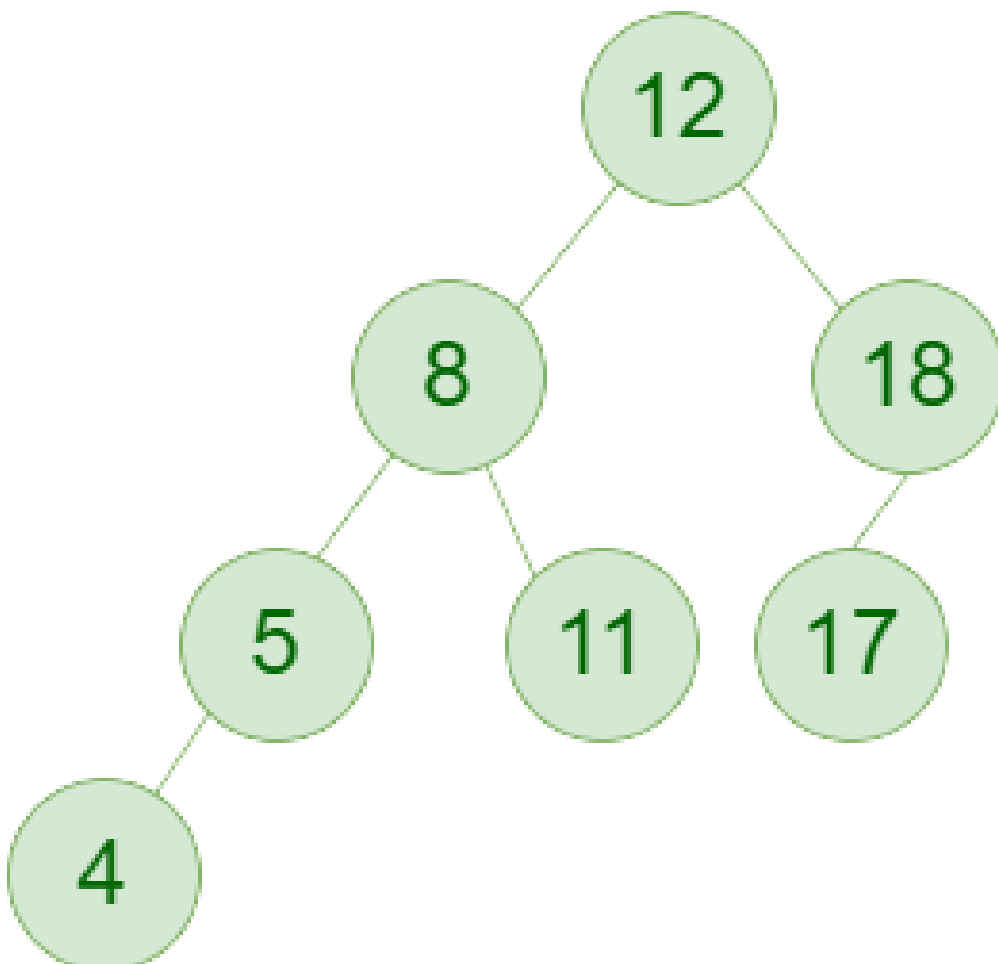
#### 2. Denge Durumu:

- Her düğümdeki denge faktörü, sol alt ağacın yüksekliğinden sağ alt ağacın yüksekliğini çıkararak hesaplanır.
- Denge faktörü şu şekilde hesaplanır:
  - Denge Faktörü = Yükseklik(Sol Alt Ağaç) - Yükseklik(Sağ Alt Ağaç)
- Denge faktörü 1, 0 veya -1 olmalıdır. Eğer denge faktörü bu aralık dışında bir değere sahipse, ağaç dengesizdir ve yeniden denge sağlanması gerekir.

#### 3. Denge Sağlama (Balanslama):

- AVL ağacında denge sağlamak için dönme (rotation) işlemi yapılır. Bu işlemler dört şekilde yapılabilir:
  - Sağ Dönme (Right Rotation): Denge faktörü -2 ve sol alt ağaçta derinlik fazla ise yapılır.
  - Sol Dönme (Left Rotation): Denge faktörü 2 ve sağ alt ağaçta derinlik fazla ise yapılır.
  - Sol-Sağ Dönme (Left-Right Rotation): Sol alt ağacın sağ alt ağacı daha derinse yapılır.
  - Sağ-Sol Dönme (Right-Left Rotation): Sağ alt ağacın sol alt ağacı daha derinse yapılır.

## AVL Tree





# Veri Yapıları Algoritmaları

## AVL Ağacının Temel İşlemleri

- **Ekleme (Insert):**
  - Yeni bir düğüm eklerken, tıpkı Binary Search Tree (BST) gibi, doğru yeri bulmak için ağacın altına gidilir.
  - Ancak, her ekleme işleminden sonra ağacın dengesinin kontrol edilmesi ve gerekirse dönme işlemi yapılması gerekir.
- **Arama (Search):**
  - AVL ağacında arama işlemi de ikili arama ağacına benzer şekilde yapılır. Ancak, AVL ağacının dengesiz olma durumu ortadan kaldırıldığı için arama işlemi daha hızlıdır.
- **Silme (Delete):**
  - Bir düğüm silindikten sonra, tıpkı ekleme işlemi gibi, ağaç yeniden dengelenmelidir.
  - Silme işleminden sonra ağacın her düğümü tekrar kontrol edilir ve denge bozulmuşsa uygun dönme işlemi yapılır.
- **Dönme İşlemleri (Rotations):**
  - Sağ Dönme (Right Rotation): Ağaçta sağa dönüş yapılır, kök düğümün sol alt ağacı sağa kaydırılır.
  - Sol Dönme (Left Rotation): Ağaçta sola dönüş yapılır, kök düğümün sağ alt ağacı sola kaydırılır.
  - Sol-Sağ Dönme (Left-Right Rotation): Sol alt ağacın sağ alt ağacı daha derinse, önce sol dönüş yapılır, ardından sağ dönüş yapılır.
  - Sağ-Sol Dönme (Right-Left Rotation): Sağ alt ağacın sol alt ağacı daha derinse, önce sağ dönüş yapılır, ardından sol dönüş yapılır.

## Avantajları

✓ **Denge:** AVL ağacı, her zaman dengede kalır, böylece arama, ekleme ve silme işlemleri hızlı bir şekilde yapılabilir ( $O(\log n)$ ).

✓ **Hızlı Erişim:** AVL ağacındaki veriler sıralı bir şekilde saklanır ve her işlemde daha hızlı erişim sağlanır.

## Dezavantajları

✗ **Dönme İşlemleri:** Her ekleme ve silme işleminden sonra dönme işlemleri yapılması gerektiği için bu işlemler ek maliyet getirir.

✗ **Daha Karmaşık Yapı:** AVL ağacı, standart ikili arama ağacından daha karmaşıktır. Kod ve mantık açısından daha fazla dikkat gerektirir.

## Kullanım Alanları

- **Veritabanı İndeksleme:** Veritabanlarında hızlı veri arama ve sıralama için AVL ağacı kullanılabilir.
- **Dosya Sistemleri:** Dosya sistemlerinde dosyaların sıralı olarak tutulması için AVL ağacı kullanılabilir.
- **Ağaç Yapıları:** Bilgisayar bilimlerinde birçok farklı ağaç yapısı AVL ağacını temel olarak geliştirilmiştir.

## Sonuç

**AVL Tree**, dengesini sürekli olarak koruyan ve her işlemde yüksek performans sunan bir ikili arama ağacıdır. Arama, ekleme ve silme işlemleri, diğer ağaç yapıları ile karşılaştırıldığında daha hızlıdır ve ağacın her zaman dengeli olmasını sağlar.



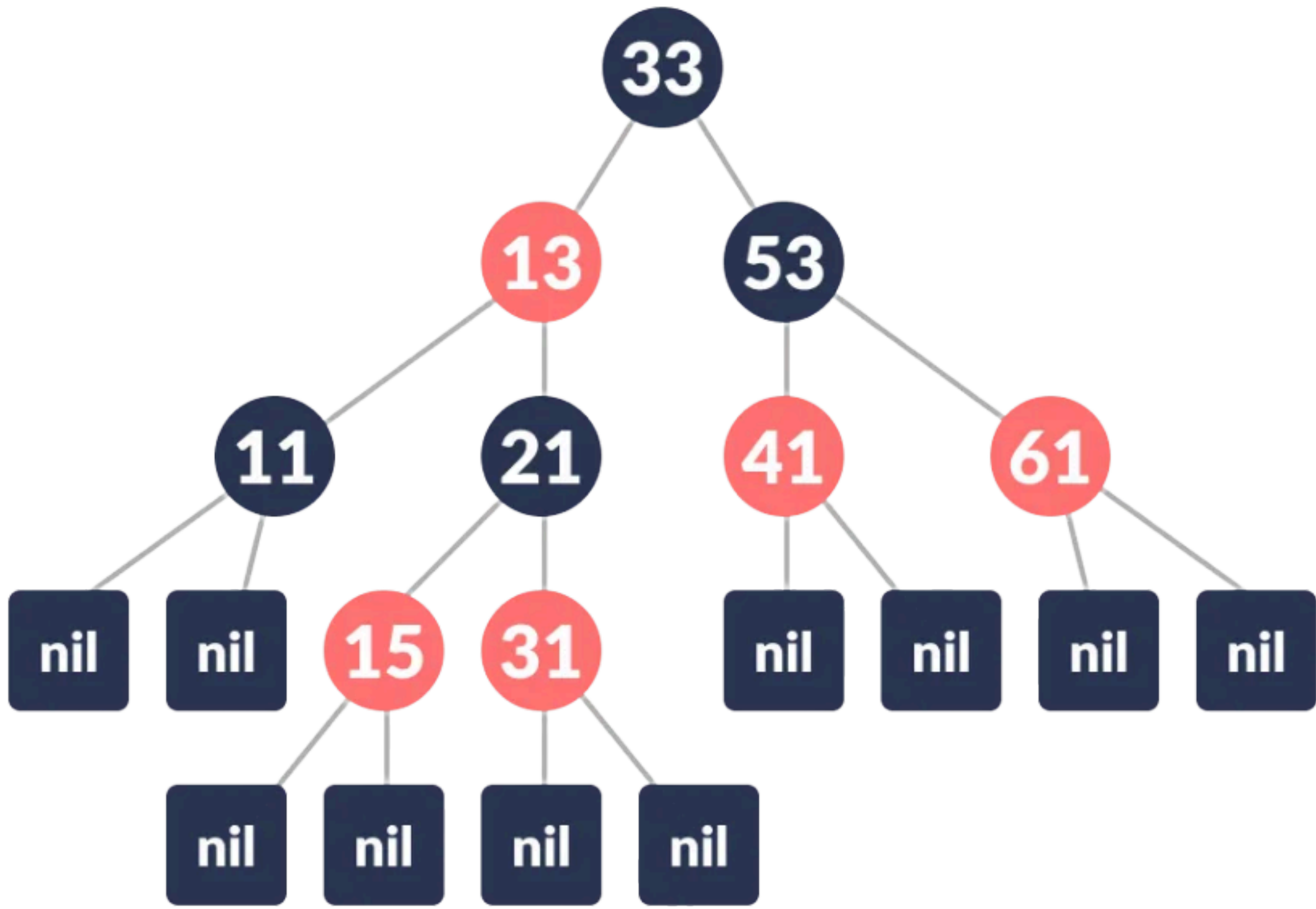
# Veri Yapıları Algoritmaları

## Red-Black Tree Algoritması

**Red-Black Tree**, ikili arama ağaçları (Binary Search Tree) üzerine kurulu bir veri yapısıdır. Bu ağaç yapısı, temel ikili arama ağacının dezavantajlarını gidermek ve daha dengeli bir yapı sunmak için tasarlanmıştır. Red-Black Tree, ekleme, silme ve arama işlemlerini  $O(\log n)$  zaman karmaşıklığında gerçekleştirir.

### Temel Özellikler

- Düğüm Renkleri:** Her düğüm ya kırmızı ya da siyah renktedir. Bu renkler, ağacın dengeli kalmasını sağlamak için kullanılır.
- Kök Düğüm:** Kök düğüm her zaman siyahtır.
- Yaprak Düğümler (NIL):** Yaprak düğümler (NIL düğümleri) siyah olarak kabul edilir. Bu düğümler, ağacın sonunu temsil eder ve veri içermez.
- Kırmızı Düğüm Kuralı:** Bir kırmızı düğümün çocukları mutlaka siyah olmalıdır. Yani, iki kırmızı düğüm üst üste gelemez.
- Siyah Yükseklik:** Herhangi bir düğümden yaprak düğümlere (NIL) giden tüm yollarda aynı sayıda siyah düğüm bulunmalıdır. Bu, ağacın dengeli olmasını sağlar.





# Veri Yapıları Algoritmaları

## Ekleme İşlemi

Red-Black Tree'ye yeni bir düğüm eklendiğinde, düğüm başlangıçta kırmızı olarak eklenir. Ancak bu ekleme işlemi, ağacın Red-Black Tree kurallarını ihlal etmesine neden olabilir. Bu durumda, ağacı yeniden dengelemek için rotasyon ve renk değiştirme işlemleri yapılır.

- Standart İkili Arama Ağacı Ekleme: Yeni düğüm, standart ikili arama ağacı kurallarına göre eklenir.
- Renk Ayarlaması: Eğer eklenen düğümün ebeveyni kırmızı ise, Red-Black Tree kurallarını ihlal edebilir. Bu durumda, ağacı dengelemek için rotasyon ve renk değiştirme işlemleri yapılır.

## Silme İşlemi

Red-Black Tree'den bir düğüm silindiğinde, ağacın dengeli kalmasını sağlamak için yine rotasyon ve renk değiştirme işlemleri yapılır. Silme işlemi, standart ikili arama ağacı silme işlemine benzer, ancak ek olarak renk ve denge ayarlamaları yapılır.

- Standart İkili Arama Ağacı Silme: Düğüm, standart ikili arama ağacı kurallarına göre silinir.
- Renk ve Denge Ayarlaması: Eğer silinen düğüm siyah ise, ağacın dengeli kalmasını sağlamak için rotasyon ve renk değiştirme işlemleri yapılır.

## Avantajları

✓ **Dengeli Yapı:** Red-Black Tree, ekleme ve silme işlemlerinden sonra kendini dengeler, bu da arama, ekleme ve silme işlemlerinin  $O(\log n)$  zaman karmaşıklığında gerçekleşmesini sağlar.

✓ **Esneklik:** AVL ağaçlarına kıyasla daha az sıkı dengelenme kurallarına sahip olduğu için, ekleme ve silme işlemlerinde daha az rotasyon gerektirir.

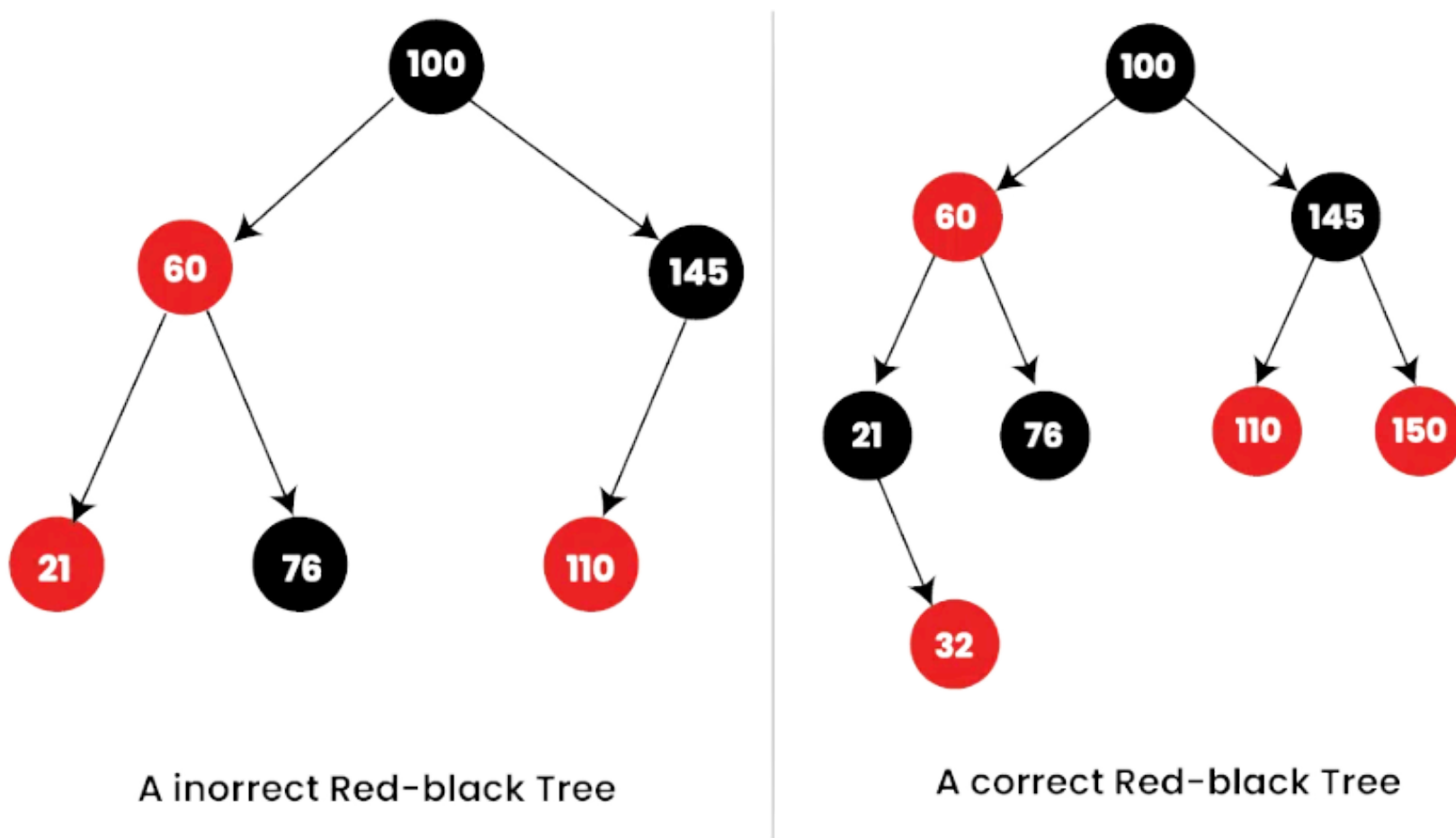
## Kullanım Alanları

**Red-Black Tree**, özellikle hızlı arama, ekleme ve silme işlemlerinin gerektiği durumlarda kullanılır. Örneğin, Java'daki TreeMap ve TreeSet gibi veri yapıları, içlerinde Red-Black Tree kullanır.

## Sonuç

**Red-Black Tree**, ikili arama ağaçlarının dengeli bir versiyonudur ve özellikle dinamik veri yapılarında sıkça kullanılır. Renkler ve kurallar sayesinde, ağaç dengeli kalır ve işlemler verimli bir şekilde gerçekleştirilir.

## Example of Red-black Tree





# Veri Yapıları Algoritmaları

## Trie Algoritması

**Trie**, bir tür ağaç veri yapısıdır ve genellikle dizeler (strings) üzerinde hızlı arama, ekleme ve silme işlemleri yapmak için kullanılır. Trie, özellikle sözlük tabanlı uygulamalarda (örneğin, otomatik tamamlama, yazım denetimi) yaygın olarak kullanılır.

### Temel Özellikler

- Düğüm Yapısı:** Trie'nin her düğümü, bir karakteri temsil eder. Her düğüm, çocuk düğümlerini tutan bir koleksiyona (örneğin, bir dizi veya hash map) sahiptir.
- Kök Düğüm:** Trie'nin kök düğümü boş bir dizeyi temsil eder. Kök düğümün çocukları, dizelerin ilk karakterlerini temsil eder.
- Yaprak Düğümler:** Bir dizenin son karakterini temsil eden düğüm, yaprak düğüm olarak işaretlenir. Bu, o düğümün bir kelimenin sonunu temsil ettiğini gösterir.
- Verimli Arama:** Trie, bir dizenin varlığını kontrol etmek için  $O(m)$  zaman karmaşıklığına sahiptir, burada  $m$ , aranan dizenin uzunluğudur. Bu, diğer veri yapılarına (örneğin, hash tabloları) kıyasla oldukça hızlıdır.

### Örnek Senaryo

Örneğin, "apple", "app", "apricot", "banana" gibi kelimeleri içeren bir Trie yapısı düşünelim:

- Kök düğümünden başlayarak, her karakter bir düğümü temsil eder.
- "apple" kelimesi için: a -> p -> p -> l -> e (e düğümü yaprak olarak işaretlenir).
- "app" kelimesi için: a -> p -> p (p düğümü yaprak olarak işaretlenir).
- "apricot" kelimesi için: a -> p -> r -> i -> c -> o -> t (t düğümü yaprak olarak işaretlenir).
- "banana" kelimesi için: b -> a -> n -> a -> n -> a (a düğümü yaprak olarak işaretlenir).

**Bu yapı sayesinde**, "app" önekiyle başlayan kelimeleri bulmak için sadece "a -> p -> p" yolunu takip etmek yeterlidir.

### Avantajları

- ✓ **Hızlı Arama:** Trie, özellikle uzun dizelerde hızlı arama yapabilir.
- ✓ **Önek Arama:** Trie, belirli bir önekle başlayan tüm dizeleri bulmak için idealdir.
- ✓ **Verimli Bellek Kullanımı:** Trie, ortak önekleri paylaşan dizeler için bellek kullanımını optimize eder.

### Dezavantajları

- ✗ **Bellek Tüketimi:** Trie, özellikle büyük alfabeler (örneğin, Unicode) için yüksek bellek tüketimine neden olabilir.
- ✗ **Karmaşıklık:** Trie'nin implementasyonu, diğer veri yapılarına kıyasla daha karmaşıktır.

### Kullanım Alanları

- Otomatik Tamamlama:** Trie, kullanıcının yazdığı metne göre öneriler sunmak için kullanılır.
- Yazım Denetimi:** Trie, bir metindeki hatalı kelimeleri bulmak için kullanılır.
- IP Yönlendirme:** Trie, IP adreslerini hızlı bir şekilde eşleştirmek için kullanılır.

### Sonuç

**Trie**, dizeler üzerinde hızlı ve verimli işlemler yapmak için güçlü bir veri yapısıdır. Özellikle önek tabanlı aramalarda ve sözlük uygulamalarında sıkça kullanılır. Ancak, bellek tüketimi ve implementasyon karmaşıklığı gibi dezavantajları da göz önünde bulundurulmalıdır.



# Veri Yapıları Algoritmaları

## Heap Algoritması

**Heap (yığın)**, özel bir ağaç tabanlı veri yapısıdır ve genellikle dinamik olarak değişen veri kümelerinde maksimum veya minimum değerleri hızlı bir şekilde bulmak için kullanılır. Heap, özellikle öncelik kuyrukları (priority queue) ve sıralama algoritmalarında (örneğin, Heap Sort) yaygın olarak kullanılır.

### Temel Özellikler

#### 1. Heap Türleri:

- **Max Heap:** Her düğümün değeri, çocuklarının değerlerinden büyük veya eşittir. Kök düğüm, ağaçtaki en büyük değeri içerir.
- **Min Heap:** Her düğümün değeri, çocuklarının değerlerinden küçük veya eşittir. Kök düğüm, ağaçtaki en küçük değeri içerir.

2. **Ağaç Yapısı:** Heap, tam bir ikili ağaç (complete binary tree) olarak organize edilir. Bu, ağacın tüm seviyelerinin dolu olduğu ve son seviyenin soldan sağa doldurulduğu anlamına gelir.

3. **Dizi Tabanlı Temsil:** Heap, genellikle bir dizi (array) kullanılarak temsil edilir. Bu, bellek kullanımını optimize eder ve ağaç yapısını kolayca yönetmeyi sağlar.

#### 4. Ekleme ve Çıkarma İşlemleri:

- **Ekleme:** Yeni bir eleman eklendiğinde, ağacın sonuna yerleştirilir ve ardından heap özelliğini korumak için yukarı doğru düzeltme (heapify up) işlemi yapılır.
- **Çıkarma:** Kök düğümünden bir eleman çıkarıldığında, ağacın son elemanı kök düğüme taşınır ve ardından heap özelliğini korumak için aşağı doğru düzeltme (heapify down) işlemi yapılır.

### Avantajları

✓ **Hızlı Erişim:** Heap, en büyük veya en küçük elemana  $O(1)$  zaman karmaşıklığında erişim sağlar.

✓ **Verimli Ekleme ve Çıkarma:** Ekleme ve çıkarma işlemleri  $O(\log n)$  zaman karmaşıklığında gerçekleşir.

✓ **Dinamik Yapı:** Heap, dinamik olarak büyüyebilir ve küçülebilir.

### Dezavantajları

✗ **Sınırlı Kullanım:** Heap, sadece maksimum veya minimum değerleri bulmak için optimize edilmiştir. Diğer işlemler (örneğin, belirli bir elemanı arama) verimli değildir.

✗ **Bellek Yönetimi:** Heap, dizi tabanlı olduğu için bellek kullanımı dikkatli bir şekilde yönetilmelidir.

### Kullanım Alanları

- **Öncelik Kuyrukları:** Heap, öncelik kuyruklarını uygulamak için idealdir.
- **Sıralama Algoritmaları:** Heap Sort, Heap veri yapısını kullanarak verimli bir sıralama algoritmasıdır.
- **Graf Algoritmaları:** Dijkstra ve Prim gibi algoritmalarda, en kısa yol veya minimum kapsayan ağaç bulmak için kullanılır.



# Veri Yapıları Algoritmaları

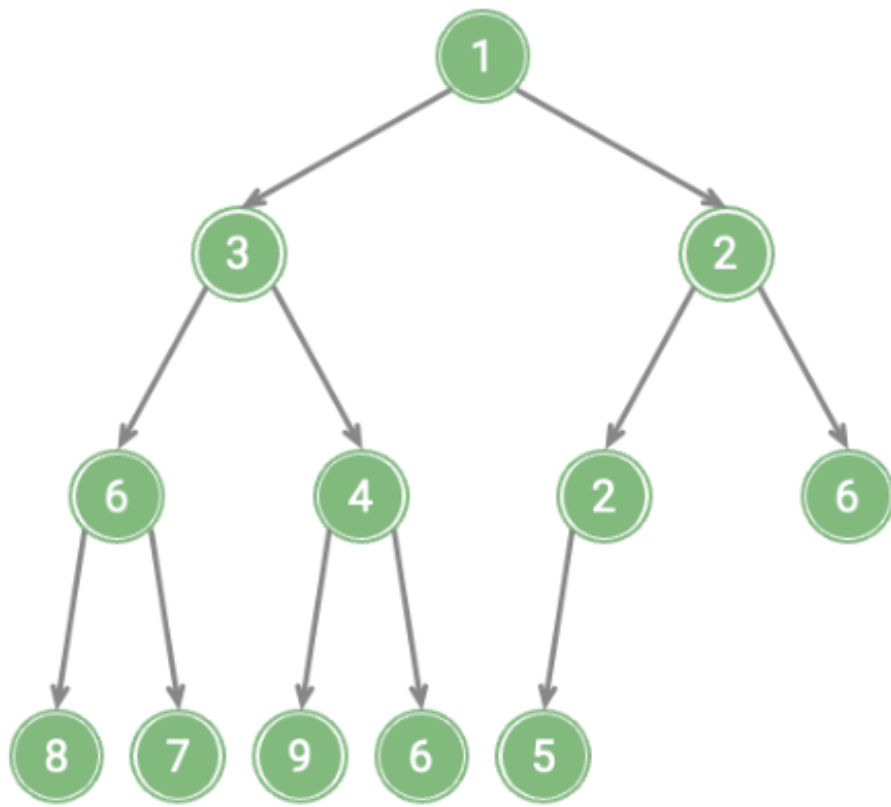
## Örnek Senaryo

Örneğin, bir Max Heap yapısı düşünelim:

- **Ekleme İşlemi:**
  - Yeni bir eleman (örneğin, 15) eklendiğinde, ağacın sonuna yerleştirilir.
  - Eleman, ebeveyninden büyükse, ebeveyniyle yer değiştirir. Bu işlem, heap özelliği korunana kadar devam eder.
- **Çıkarma İşlemi:**
  - Kök düğüm (en büyük eleman) çıkarılır.
  - Ağacın son elemanı kök düğümüne taşınır.
  - Eleman, çocuklarından küçükse, çocuklarıyla yer değiştirir. Bu işlem, heap özelliği korunana kadar devam eder.

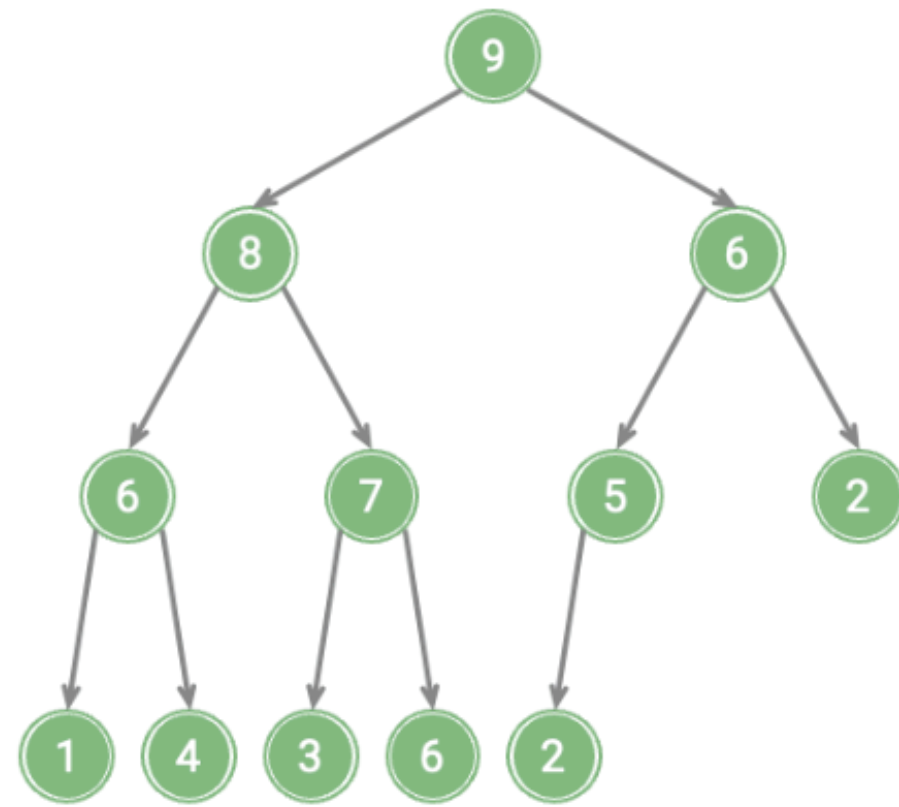
## Sonuç

**Heap**, özellikle dinamik veri kümelerinde maksimum veya minimum değerleri hızlı bir şekilde bulmak için güçlü bir veri yapısıdır. Öncelik kuyrukları ve sıralama algoritmaları gibi birçok uygulamada yaygın olarak kullanılır. Ancak, sınırlı kullanım alanları ve bellek yönetimi gibi dezavantajları da göz önünde bulundurulmalıdır.



It is a "complete binary tree" and  
any node's value  $\leq$  its children's values

**Min-heap**



It is a "complete binary tree" and  
any node's value  $\geq$  its children's values

**Max-heap**



# Veri Yapıları Algoritmaları

## Graph Algoritması

**Graph (çizge)**, düğümler (node) ve bu düğümleri birbirine bağlayan kenarlardan (edge) oluşan bir veri yapısıdır. Graphlar, gerçek dünyadaki birçok problemi modellemek için kullanılır. Örneğin, sosyal ağlar, ulaşım ağları, bilgisayar ağları ve daha birçok alanda graphlar yaygın olarak kullanılır.

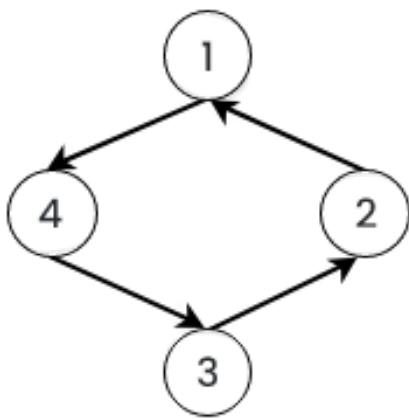
### Temel Kavramlar

- Düğüm (Node):** Graph'taki bir nesneyi temsil eder. Örneğin, bir sosyal ağda bir kişi veya bir ulaşım ağında bir şehir.
- Kenar (Edge):** İki düğüm arasındaki bağlantıyı temsil eder. Kenarlar, yönlü (directed) veya yönsüz (undirected) olabilir.
  - Yönlü Graph: Kenarların bir yönü vardır. Örneğin, A'dan B'ye bir kenar, B'den A'ya bir kenar anlamına gelmez.
  - Yönsüz Graph: Kenarların yönü yoktur. Örneğin, A'dan B'ye bir kenar, B'den A'ya da bir kenar anlamına gelir.
- Ağırlıklı Graph:** Kenarların bir ağırlığı (weight) vardır. Bu, iki düğüm arasındaki mesafe, maliyet veya başka bir metriği temsil edebilir.
- Komşuluk (Adjacency):** Bir düğümün komşuları, ona doğrudan bağlı olan düğümlerdir.
- Yol (Path):** Bir düğümden diğerine giden bir dizi kenardır.
- Döngü (Cycle):** Bir düğümden başlayıp aynı düğüme dönen bir yol.

### Graph Temsil Yöntemleri

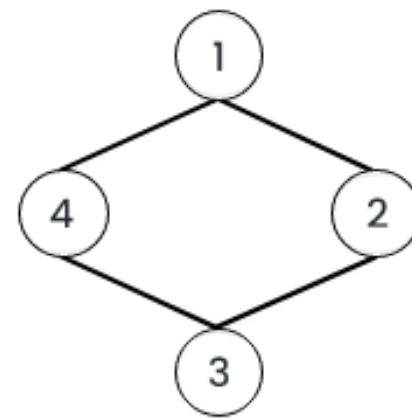
- Komşuluk Matrisi (Adjacency Matrix):**
  - Bir 2D dizi kullanılarak graph temsil edilir.
  - Dizi boyutu  $n \times n$  olur, burada  $n$  düğüm sayısıdır.
  - $matrix[i][j]$ ,  $i$  ve  $j$  düğümleri arasında bir kenar olup olmadığını gösterir.
  - Ağırlıklı graphlarda,  $matrix[i][j]$  kenarın ağırlığını temsil eder.
- Komşuluk Listesi (Adjacency List):**
  - Her düğüm için bir liste kullanılarak graph temsil edilir.
  - Liste, bir düğümün komşularını içerir.
  - Ağırlıklı graphlarda, her komşu için ağırlık da saklanır.

### Directed Graph



VS

### Undirected Graph





# Veri Yapıları Algoritmaları

## Graph Algoritmaları

- **Gezgin Algoritmaları (Traversal Algorithms):**
  - **BFS (Breadth-First Search):** Graph'ı genişlik öncelikli olarak dolaşır. Kısa yol bulma ve bağlantılı bileşenleri bulma gibi problemlerde kullanılır.
  - **DFS (Depth-First Search):** Graph'ı derinlik öncelikli olarak dolaşır. Döngü tespiti ve topolojik sıralama gibi problemlerde kullanılır.
- **En Kısa Yol Algoritmaları:**
  - **Dijkstra Algoritması:** Bir düğümden diğer tüm düğümlere olan en kısa yolları bulur. Ağırlıklı graphlarda kullanılır.
  - **Bellman-Ford Algoritması:** Negatif ağırlıklı kenarların olduğu graphlarda en kısa yolu bulur.
  - **Floyd-Warshall Algoritması:** Tüm düğüm çiftleri arasındaki en kısa yolları bulur.
- **Minimum Kapsayan Ağaç (Minimum Spanning Tree):**
  - **Kruskal Algoritması:** Kenarları ağırlıklarına göre sıralar ve en küçük ağırlıklı kenarları seçer.
  - **Prim Algoritması:** Bir düğümden başlayarak en küçük ağırlıklı kenarları ekler.
- **Topolojik Sıralama:** Yönlü graphlarda, düğümleri bir sıraya dizme işlemidir. Özellikle bağımlılık grafiklerinde kullanılır.

## Örnek Senaryo

Örneğin, bir ulaşım ağı düşünelim:

- **Düğümmler:** Şehirler (A, B, C, D).
- **Kenarlar:** Şehirler arasındaki yollar (A-B, B-C, C-D, D-A).
- **Ağırlıklar:** Yolların uzunlukları (A-B: 10, B-C: 20, C-D: 15, D-A: 25).

Bu graph üzerinde Dijkstra algoritması kullanılarak, bir şehirden diğerine en kısa yol bulunabilir.

## Kullanım Alanları

- **Sosyal Ağlar:** İnsanlar arasındaki ilişkileri modellemek için.
- **Ulaşım Ağları:** Şehirler veya duraklar arasındaki bağlantıları modellemek için.
- **Bilgisayar Ağları:** Bilgisayarlar veya router'lar arasındaki bağlantıları modellemek için.
- **Yapay Zeka:** Durum uzayı aramaları ve planlama problemlerinde.

## Sonuç

**Graph**, gerçek dünyadaki birçok problemi modellemek için güçlü bir veri yapısıdır. Gezgin algoritmaları, en kısa yol algoritmaları ve minimum kapsayan ağaç algoritmaları gibi birçok algoritma, graphlar üzerinde çalışır. Bu algoritmalar, sosyal ağlar, ulaşım ağları ve bilgisayar ağları gibi birçok alanda yaygın olarak kullanılır.



# Graf Algoritmaları

## Depth-First Search (DFS) Algoritması

**Depth-First Search (DFS)**, bir graph veya ağaç yapısını dolaşmak için kullanılan bir algoritmadır. DFS, bir düğümden başlayarak mümkün olduğunca derine iner ve geri dönüş (backtracking) yaparak tüm düğümleri ziyaret eder. Bu algoritma, özellikle graph'ta döngü tespiti, bağlantılı bileşenleri bulma ve topolojik sıralama gibi problemlerde kullanılır.

### Temel Kavramlar

1. **Düğüm (Node)**: Graph'taki bir nesneyi temsil eder.
2. **Kenar (Edge)**: İki düğüm arasındaki bağlantıyı temsil eder.
3. **Ziyaret Edilenler (Visited)**: Ziyaret edilen düğümleri takip etmek için kullanılır.
4. **Stack veya Rekürsif Yaklaşım**: DFS, stack veri yapısı veya rekürsif fonksiyonlar kullanılarak gerçekleştirilebilir.

### DFS Algoritmasının Adımları

1. **Başlangıç**: Belirli bir düğümden başla.
2. **Ziyaret Et**: Düğümü ziyaret et ve ziyaret edildi olarak işaretle.
3. **Komşuları Keşfet**: Ziyaret edilen düğümün komşularını gez. Eğer bir komşu ziyaret edilmemişse, o komşuya git ve aynı işlemi tekrarla.
4. **Geri Dönüş (Backtracking)**: Tüm komşular ziyaret edildiğinde, bir önceki düğüme geri dön ve işlemi tekrarla.

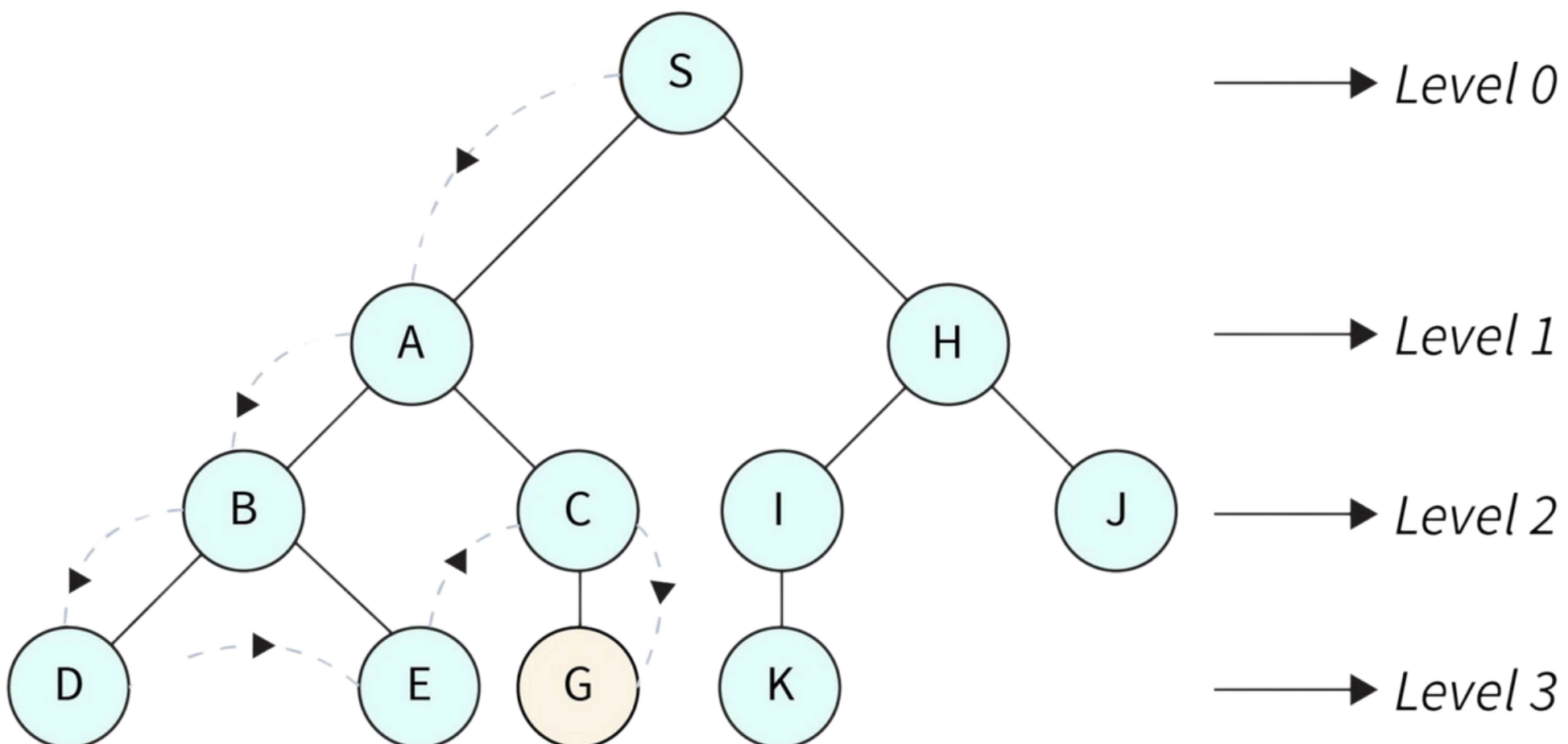
### DFS Türleri

1. **Pre-order DFS**: Düğüm ziyaret edildikten sonra komşuları keşfedilir.
2. **Post-order DFS**: Komşular keşfedildikten sonra düğüm ziyaret edilir.

### DFS'nin Kullanım Alanları

- **Döngü Tespiti**: Graph'ta döngü olup olmadığını kontrol etmek için.
- **Bağlantılı Bileşenler**: Graph'taki bağlantılı bileşenleri bulmak için.
- **Topolojik Sıralama**: Yönlü graph'larda düğümleri sıralamak için.
- **Yol Bulma**: İki düğüm arasında bir yol olup olmadığını kontrol etmek için.

## Depth First Search





# Graf Algoritmaları

## Avantajları

- ✓ **Bellek Verimliliği:** Rekürsif yaklaşım veya stack kullanıldığında, BFS'ye göre daha az bellek kullanır.
- ✓ **Derinlik Öncelikli:** Özellikle derin graph'larda hızlı sonuçlar verebilir.

## Dezavantajları

- ✗ **Yerel Minimum:** Bazı problemlerde yerel minimumda takılıp kalabilir.
- ✗ **Stack Taşması:** Rekürsif yaklaşımda derin graph'larda stack taşmasına neden olabilir.

## Örnek Senaryo

Örneğin, aşağıdaki graph'ı düşünelim:

A -- B -- C  
| |  
D -- E

**DFS algoritması**, A'dan başlayarak şu sırayla düğümleri ziyaret edebilir:

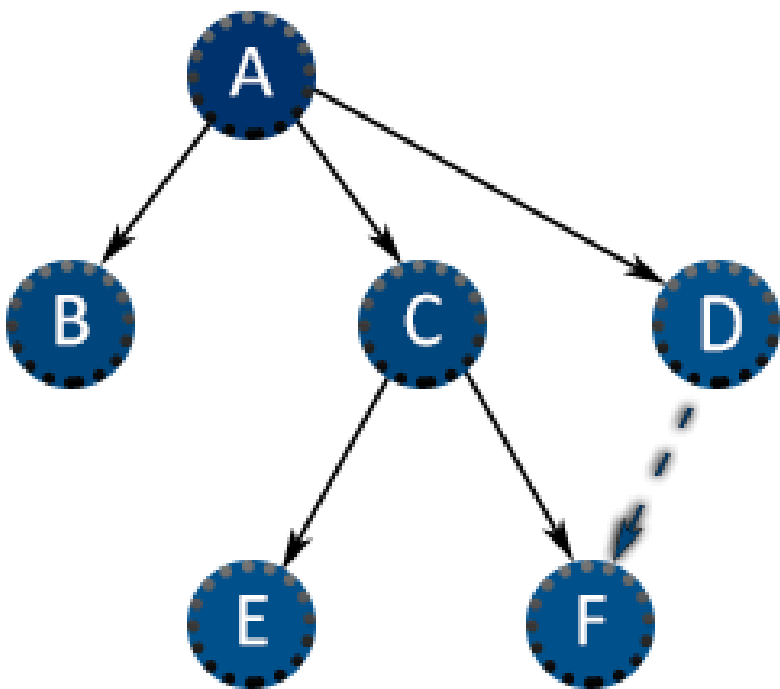
- A -> B -> C (C'nin komşusu yok, geri dön)
- B -> E (E'nin komşusu D, ziyaret edilmemiş)
- E -> D (D'nin komşusu A, ziyaret edilmiş, geri dön)
- Tüm düğümler ziyaret edildi.

Sonuç: **A, B, C, E, D.**

## Sonuç

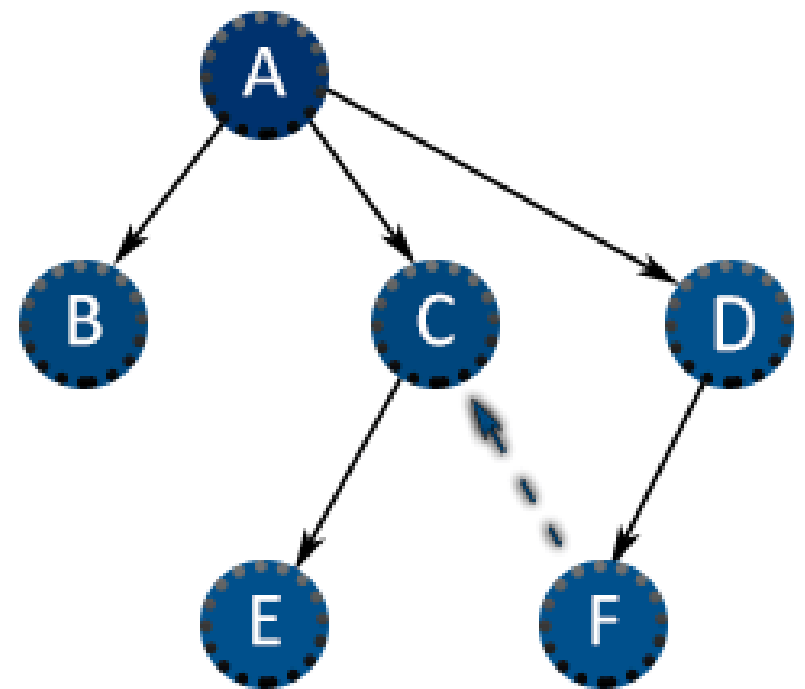
DFS, graph veya ağaç yapılarını dolaşmak için güçlü bir algoritmadır. Özellikle derinlik öncelikli aramalarda ve döngü tespiti gibi problemlerde yaygın olarak kullanılır. Ancak, stack taşması ve yerel minimum gibi dezavantajları da göz önünde bulundurulmalıdır.

## BFS



A B C D E F

## DFS



A D F C E B



# Graf Algoritmaları

## Breadth-First Search (BFS) Algoritması

**Breadth-First Search (BFS)**, bir graf veya ağaç yapısı üzerinde dolaşmak için kullanılan bir algoritmadır. Bu algoritma, başlangıç düğümünden başlayarak, önce başlangıç düğümünün komşularını, ardından bu komşuların komşularını ve bu şekilde tüm düğümleri ziyaret eder. BFS, genişleme önceliğine sahip olduğu için "genişlik öncelikli arama" olarak da adlandırılır.

### Algoritmanın Adımları

- Başlangıç Düğümü Seçimi:** Algoritma, başlangıç düğümü olarak belirlenen bir düğümden başlar. Bu düğüm, genellikle grafın kök düğümü veya belirli bir hedef düğüm olabilir.
- Kuyruk Yapısı Kullanımı:** BFS, bir kuyruk (queue) veri yapısı kullanır. Kuyruk, ziyaret edilecek düğümleri sırayla tutar. Başlangıç düğümü kuyruğa eklenir.
- Düğüm Ziyareti:** Kuyruktan bir düğüm çıkarılır ve bu düğüm ziyaret edilir. Ziyaret edilen düğüm, hedef düğüm olabilir veya belirli bir işlem gerçekleştirilebilir.
- Komşu Düğümlerin Eklenmesi:** Ziyaret edilen düğümün henüz ziyaret edilmemiş komşu düğümleri kuyruğa eklenir. Bu işlem, tüm komşu düğümler için tekrarlanır.
- Tekrarlama:** Kuyruk boşalana kadar adımlar 3 ve 4 tekrarlanır. Kuyruk boşaldığında, tüm erişilebilir düğümler ziyaret edilmiş demektir.

### BFS'in Özellikleri

- Kısa Yolu Bulma:** BFS, başlangıç düğümünden diğer düğümlere olan en kısa yolu bulmak için kullanılabilir. Bu özellik, özellikle ağırlıksız graflarda etkilidir.
- Döngü Tespiti:** BFS, graf içinde döngü olup olmadığını tespit etmek için kullanılabilir.
- Seviye Bazlı Gezinme:** BFS, düğümleri seviye seviye gezdiği için, grafın katmanlar halinde incelenmesini sağlar.

### Örnek Senaryo

Bir graf üzerinde BFS uygulandığını düşünelim:

- Başlangıç düğümü A olsun.
- A kuyruğa eklenir.
- A ziyaret edilir ve komşuları B ve C kuyruğa eklenir.
- B ziyaret edilir ve komşusu D kuyruğa eklenir.
- C ziyaret edilir ve komşusu E kuyruğa eklenir.
- D ziyaret edilir ve komşusu F kuyruğa eklenir.
- E ziyaret edilir ve komşusu G kuyruğa eklenir.
- F ve G ziyaret edilir.

Bu şekilde, tüm düğümler ziyaret edilmiş olur ve en kısa yollar bulunur.

### Sonuç

**BFS**, graf teorisinde sıkça kullanılan temel bir algoritmadır. Özellikle kısa yol bulma ve seviye bazlı gezinme ihtiyaçlarında etkili bir çözüm sunar. Kuyruk yapısı kullanımı sayesinde, düğümlerin sırayla işlenmesini sağlar ve bu da algoritmanın basit ve anlaşılır olmasını sağlar.



# Graf Algoritmaları

## Dijkstra Algoritması Nedir?

**Dijkstra Algoritması**, bir kaynaktan diğer tüm düğümlere en kısa yolları bulmak için kullanılan bir grafik algoritmasıdır. Ağırlıklı ve yönlendirilmiş grafiklerde çalışır. En düşük maliyetli yolları hesaplamak için kullanılır.

### Algoritmanın Çalışma Mantığı

- Başlangıç Noktası Seçimi:** Algoritma, başlangıç düğümünü seçerek başlar.
- Mesafe Değerlerinin Atanması:** Seçilen başlangıç düğümünün mesafesi 0 olarak atanır, diğer tüm düğümlerin mesafesi sonsuz olarak kabul edilir.
- Ziyaret Edilecek Düğümün Seçilmesi:** En düşük mesafeye sahip düğüm seçilir ve ziyaret edilir.
- Komşu Düğümler İçin Güncelleme:** Ziyaret edilen düğümün komşuları üzerinden mesafe hesaplamaları yapılır.
- Tüm Düğümler Ziyaret Edilene Kadar Devam Etme:** Tüm düğümler ziyaret edilene kadar bu işlem tekrarlanır.

### Algoritmanın Adımları

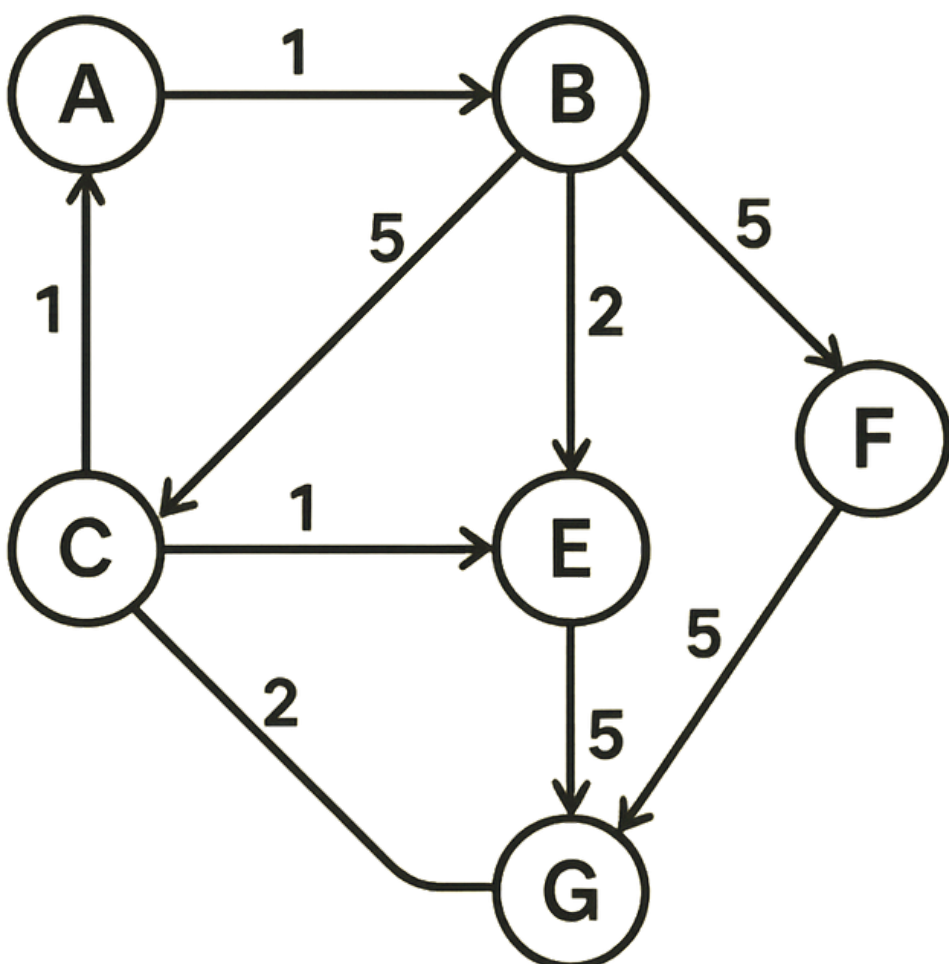
- Başlangıç düğümünü belirle ve mesafesini 0 yap.
- Tüm diğer düğümleri sonsuz mesafe ile başlat.
- Ziyaret edilmemiş düğümler arasından en düşük mesafeli düğümü seç.
- Seçilen düğümün komşularının mesafelerini güncelle.
- Ziyaret edilen düğümleri işaretle.
- Tüm düğümler ziyaret edilene kadar işlemi devam ettir.

### Kullanım Alanları

- Harita ve Navigasyon Sistemleri:** İki nokta arasındaki en kısa yolu bulmada kullanılır.
- Ağ Yönlendirme Protokolleri:** Ağ trafiğini optimize etmek için kullanılır.
- Oyun Geliştirme:** Karakter veya nesnelerin hedefe ulaşması için en kısa yolu belirler.

**Dijkstra Algoritması**, negatif ağırlıklı kenarlara sahip grafiklerde çalışmaz. Bu durumda Bellman-Ford gibi başka algoritmalar tercih edilmelidir.

## Dijkstra Algoritması





# Graf Algoritmaları

## Bellman-Ford Algoritması

**Bellman-Ford algoritması**, bir kaynak düğümden tüm düğümlere olan en kısa yolları bulmak için kullanılır. En önemli özelliği, negatif ağırlıklı kenarları işleyebilmesi ve negatif döngüleri tespit edebilmesidir. Bu yönüyle Dijkstra algoritmasından farklıdır.

### Temel Özellikler

- Negatif ağırlıklı kenarları işleyebilir.
- Negatif döngüleri tespit edebilir.
- Zaman karmaşıklığı:  $O(V \times E)$

### Algoritmanın Adımları

#### 1. Başlangıç:

- Tüm düğümlerin mesafesi sonsuz ( $\infty$ ) olarak ayarlanır.
- Kaynak düğümün mesafesi 0 yapılır.

#### 2. V-1 Kez Tüm Kenarları Gezin:

- Her kenar için (u, v, w):
  - Eğer  $mesafe[u] + w < mesafe[v]$  ise,  $mesafe[v]$  güncellenir.

#### 3. Negatif Döngü Kontrolü:

- V-1 turdan sonra hâlâ bir kenarda güncelleme oluyorsa, graf negatif döngü içeriyor demektir.

### Örnek

Aşağıdaki grafi ele alalım:

A  $--(5)-->$  B  $--(2)-->$  C  $--(-1)-->$  A

Başlangıç: A

Mesafeler: A = 0, B =  $\infty$ , C =  $\infty$

#### 1. Tur:

- A  $\rightarrow$  B  $\rightarrow$  B = 5
- B  $\rightarrow$  C  $\rightarrow$  C = 7
- C  $\rightarrow$  A  $\rightarrow$  A = 6 (mesafe artıyor)

#### 2. Tur:

- Güncellemeler devam ediyorsa, negatif döngü vardır.

### Kullanım Alanları

- Finansal sistemlerde döngüsel arbitraj tespiti
- Ağ yönlendirme protokolleri (ör. RIP - Routing Information Protocol)
- Zaman bazlı optimizasyon problemleri
- Negatif maliyetli yolların analiz edilmesi gereken ulaşım ağları
- Blockchain konsensüs algoritmalarında ağırlıklı graf yapılarında doğrulama

### Sonuç

**Bellman-Ford algoritması**, negatif ağırlıklı grafiklerde etkili bir çözümdür. Dijkstra'dan daha yavaş çalışmasına rağmen, negatif döngü kontrolü ile öne çıkar. Özellikle bu tür durumların kontrolü gereken uygulamalarda tercih edilir.



# Graf Algoritmaları

## Floyd-Warshall Algoritması

**Floyd-Warshall algoritması**, bir graf üzerindeki tüm düğümler arasındaki en kısa yolları bulmak için kullanılan bir algoritmadır. Dinamik programlama yaklaşımına dayanır ve negatif ağırlıklı kenarlar ile negatif döngüleri tespit edebilir.

### Temel Özellikler

- Tüm düğüm çiftleri için en kısa yolu bulur.
- Negatif ağırlıklı kenarları işleyebilir.
- Negatif ağırlıklı döngüleri tespit edebilir.
- Zaman karmaşıklığı:  $O(V^3)$

### Algoritmanın Adımları

#### 1. Başlangıç:

- Her düğüm çifti için mesafeleri belirle:
  - Eğer doğrudan bir kenar varsa, mesafeyi kenarın ağırlığı yap.
  - Kenar yoksa, mesafeyi sonsuz ( $\infty$ ) yap.
  - Bir düğümün kendisine olan mesafe sıfırdır.

#### 2. Üçlü Döngü:

- Her  $k$  düğümü için:
  - Her  $u$  düğümü için:
    - Her  $v$  düğümü için:
      - Eğer  $mesafe[u][k] + mesafe[k][v] < mesafe[u][v]$  ise,  $mesafe[u][v]$  güncellenir.

#### 3. Negatif Döngü Kontrolü:

- Eğer herhangi bir düğümün kendi kendine olan mesafesi negatifse, graf negatif döngü içeriyor demektir.

### Örnek

**Graf:**  $A \xrightarrow{(3)} B \xrightarrow{(1)} C \xrightarrow{(2)} A$

### Başlangıç mesafeleri:

- $A \rightarrow B$ : 3
- $B \rightarrow C$ : 1
- $C \rightarrow A$ : 2

**Floyd-Warshall algoritması**, her düğüm üzerinden geçerek daha kısa yollar arar. Eğer bir düğüm çifti için daha kısa bir yol bulunursa, ilgili mesafe güncellenir.

### Kullanım Alanları

- Tüm çiftler arasındaki en kısa yolun gerekli olduğu uygulamalar
- Ağlar arası gecikme ve maliyet analizleri
- Yol planlama ve ulaşım sistemlerinde rota hesaplamaları
- Sosyal ağ analizlerinde ilişki uzaklıklarının belirlenmesi
- Yük dengeleme ve iletişim gecikmelerinin optimize edilmesi gereken paralel sistemler

### Sonuç

**Floyd-Warshall algoritması**, tüm düğümler arasındaki en kısa yolları bulmak için güçlü bir yöntemdir. Ancak  $O(V^3)$  zaman karmaşıklığı nedeniyle büyük grafiklerde yavaş çalışabilir. Negatif ağırlıklı kenarlar ve negatif döngüler için güvenilir bir çözümdür.



# Graf Algoritmaları

## Kruskal's Algoritması

**Kruskal's algoritması**, bir grafın minimum spanning tree (MST) yani minimum gerilim ağacını bulmak için kullanılan bir algoritmadır. Bu algoritma, grafın kenarlarını sıralayarak ve en küçük ağırlıklı kenarları seçerek ağacın inşa edilmesini sağlar.

### Temel Özellikler

- **Minimum Spanning Tree (MST):** Bir grafın minimum spanning tree'si, grafın tüm düğümlerini birbirine bağlayan kenarların toplam ağırlığının en düşük olduğu ağacdır.
- **Ağırlıklı Kenarlar:** Kruskal's algoritması, ağırlıklı kenarlarla çalışan bir algoritmadır.
- **Zaman Karmaşıklığı:**  $O(E \log E)$  — burada E, kenar sayısını temsil eder. Bu karmaşıklık, kenarların sıralanmasının ve küme birleştirme işlemlerinin zaman alması nedeniyle bu kadar yüksektir.

### Algoritmanın Çalışma Prensipleri

1. **Kenarları Sıralama:** İlk olarak, grafın kenarları ağırlıklarına göre sıralanır.
2. **Küme Birleştirme:** Ardından, her kenar için, o kenarın başlangıç ve bitiş düğümleri aynı kümeye ait değilse, o kenar ağaca eklenir. Kenar ekledikçe, bu iki kümeyi birleştiririz.
3. **Ağaç Tamamlanması:** Tüm düğümler birbirine bağlanana kadar bu işlem devam eder.

### Adım Adım Algoritma

1. **Başlangıç:**
  - Kenarları ağırlıklarına göre küçükten büyüğe sıralayın.
2. **Kenarlara Bakmak:**
  - En küçük ağırlıklı kenarı alarak, başlangıç ve bitiş düğümlerinin aynı kümede olmadığından emin olun. Eğer değilse, bu kenarı seçin ve her iki kümeyi birleştirin.
3. **Sonuç:**
  - Sonuç olarak, minimum spanning tree'yi (MST) elde edersiniz.

### Örnek

Aşağıdaki gibi bir grafi ele alalım:

A --(1)--> B --(4)--> C --(2)--> D --(3)--> A

İlk adımda kenarları sıralayalım:

- A-B: 1
- C-D: 2
- D-A: 3
- B-C: 4

Daha sonra, sıralanan kenarları birleştirerek minimum spanning tree'yi oluştururuz.

### Kullanım Alanları

- **Ağ tasarımı:** En az maliyetle bilgisayar, elektrik veya su ağlarının kurulumu
- **Kablo döşeme sistemleri:** Minimum uzunlukta bağlantı yollarının belirlenmesi
- **Harita ve yol planlama:** Şehirleri birbirine bağlayan minimum maliyetli yollar
- Kümelenme algoritmalarında veri gruplama

### Sonuç

**Kruskal's algoritması**, her zaman minimum spanning tree'yi bulur ve her kenarı yalnızca bir kez inceler. Bu algoritma, özellikle ağaç yapıları ve ağlar ile ilgili problemlerde çok kullanışlıdır. Ancak, algoritmanın verimli çalışabilmesi için grafın kenarlarının sıralanması gerektiğinden, kenar sayısının fazla olduğu durumlarda zaman karmaşıklığı artabilir.



# Graf Algoritmaları

## Prim's Algorithm

**Prim's Algorithm**, bir bağlantılı grafın minimum yayılma ağacını (MST - Minimum Spanning Tree) bulmak için kullanılan bir algoritmadır. Bu algoritma, başlangıç noktasından başlayarak en kısa kenarları ekleyerek ağacın tamamını oluşturur.

### Temel Adımlar

- Başlangıç Düğümünü Seçme:** Algoritma, herhangi bir düğümü başlangıç olarak seçer.
- Kenarı Seçme:** Başlangıç noktasından, en küçük ağırlıklı kenarı seçer ve bu kenar ile bir düğümü daha ağaca dahil eder.
- Yeni Düğümleri Ağaca Dahil Etme:** Her seferinde ağaca en kısa kenarı ekleyerek tüm düğümler ağaçta yer alana kadar devam eder.
- Sonuç:** Algoritma, minimum yayılma ağacını oluşturduğunda sona erer.

### Örnek

Düşünün ki bir grafınız var ve ağırlıklı kenarlara sahip. Prim's algorithm ile en küçük toplam ağırlığı olan bağlantıları seçerek tüm grafiği bir ağaç şeklinde birbirine bağlarsınız.

### Önemli Özellikler

- Ağırlıklı Bağlantılar:** Kenarların her biri bir ağırlığa sahiptir ve bu ağırlıklar, ağacın toplam maliyetini etkiler.
- Tek Seçim:** Her adımda, ağaca ekleyeceğiniz kenar yalnızca bir tane olacaktır ve bu kenar, şu anki ağacın dışında olup en düşük ağırlığa sahip olan kenar olacaktır.
- Zaman Karmaşıklığı:** Verimli bir şekilde uygulanması durumunda, Prim's algoritması genellikle  $O(E \log V)$  zaman karmaşıklığına sahiptir ( $V$  düğüm sayısı,  $E$  kenar sayısı).

### Kullanım Alanları

- Karmaşık Ağaç Yapıları:** Örneğin, ağ iletişimi, elektrik hatları ya da şehirlerarası yolların inşası gibi.
- Optimizasyon Problemleri:** Minimum maliyetli bağlantılar kurma problemlerinde kullanılır.

### Algoritmanın Pseudocode'u

- Başlangıçta bir düğüm seç.
- Kenarları sırayla kontrol et ve en küçük ağırlığa sahip olanı seç.
- Seçilen kenarı ağaca ekle ve yeni düğüme geç.
- Adım 2 ve 3'ü, tüm düğümler ağaca eklenene kadar tekrar et.

### Avantajları

- ✓ Basit ve etkili bir çözüm sunar.
- ✓ Güçlü bir ağ yapılandırma aracıdır.

### Dezavantajları

- ✗ Çok büyük graf yapılarında verimsiz olabilir.
- ✗ Her kenarın ağırlıklarını dikkatlice incelemek gerekir.

Bu algoritma, çok sayıda uygulamada kullanılabilir ve özellikle ağ mühendisliği, yol ağları ve veri yapıları gibi alanlarda yaygın bir şekilde tercih edilir.



# Graf Algoritmaları

## Topolojik Sıralama (Topological Sorting)

**Topolojik sıralama**, yalnızca yönlendirilmiş asiklik grafikler (DAG – Directed Acyclic Graph) üzerinde uygulanabilen bir sıralama algoritmasıdır. Her kenarın yönüne saygı göstererek düğümleri sıralar.

### Temel Kavramlar

- DAG (Directed Acyclic Graph): Yönlü, ancak döngü içermeyen graf yapısıdır.
- Topolojik Sıralama: Her düğüm, yalnızca kendisinden önce gelmesi gereken düğümler sıralandıktan sonra yerleştirilir.

### Algoritma Adımları

1. **Başlangıç:** Kenarı olmayan (bağımsız) düğümler belirlenir.
2. **Sıralama:** Bağımsız düğüm sıralanır ve ona bağlı kenarlar kaldırılır.
3. **Tekrar:** Yeni bağımsız düğümler sıralamaya eklenir.
4. **Devam:** Tüm düğümler sıralanana kadar işlem tekrarlanır.

### Örnek

Bir okulda, bazı derslerin diğerlerinden önce alınması gerekiyorsa (örneğin, Fizik → Matematik), bu durum topolojik sıralama ile modellenenebilir.

### Özellikler

- Döngüsüzlük: Sadece döngüsüz graf yapılarında çalışır.
- Çoklu Çözüm: Birden fazla geçerli sıralama olabilir.
- Zaman Karmaşıklığı:  $O(V + E)$  – V: düğüm sayısı, E: kenar sayısı

### Pseudocode

1. Tüm düğümlerin giriş derecesini hesapla.
2. Giriş derecesi sıfır olan düğümleri sıraya al.
3. Sıradan düğüm çıkar, sıralamaya ekle.
4. Komşu düğümlerin giriş derecesini azalt.
5. Yeni sıfır girişli düğümleri sıraya ekle.
6. Tüm düğümler sıralanana kadar devam et.

### Avantajları

- ✓ Basit ve mantıklı bir sıralama yapısı
- ✓ Görev bağımlılıklarını doğru şekilde yönetir

### Dezavantajları

- ✗ Döngü varsa çalışmaz
- ✗ Büyük grafiklerde işlem süresi uzayabilir

### Kullanım Alanları

- Yazılım bağımlılık çözümü (build sistemleri)
- Görev planlama ve yönetimi
- Veritabanı güncellemeleri
- İşlem sıralamaları

### Sonuç

Topolojik sıralama, görevlerin veya işlemlerin belirli bir sıraya göre yürütülmesi gereken her durumda güçlü ve pratik bir araçtır.



# Graf Algoritmaları

## Ford-Fulkerson Algoritması

**Ford-Fulkerson algoritması**, bir akış ağında kaynak (source) düğümünden hedef (sink) düğümüne ulaşan maksimum akışı bulmak için kullanılır. Genellikle su, veri, enerji gibi taşıma gerektiren sistemlerde uygulanır.

### Temel Kavramlar

- **Akış Ağı:** Kapasiteli yönlü bir graf (her kenarın taşıma sınırı vardır).
- **Maksimum Akış:** Kaynaktan hedefe yönlendirilebilecek en yüksek miktar.
- **Artık Yol (Residual Path):** Halen kapasitesi olan, kullanılabilir bir yol.

### Algoritma Adımları

1. **Başlangıç:** Tüm kenarlardaki akış = 0.
2. **Artık Yol Bul:** Kaynaktan hedefe kapasiteye sahip bir yol bulunur. (DFS veya BFS)
3. **Akışı Artır:** Yol üzerindeki minimum kapasite kadar akış eklenir.
4. **Yolu Güncelle:** Ters yönde kapasite oluşturulur (geri akış için).
5. **Tekrarla:** Artık yol bulunamadığında algoritma biter.

### Örnek Uygulama

Bir su boru hattı sisteminde, kaynak noktadan hedefe ne kadar su taşınabileceğini bulmak için kullanılır. Her boru bir kenardır, kapasite borunun taşıyabileceği maksimum su miktarıdır.

### Zaman Karmaşıklığı

- $O(E \times F)$
- $E$  = Kenar sayısı
- $F$  = Maksimum akış
- (DFS kullanıldığında bu karmaşıklık oluşur.)

### Pseudocode

1. Başlangıçta tüm kenarlardaki akış 0 olarak ayarlanır.
2. Kaynaktan hedefe kapasitesi kalan bir yol bulunur.
3. Yolun minimum kapasitesi kadar akış eklenir.
4. Artık graf güncellenir.
5. Artık yol kalmayana kadar adımlar tekrarlanır.
6. Toplam akış değeri döndürülür.

### Avantajları

- ✓ Maksimum akışı doğru şekilde bulur
- ✓ Gerçek hayattaki akış problemlerinde doğrudan uygulanabilir
- ✓ DFS veya BFS ile esnek kullanım sunar

### Dezavantajları

- ✗ Büyük ağlarda yavaş olabilir
- ✗ Uygun olmayan yol seçiminde verim düşebilir
- ✗ Kesirli kapasitelere sahip ağlar için özel varyantlar gerekebilir

### Kullanım Alanları

- Ağ üzerinden veri transferi
- Su / elektrik dağıtım sistemleri
- Karar destek sistemlerinde kaynak optimizasyonu
- İletişim ağı planlaması



# Dinamik Programlama Algoritmaları

## Fibonacci Sequence Algorithm (Dynamic Programming Yaklaşımı)

### Fibonacci Serisi Nedir?

Fibonacci serisi, her sayının kendisinden önce gelen iki sayının toplamı olduğu bir sayı dizisidir. Serinin ilk iki elemanı genellikle 0 ve 1 olarak kabul edilir.

Örnek Fibonacci serisi: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

### Dynamic Programming ile Fibonacci Hesaplama

Dynamic Programming (DP), önceki hesaplamaları saklayarak tekrar eden hesaplamaları önleyen bir tekniktir.

Fibonacci serisini hesaplarken DP kullanmanın iki temel yolu vardır:

#### 1. Memoization (Üstten Aşağı Yöntem)

- Açıklama: Hesaplanan Fibonacci değerlerini bir dizi veya sözlükte saklayarak, tekrar hesaplanmasını engeller.
- Avantaj: Gereksiz hesaplamaları önler ve performansı artırır.
- Dezavantaj: Ekstra bellek kullanımı gerektirir.

#### 2. Tabulation (Alttan Yukarı Yöntem)

- Açıklama: Küçük alt problemlerin çözülmesiyle büyük probleme ulaşılır.
- Avantaj: Bellek kullanımını optimize eder.
- Dezavantaj: Önce tüm küçük alt problemlerin hesaplanması gerekir.

### Neden Dynamic Programming Kullanılır?

- Klasik rekürsif yöntem, aynı değeri birçok kez hesapladığı için verimsizdir.
- DP kullanımı, hesaplamaları saklayarak gereksiz işlemleri ortadan kaldırır.
- Büyük giriş değerlerinde zaman karmaşıklığını azaltarak performansı artırır.

### Kullanım Alanları

- **Dinamik Programlama Problemleri:** DP öğrenmek ve uygulamak için kullanılır.
- **Optimizasyon Sorunları:** Zaman ve maliyet minimizasyonu için temel oluşturur.
- **Biyoloji ve Genetik:** Bitki büyümesi gibi doğa olaylarını modellemede kullanılır.
- **Yapay Zeka:** Karar verme ve algoritma optimizasyonu için faydalıdır.
- **Ağ ve Veri Yapıları:** Ağ yol optimizasyonlarında kullanılır.
- **Hesaplama Teorisi:** Hesaplama problemlerinde performans iyileştirmesi sağlar.

### Sonuç

Dynamic Programming kullanarak Fibonacci serisini hesaplamak, hesaplama süresini önemli ölçüde azaltır ve daha büyük sayılar için verimli bir yaklaşım sunar. Özellikle büyük girişlerde, DP'nin sağladığı optimizasyon sayesinde hesaplamalar çok daha hızlı tamamlanır.



# Dinamik Programlama Algoritmaları

## Knapsack Problem Algorithm (Dynamic Programming Yaklaşımı)

### Knapsack Problemi Nedir?

**Knapsack (Sırt Çantası) Problemi**, sınırlı kapasiteli bir çantaya, maksimum değeri elde etmek amacıyla belirli ağırlık ve değerlere sahip nesneleri yerleştirme problemidir. Bu problem, özellikle kaynak tahsisi ve optimizasyon alanlarında kullanılır.

### Problem Tanımı

- **Verilenler:**
  - N adet nesne
  - Her nesnenin bir ağırlığı ve değeri var
  - Knapsack'in (çanta) maksimum taşıyabileceği ağırlık W
- **Amaç:** Seçilen nesnelerin toplam ağırlığı W'yi geçmeyecek şekilde, toplam değeri en büyük olacak kombinasyonu bulmak.

### Dynamic Programming ile Çözüm

Knapsack problemi için kullanılan Dynamic Programming çözümü, Tabulation (Alttan Yukarı Yaklaşım) ile gerçekleştirilir.

#### 1. Durum Tanımı

- $DP[i][w]$  ifadesi, i'inci nesneye kadar bakıldığında ve toplam çanta kapasitesi w olduğunda elde edilebilecek maksimum değeri ifade eder.

#### 2. Geçiş Formülü

- Eğer nesnenin ağırlığı w kapasitesinden küçükse:  $DP[i][w] = \max(DP[i-1][w], Değer[i] + DP[i-1][w - Ağırlık[i]])$
- Aksi halde nesne eklenemez:  $DP[i][w] = DP[i-1][w]$

#### 3. Tablo Doldurma

- İlk satır ve ilk sütun genellikle sıfır olarak başlar.
- Tablodaki her hücre, önceki hesaplamalara dayanarak doldurulur.

### Neden Dynamic Programming Kullanılır?

- Klasik rekürsif çözüm, tekrar eden hesaplamalar nedeniyle zaman açısından verimsizdir.
- Dynamic Programming, hesaplamaları kaydederek tekrar eden işlemleri önler ve verimliliği artırır.
- Zaman Kompleksitesi:  $O(N * W)$  olup, rekürsif yönteme göre çok daha hızlıdır.

### Kullanım Alanları

- **Kaynak tahsisi problemleri:** Sınırlı kaynaklarla maksimum verim elde etmek.
- **Yatırım planlaması:** Belirli bütçeyle en iyi yatırım kombinasyonunu seçmek.
- **Kargo ve depolama:** Maksimum değerli ürünlerin sınırlı alana yerleştirilmesi.
- **Proje yönetimi:** Bütçe veya zaman kısıtlarıyla en değerli görevlerin seçimi.
- **E-ticaret kampanyaları:** Belirli bütçeyle maksimum kazanç sağlayan ürünleri öne çıkarmak.

### Sonuç

**Knapsack Problemi**, optimizasyon problemlerinde sıkça karşılaşılan bir durumdur. Dynamic Programming sayesinde, problem daha verimli ve optimal şekilde çözülebilir.



# Dinamik Programlama Algoritmaları

## LCS Algorithm (Dynamic Programming Yaklaşımı)

### LCS (En Uzun Ortak Alt Dizi) Nedir?

**LCS (Longest Common Subsequence)**, iki dizgi (string) içinde sıraları korunarak ortak olan en uzun alt dizinin bulunması problemidir. Bu problem, biyoinformatik, metin karşılaştırma ve dosya farklılık analizlerinde yaygın olarak kullanılır.

### Problem Tanımı

- **Verilenler:**
  - İki dizgi X ve Y
  - X ve Y içinde sıraları korunarak bulunabilecek en uzun ortak alt dizi bulunmalıdır.
- **Amaç:** En uzun ortak alt dizinin uzunluğunu ve içeriğini belirlemek.

### Dynamic Programming ile Çözüm

**LCS problemi**, rekürsif ve brute-force yöntemlerle çözülebilir, ancak zaman karmaşıklığı çok yüksek olur. Dynamic Programming (DP) ile daha verimli bir çözüm elde edilir.

#### 1. Durum Tanımı

- $DP[i][j]$  ifadesi, X dizisinin ilk i karakteri ile Y dizisinin ilk j karakteri arasındaki en uzun ortak alt dizinin uzunluğunu temsil eder.

#### 2. Geçiş Formülü

- Eğer  $X[i] == Y[j]$  ise:  $DP[i][j] = DP[i-1][j-1] + 1$
- Eğer  $X[i] != Y[j]$  ise:  $DP[i][j] = \max(DP[i-1][j], DP[i][j-1])$

#### 3. Tablo Doldurma

- DP tablosu iki diziye bağlı olarak doldurulur.
- İlk satır ve ilk sütun genellikle 0 ile başlar.
- Tablo doldurulduktan sonra,  $DP[m][n]$  en uzun ortak alt dizinin uzunluğunu verir.

### Neden Dynamic Programming Kullanılır?

- Brute-force yöntem  $O(2^N)$  zaman karmaşıklığına sahipken, DP yöntemi  $O(m*n)$  zaman karmaşıklığıyla çok daha hızlı çalışır.
- Önceki hesaplamaları saklayarak gereksiz tekrarları önler.
- Gerçek dünya problemlerinde (DNA dizilimi, metin karşılaştırma vb.) çok daha verimli bir yaklaşım sunar.

### Kullanım Alanları

- **Metin karşılaştırma:** İki belge ya da dosya arasındaki farkları bulmak (diff araçları).
- **Sürüm kontrol sistemleri:** Git gibi sistemlerde dosya değişikliklerini takip etmek.
- **DNA dizilimi karşılaştırmaları:** Biyoinformatik alanında genetik benzerlikleri tespit etmek.
- **Bilgisayarla eğitim (e-learning):** Öğrencilerin verdiği cevaplarla doğru cevaplar arasındaki benzerliği ölçmek.
- **Veri sıkıştırma algoritmaları:** Ortak alt dizileri tespit ederek veriyi daha iyi sıkıştırmak.

### Sonuç

**LCS problemi**, Dynamic Programming kullanılarak verimli bir şekilde çözülebilir. En uzun ortak alt dizi, oluşturulan DP tablosu kullanılarak geri izleme (backtracking) yöntemiyle elde edilebilir.



# Dinamik Programlama Algoritmaları

## Longest Increasing Subsequence (LIS) Algorithm

### LIS (En Uzun Artan Alt Dizi) Nedir?

**LIS (Longest Increasing Subsequence)**, verilen bir dizide sıralı ve artan şekilde bulunan en uzun alt diziyi bulma problemidir. Bu problem, sıralama, analiz ve veri işleme alanlarında önemli bir yer tutar.

### Problem Tanımı

- **Verilenler:**
  - $n$  elemanlı bir dizi
  - Dizi içinde sıralı ve artan şekilde bulunan en uzun alt diziyi bulmamız gerekiyor.
- **Amaç:** Bu alt dizinin uzunluğunu ve içeriğini belirlemek.

### Dynamic Programming ile Çözüm

**LIS problemi**, brute-force yöntemiyle  $O(2^N)$  zaman karmaşıklığında çözülebilir, ancak Dynamic Programming ile  $O(N^2)$  gibi daha verimli bir çözüme ulaşılabilir.

#### 1. Durum Tanımı

- $DP[i]$  ifadesi,  $i$ 'inci elemana kadar olan en uzun artan alt dizinin uzunluğunu temsil eder.

#### 2. Geçiş Formülü

- Her eleman için, kendisinden önceki tüm elemanlar kontrol edilir ve şu şekilde güncellenir:  $DP[i] = \max(DP[j] + 1)$  (Eğer  $dizi[i] > dizi[j]$  ve  $j < i$  ise)
- Başlangıçta tüm elemanlar için  $DP[i] = 1$  olarak başlatılır.

#### 3. Tablo Doldurma

- Her eleman için önceki elemanlara bakılarak DP dizisi doldurulur.
- $\max(DP[i])$  işlemi ile en uzun alt dizinin uzunluğu bulunur.

### Neden Dynamic Programming Kullanılır?

- Brute-force yöntem çok fazla tekrar eden hesaplama yaparken, DP yöntemi önceki hesaplamaları saklayarak daha verimli bir yaklaşım sunar.
- $O(N^2)$  zaman karmaşıklığı,  $O(N \log N)$  çözümüne göre daha yavaş olsa da, DP yöntemi açıklık ve kullanım açısından daha yaygındır.
- Gerçek dünya problemlerinde, veri analizi ve optimizasyon süreçlerinde kullanılabilir.

### Kullanım Alanları

- **Veri analizi:** Artan trendleri belirleme ve veri temizleme
- **Finansal analiz:** Hisse senedi gibi zaman serilerinde artan değerleri bulma
- **Makine öğrenmesi:** Özellik seçimi veya zaman serisi içindeki model örüntülerini yakalama
- **Oyun geliştirme ve animasyon:** Skor veya ilerleme artışı gibi mekanizmaların kontrolü
- **Biyoinformatik:** DNA dizilerindeki mutasyonların takibi

### Sonuç

**LIS problemi**, Dynamic Programming kullanılarak verimli bir şekilde çözülebilir. DP tablosu kullanılarak en uzun artan alt dizi elde edilebilir ve gerektiğinde dizi elemanları geri izleme (backtracking) ile bulunabilir.



# Dinamik Programlama Algoritmaları

## Coin Change Problem (Dynamic Programming Yaklaşımı)

### Coin Change Problemi Nedir?

Coin Change Problemi, verilen farklı para birimleriyle belirli bir miktarı oluşturmak için en az kaç tane para gerektiğini bulma problemidir. Bu problem, finansal sistemler ve optimizasyon alanlarında önemli bir rol oynar.

### Problem Tanımı

- **Verilenler:**
  - Farklı para birimlerini içeren bir dizi (paralar)
  - Hedef toplam (hedef)
- **Amaç:** Hedef toplamı oluşturmak için kullanılabilecek minimum sayıda para miktarını bulmak.

### Dynamic Programming ile Çözüm

**Coin Change problemi**, brute-force yöntemiyle  $O(2^N)$  zaman karmaşıklığında çözülebilir, ancak Dynamic Programming ile  $O(N * M)$  gibi daha verimli bir çözüme ulaşılabilir.

#### 1. Durum Tanımı

- $DP[i]$  ifadesi,  $i$  değerini elde etmek için gereken minimum para sayısını temsil eder.

#### 2. Geçiş Formülü

- $DP[0] = 0$  başlangıç değeri olarak atanır.
- $DP[i]$  hesaplanırken, her para birimi için şu formül kullanılır:  $DP[i] = \min(DP[i], DP[i - para] + 1)$
- Eğer  $i - para \geq 0$  koşulu sağlanıyorsa bu hesaplama yapılır.

#### 3. Tablo Doldurma

- İlk olarak, DP dizisi  $\infty$  (sonsuz) ile doldurulur.
- $DP[0] = 0$  olarak atanır.
- Daha sonra, tüm paralar üzerinden geçilerek DP dizisi güncellenir.
- Sonuç olarak,  $DP[hedef]$  minimum para sayısını verir.

### Neden Dynamic Programming Kullanılır?

- Brute-force yöntem aşırı sayıda tekrar eden hesaplama yaparken, DP yöntemi önceki hesaplamaları saklayarak daha verimli bir yaklaşım sunar.
- $O(N*M)$  zaman karmaşıklığı, çok büyük değerler için bile hesaplamayı hızlandırır.
- Gerçek dünya problemlerinde, ödeme sistemleri ve optimizasyon süreçlerinde uygulanabilir.

### Kullanım Alanları

- **Finansal sistemler:** Bozuk para makineleri, otomatik ödeme sistemleri
- **Banka yazılımları:** En az işlemle para çekme veya bozdurma
- **Mobil ödeme uygulamaları:** Minimum işlem adımıyla ödeme tutarlarını bölme
- **Oyun ekonomisi:** Sanal para sistemlerinde denge sağlama
- **Optimizasyon:** Paketleme, kaynak tahsisi, sınırlı stok problemi gibi alanlarda da benzer prensipler kullanılır.

### Sonuç

**Coin Change problemi**, Dynamic Programming kullanılarak verimli bir şekilde çözülebilir. DP tablosu kullanılarak minimum kaç para gerektiği bulunabilir ve gerektiğinde kullanılan paraların kombinasyonu geri izleme (backtracking) ile elde edilebilir.



# Dinamik Programlama Algoritmaları

## Matris Zinciri Çarpımı (Matrix Chain Multiplication)

### Problem Tanımı

Verilen bir matris dizisini en verimli şekilde çarpmak istiyoruz. Matris çarpımı birleşimlidir, yani işlem sırası sonucu değiştirmez ancak işlem maliyetini büyük ölçüde etkiler. Amaç, en az skaler çarpma sayısı ile matrisleri çarpmaktır.

### Örnek Problem

3 matrisimiz olsun:

- $A_1$ :  $10 \times 20$
- $A_2$ :  $20 \times 30$
- $A_3$ :  $30 \times 40$

Farklı parantezleme sıraları:

- $((A_1 \times A_2) \times A_3)$
- $(A_1 \times (A_2 \times A_3))$
- Her seçenek farklı işlem sayısına neden olur.

### Yaklaşım: Dinamik Programlama

Bu problem, alt problemleri çözerek daha büyük problemi çözme ilkesine dayanır.

### Adım Adım Çözüm

1. Boyut Dizisi
2. Matrislerin boyutları:
3.  $[p_0, p_1, \dots, p_n]$ 
  - $A_i$  matrisinin boyutu:  $p_{i-1} \times p_i$
4. DP Tablosu ( $dp[i][j]$ )
5.  $A_i$  ile  $A_j$  arası matrislerin çarpılması için gereken minimum skaler çarpma sayısını tutar.
6. Formül
7.  $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + p_{i-1} \times p_k \times p_j)$
8. Burada  $k$ ,  $i$  ile  $j$  arasındaki bölme noktalarıdır.
9. Temel Durum
10. Tek bir matris çarpmaya gerek olmadığı için:
11.  $dp[i][i] = 0$
12. İteratif Hesaplama
13. Zincir uzunluklarına göre tabloyu doldur.
14. Daha kısa zincirlerden başlayarak daha uzun zincirlere geç.
15. Sonuç
16. Minimum maliyet:  $dp[1][n-1]$
17. ( $n$ : matris sayısı + 1)

**Örnek:** Boyut dizisi:  $[10, 20, 30, 40]$

3 matris için minimum skaler çarpma sayısı dinamik programlama ile hesaplanır.

### Avantajları

- ✓ Mükemmel alt problem yapısı
- ✓ Tekrar eden hesaplamaları önler
- ✓ Gerçek dünya uygulamalarında performansı artırır

### Sonuç

**Matris Zinciri Çarpımı problemi**, dinamik programlamanın temel örneklerinden biridir. Karmaşık hesaplamaları en aza indirerek matris çarpımını en verimli şekilde yapmayı sağlar.



# Dinamik Programlama Algoritmaları

## Rod Cutting Problemi - Dinamik Programlama

### Problemin Genel Tanımı

**Rod Cutting problemi**, bir çubuğu (veya çubuğun belirli bir uzunluktaki parçalarını) keserek en yüksek karı elde etmeye çalıştığınız bir optimizasyon problemidir. Bu problem genellikle bir çubuğun fiyatları verilerek, bu çubuğun uzunluklarına göre maksimum karı elde etmek için hangi parçalarla kesileceğinin belirlenmesini içerir.

Bir çubuğun belirli uzunluktaki kesitlerinin fiyatları verilir ve amacı, bu çubuğu en verimli şekilde kesmek, yani toplam gelir (kar) miktarını en üst düzeye çıkarmaktır.

### Problem Tanımı

- Bir çubuğun uzunluğu  $n$  verilmiştir.
- Çubuğun her bir uzunluğu için bir fiyat listesi verilmiştir.
- Amaç, çubuğu farklı parçalara keserek toplam karı en yüksek şekilde elde etmektir.

### Örnek olarak:

- Çubuğun fiyatları:  $p[1] = 2$ ,  $p[2] = 5$ ,  $p[3] = 7$ ,  $p[4] = 8$
- Çubuğun toplam uzunluğu  $n = 4$ .

Bu durumda, 4 birim uzunluğunda bir çubuğu kesmek için farklı stratejiler olabilir.

### Yaklaşım: Dinamik Programlama

**Rod Cutting problemi**, dinamik programlama ile çözülebilecek klasik bir optimizasyon problemidir. Bu problem, büyük bir problemi daha küçük parçalara ayırarak çözme prensibine dayanır.

### Adımlar

- 1. Fiyat Listesini Tanımlama:** Çubuğun farklı uzunluktaki parçalarının fiyatlarını bir dizi olarak tanımlayalım:  $p[1]$ ,  $p[2]$ , ...,  $p[n]$ .
- 2. Maliyet Matrisi:**  $dp[i]$ , uzunluğu  $i$  olan çubuğun kesilmesiyle elde edilecek maksimum karı saklar.
- 3. Dinamik Programlama İterasyonu:** Her uzunluk için, çubuğu farklı parçalara kesmenin en yüksek karını hesaplamak için önceki alt problemlerin sonuçlarını kullanırız.
- 4. Kar Hesaplama:** Her alt problemin çözümünü kullanarak, uzunluğu  $n$  olan çubuğun kesilmesiyle elde edilecek maksimum karı hesaplarız.
- 5. Sonuç:** Sonuç,  $dp[n]$ 'de saklanır ve bu değer çubuğun tamamının kesilmesiyle elde edilecek en yüksek karı gösterir.

### Temel Durum

Başlangıçta, hiçbir uzunluk için kar olmadığı kabul edilir. Yani  $dp[0] = 0$ .

### Örnek

Fiyat listesi ve çubuğun uzunluğu şu şekilde verilmiş olsun:

- $p[1] = 2$ ,  $p[2] = 5$ ,  $p[3] = 7$ ,  $p[4] = 8$
- Çubuğun uzunluğu:  $n = 4$

Bu durumda, en yüksek karı elde etmek için 2 birimlik 2 parça almak en karlı çözüm olacaktır.

### Sonuç

**Rod Cutting problemi**, dinamik programlamayı kullanarak çözülebilecek klasik bir optimizasyon problemidir. Çubuğun her uzunluğu için en iyi karı elde etmek için alt problemlerin çözümleri birleştirilir. Bu yöntem, birçok benzer optimizasyon probleminin çözümünde de kullanılabilir.



# Dinamik Programlama Algoritmaları

## Edit Distance (Levenshtein Distance) - Dynamic Programming

### Problem Geneli Tanımı

Edit Distance veya Levenshtein Distance problemi, iki kelime arasındaki farkı ölçmek amacıyla kullanılır. Bu fark, bir kelimeyi diğerine dönüştürmek için gereken minimum edit işlemleri sayısıdır.

### Edit işlemleri şunlardır:

- 1. Ekleme:** Bir karakteri eklemek.
- 2. Silme:** Bir karakteri silmek.
- 3. Değiştirme:** Bir karakteri başka bir karakterle değiştirmek.

**Amaç,** iki kelimeyi birbirine dönüştürmek için yapılan bu işlemlerin minimum sayısını bulmaktır.

### Problem Tanımı

Verilen iki kelime A ve B, bu kelimeleri birbirine dönüştürmek için yapılacak minimum işlemler sayısını bulmak hedeflenir.

### Örnek olarak:

A = "kitten"

B = "sitting"

Bu durumda minimum edit mesafesi hesaplanarak, kelimeler arasındaki fark bulunur.

### Dynamic Programming ile Çözüm

**Levenshtein Distance problemi,** dinamik programlama ile çözülebilecek bir problemdir. Bu problemde, her iki kelimenin her karakteri için alt problemlerin çözümlerini kullanarak toplam sonucu elde ederiz.

### Adımlar

#### 1. İlk Durum:

- İki kelimenin boş olması durumunda mesafe sıfırdır. Eğer bir kelime boşsa, diğer kelimenin tüm karakterlerini eklemek gerekecektir.

#### 2. Maliyet Matrisi:

- İki kelimenin her karakteri için bir tablo (matris) oluşturulur. Bu matrisin her hücresi, o noktaya kadar yapılan edit işlemlerinin toplam sayısını saklar.

#### 3. Dinamik Programlama İterasyonu:

- Matris her adımda hesaplanır. Eğer kelimenin karakterleri eşleşiyorsa, o hücredeki değer bir önceki hücreye kopyalanır. Eğer eşleşmiyorsa, o hücredeki değer, ekleme, silme veya değiştirme işleminin minimum maliyeti ile güncellenir.

#### 4. Sonuç:

- Sonuç, matrisin sağ alt köşesinde bulunur. Bu, iki kelime arasındaki minimum edit mesafesini temsil eder.

### Temel Durum

Başlangıçta, bir kelimenin boşken diğer kelimeyi elde etmek için gereken işlem sayısı hesaplanır.



# Dinamik Programlama Algoritmaları

Örnek olarak:

A = "kitten"

B = "sitting"

Bu durumda, minimum edit mesafesi şu adımlarla hesaplanır:

1. "k" ve "s" farklı, değiştirilir.
2. "i" ve "i" aynı, değiştirilmez.
3. "t" ve "t" aynı, değiştirilmez.
4. "t" ve "t" aynı, değiştirilmez.
5. "e" ve "i" farklı, değiştirilir.
6. "n" ve "n" aynı, değiştirilmez.
7. "s" ve "g" farklı, değiştirilir.

Toplamda 3 işlem gereklidir.

## Sonuç

**Levenshtein Distance problemi**, iki kelime arasındaki farkı hesaplamak için dinamik programlamayı kullanan klasik bir problemdir. Bu problem, kelimeleri birbirine dönüştürmek için gereken minimum sayıda edit işlemini bulmak için alt problemlerin çözümlerini birleştirir.

|              |    | Empty String | String "B" |   |   |   |   |   |   |   |
|--------------|----|--------------|------------|---|---|---|---|---|---|---|
|              |    | ""           | A          | L | T | R | U | I | S | M |
| Empty String | "" | 0            | 1          | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| String "A"   | P  | 1            | 1          | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|              | L  | 2            | 2          | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|              | A  | 3            | 2          | 2 | 2 | 3 | 4 | 5 | 6 | 7 |
|              | S  | 4            | 3          | 3 | 3 | 3 | 4 | 5 | 5 | 6 |
|              | M  | 5            | 4          | 4 | 4 | 4 | 4 | 5 | 6 | 5 |
|              | A  | 6            | 5          | 5 | 5 | 5 | 5 | 5 | 6 | 6 |



# Böl ve Fethet Algoritmaları

## Merge Sort Algoritması

**Merge Sort**, "Divide and Conquer" (Böl ve Fethet) stratejisini kullanan etkili bir sıralama algoritmasıdır. Algoritma, bir diziyi küçük parçalara ayırarak sıralar ve ardından bu parçaları birleştirerek sıralı hale getirir.

### Merge Sort'un Çalışma Mantığı

#### 1. Bölme (Divide):

- Dizi, ortadan ikiye bölünerek alt dizilere ayrılır.
- Bu işlem, her alt dizi tek bir elemana sahip olana kadar devam eder.

#### 2. Fethetme (Conquer):

- Tek elemanlı diziler zaten sıralıdır.
- İkili gruplar halinde birleştirilerek sıralı diziler oluşturulur.
- Küçük diziler birleşerek daha büyük sıralı diziler haline getirilir.

#### 3. Birleştirme (Combine):

- Alt diziler, karşılaştırmalar yapılarak birleştirilir ve tam sıralı dizi elde edilir.

### Merge Sort'un Zaman Karmaşıklığı

- En iyi durum:  $O(n \log n)$
- Ortalama durum:  $O(n \log n)$
- En kötü durum:  $O(n \log n)$

Bu nedenle Merge Sort, büyük veri kümeleri için oldukça verimli bir sıralama algoritmasıdır.

### Avantajları

- ✓ **Stabil bir algoritmadır:** Aynı değerlere sahip elemanların sıralanma sırası korunur.
- ✓ Büyük veri kümeleri için etkilidir.
- ✓ Kötü durum senaryolarında bile  $O(n \log n)$  karmaşıklığını korur.

### Dezavantajları

- ✗ **Ekstra bellek gerektirir:** Yeni diziler oluşturulduğu için  $O(n)$  ek bellek kullanımına ihtiyaç duyar.
- ✗ Küçük veri kümeleri için daha yavaş olabilir.

### Kullanım Alanları

- **Büyük veri kümeleri ve dosya sıralama:** Özellikle disk tabanlı algoritmalarda, dış sıralama algoritmalarında kullanılır. Büyük verinin belleğe sığmadığı durumlarda merge sort, dış bellek sıralama (external sorting) için yaygın olarak tercih edilir.
- **Stabil sıralama gerektiren durumlar:** Aynı değerlere sahip öğelerin sıralama sırasının korunması gerektiğinde (örneğin, veritabanlarında kayıtların sıralanması), Merge Sort kullanılır.
- **Zayıf donanımda:** Düşük bellekli sistemlerde, gereksiz bellek kullanımını minimize etmek adına diğer sıralama algoritmalarına göre daha verimli olabilir.
- **Paralel ve Dağıtık Hesaplamalar:** Merge Sort, paralel işleme için uygundur. Verinin bölünmesi ve sıralama işlemleri paralel olarak yapılabilir, bu da büyük veri setleri için performans kazancı sağlar.
- **İleriye dönük sıralama işlemleri:** Merge Sort, sıralanan iki diziyi birleştirirken daha önce sıralı dizilere benzer verileri hızlıca sıralamak için kullanılabilir. Örneğin, birleşim ve sıralama işlemleri sürekli olarak yapılacaksa, Merge Sort çok verimli bir seçenek olabilir.



# Böl ve Fethet Algoritmaları

## Quick Sort Algoritması

**Quick Sort**, "**Divide and Conquer**" (**Böl ve Fethet**) stratejisini kullanan hızlı ve verimli bir sıralama algoritmasıdır. Algoritma, bir pivot (dönel nokta) seçerek diziyi küçük ve büyük olmak üzere iki alt gruba ayırır ve ardından bu alt grupları ayrı ayrı sıralar.

### Quick Sort'un Çalışma Mantiği

#### 1. Pivot Seçimi:

- Diziden rastgele veya belirli bir kurala göre bir pivot eleman seçilir.

#### 2. Bölme (Partitioning):

- Seçilen pivot eleman etrafında dizinin elemanları yeniden düzenlenir.
- Pivot'tan küçük olanlar sol tarafa, büyük olanlar sağ tarafa yerleştirilir.

#### 3. Rekursif Sıralama:

- Sol ve sağ alt diziler için aynı işlem tekrar edilir.
- Alt diziler tek eleman kalana kadar bölünmeye devam eder.

### Quick Sort'un Zaman Karmaşıklığı

- En iyi durum:  $O(n \log n)$
- Ortalama durum:  $O(n \log n)$
- En kötü durum:  $O(n^2)$  (Eğer pivot seçimi kötü yapılırsa)

### Avantajları

- ✓ **Hızlıdır:** Ortalama durumda Merge Sort ile benzer performansa sahiptir.
- ✓ **Ekstra bellek gerektirmez:** Çoğu durumda ek hafıza gerektirmeden çalışır.
- ✓ Pratikte çok verimli bir algoritmadır.

### Dezavantajları

- ✗ Kötü pivot seçimi performansı düşürebilir.
- ✗ **Merge Sort kadar stabil değildir:** Aynı değerlere sahip elemanların sıralanma sırası değişebilir.

### Kullanım Alanları

- **Genel sıralama problemleri:** Hızlı ve verimli sıralama gereken durumlarda, Quick Sort sıklıkla tercih edilir. Özellikle küçük ve orta büyüklükteki veri kümelerinde oldukça etkilidir.
- **Yüksek performans gerektiren uygulamalar:** Veritabanları, arama motorları ve büyük veri analizi gibi yüksek verim gerektiren alanlarda, Quick Sort'un hızlı çalışması büyük avantaj sağlar.
- **İç bellek sıralama:** Quick Sort, diziyi yerinde sıralayabilen ve ekstra bellek kullanmayan bir algoritma olduğu için, iç bellek sıralama işlemlerinde kullanılır.
- **Paralel sıralama:** Quick Sort, paralel işleme için uygun bir algoritmadır. Özellikle büyük veri setlerinde paralel olarak işlenerek performans artırılabilir.
- **Sürekli sıralama gereken durumlar:** Quick Sort, verinin sürekli değiştiği durumlarda (örneğin, gerçek zamanlı sıralama gerektiren uygulamalarda) de verimli bir sıralama algoritması olarak kullanılabilir.



# Böl ve Fethet Algoritmaları

## Binary Search Algoritması

**Binary Search (İkili Arama)**, "Divide and Conquer" (Böl ve Fethet) stratejisini kullanan etkili bir arama algoritmasıdır. Algoritma, sıralı bir dizide belirli bir elemanı bulmak için diziyi sürekli ikiye böler.

### Binary Search'un Çalışma Mantığı

#### 1. Orta Elemanı Bulma:

- Dizinin ortasındaki eleman seçilir.

#### 2. Karşılaştırma:

- Eğer orta eleman aranan değere eşitse, arama tamamlanır.
- Eğer aranan değer ortadaki elemandan küçükse, arama sol yarıda devam eder.
- Eğer aranan değer ortadaki elemandan büyükse, arama sağ yarıda devam eder.

#### 3. Tekrar:

- Arama işlemi, alt diziler üzerinde tekrar edilerek devam eder.
- Dizinin boyutu 0 olduğunda aranan eleman bulunamaz ve işlem sona erer.

### Binary Search'un Zaman Karmaşıklığı

- En iyi durum:  $O(1)$  (Aranan eleman ilk tahminde bulunursa)
- Ortalama durum:  $O(\log n)$
- En kötü durum:  $O(\log n)$

### Avantajları

- ✓ **Çok hızlıdır:** Büyük veri kümelerinde bile  $O(\log n)$  performansı sunar.
- ✓ **Daha az işlem yapar:** Doğrusal arama ( $O(n)$ ) ile karşılaştırıldığında çok daha az karşılaştırma yapar.
- ✓ **Verimli çalışır:** Bellek açısından ek yer kaplamaz.

### Dezavantajları

- ✗ Sadece sıralı dizilerde çalışır.
- ✗ Dizinin sıralanması gerekiyorsa ek maliyet doğurur.

Binary Search, büyük ve sıralı veri kümelerinde oldukça etkili bir arama algoritmasıdır.

### Kullanım Alanları

- **Sıralı veri kümeleri:** Binary Search, sıralı dizilerde verimli arama yapmak için en ideal algoritmadır. Bu, veritabanı sorguları, arama motorları, sıralı listeler gibi durumlarda yaygın olarak kullanılır.
- **Arama gerektiren büyük veri kümeleri:** Veri kümeleri çok büyük olduğunda, doğrusal aramalar oldukça yavaş olabilir. Binary Search bu tür büyük veri kümelerinde hızlı bir şekilde sonuç verir.
- **Veritabanı ve indeksleme sistemleri:** Veritabanlarında sıralı verilerin arandığı yerlerde, Binary Search kullanılarak arama hızları artırılabilir. Ayrıca, sıralı dizilerde indeksleme işlemleri için de kullanılır.
- **Sürekli değişmeyen ve sıralı diziler:** Özellikle verilerin değişmediği ve sıralı olduğu durumlarda, Binary Search çok hızlı aramalar yapabilmek için kullanılır.



# Böl ve Fethet Algoritmaları

## Strassen's Matrix Multiplication Algoritması

**Strassen algoritması, Divide and Conquer (Böl ve Fethet)** yaklaşımını kullanan bir matris çarpma algoritmasıdır. Geleneksel matris çarpma algoritmasına göre daha hızlıdır ve özellikle büyük matrisler için etkilidir.

### Strassen Algoritmasının Çalışma Mantığı

#### 1. Matrisleri Bölme:

- İki  $n \times n$  boyutundaki matrisi,  $n/2 \times n/2$  boyutunda 8 alt matrise ayırır.

#### 2. Yedi Özel Çarpma Yapma:

- Geleneksel yöntem 8 çarpma işlemi yaparken, Strassen 7 özel çarpma kullanır.
- Bu özel çarpma işlemleri şu şekildedir:
  - $M1 = (A11 + A22) * (B11 + B22)$
  - $M2 = (A21 + A22) * B11$
  - $M3 = A11 * (B12 - B22)$
  - $M4 = A22 * (B21 - B11)$
  - $M5 = (A11 + A12) * B22$
  - $M6 = (A21 - A11) * (B11 + B12)$
  - $M7 = (A12 - A22) * (B21 + B22)$

#### 3. Sonuç Matrisi Hesaplama:

- Alt matrisleri toplayarak son matrisi oluşturur:
  - $C11 = M1 + M4 - M5 + M7$
  - $C12 = M3 + M5$
  - $C21 = M2 + M4$
  - $C22 = M1 - M2 + M3 + M6$

#### 4. Rekursif Uygulama:

- Eğer matris boyutu küçükse (örneğin  $2 \times 2$ ), doğrudan çarpma yapılır.
- Büyük matrisler için aynı işlem alt matrislere uygulanarak devam edilir.

### Strassen Algoritmasının Zaman Karmaşıklığı

- Geleneksel matris çarpma:  $O(n^3)$
- Strassen matris çarpma:  $O(n^{2.81})$

### Avantajları

- ✓ Geleneksel çarpma yöntemine göre daha hızlıdır.
- ✓ Büyük matrislerde performans avantajı sağlar.
- ✓ Rekürsif olarak çalıştığı için paralel işlemeye uygundur.

### Dezavantajları

- ✗ Ek bellek kullanımı fazladır.
- ✗ Matrislerin boyutları 2'nin katı olmalıdır.
- ✗ Küçük matrisler için geleneksel yöntem daha verimli olabilir.

Strassen algoritması, büyük boyutlu matris çarpmalarında zaman kazandıran önemli bir algoritmadır.



# Geri İzleme Algoritmaları

## N-Queens Problemi ve Geri İzleme Algoritması

### Problem Tanımı

N-Queens problemi, N adet şahını, N x N boyutunda bir satranç tahtasında öyle yerleştirmeyi amaçlayan klasik bir bulmacadır ki, hiçbir iki şah birbirini tehdit etmemelidir.

### Yani:

- İki şah aynı satırda yer alamaz.
- İki şah aynı sütunda yer alamaz.
- İki şah aynı diyagonal üzerinde yer alamaz.

### Geri İzleme Yaklaşımı

**Geri İzleme**, çözüm adaylarını adım adım oluşturan ve eğer bir aday kısıtları ihlal ederse, önceki adıma geri dönerek yeni bir aday deneyen genel bir algoritma tekniğidir.

### N-Queens Problemini Geri İzleme ile Çözme Adımları

1. **İlk satırdan başla:** İlk satıra bir şah yerleştir, sonra bir sonraki satıra geç ve aynı işlemi tekrarla.
2. **Sütun kısıtı:** Her satır için, şahı öyle bir sütuna yerleştir ki, o sütunda başka bir şah olmasın.
3. **Diyagonal kısıtı:** Her satır için, yerleştirilen şahın başka bir şahla aynı diyagonal üzerinde olmaması sağlanmalıdır.
4. **Geriye doğru adım at:** Bir satıra geçerken uygun bir konum bulamazsan, son yerleştirilen şahı kaldırarak bir önceki satırda yeni bir konum dene.
5. **İşlemi tekrarla:** Her satır için şahları yerleştirmeye devam et, ta ki tüm şahlar tahtaya yerleştirilene kadar.
6. **Çözüm bulundu:** Bütün şahlar, kısıtlamalara uyarak tahtaya yerleştirildiğinde, bu düzen geçerli bir çözüm olacaktır.
7. **Tüm olasılıkları keşfet:** Algoritma, tüm olası çözümleri keşsederek işlemeye devam eder.

### Kullanım Alanları

- **Yapay Zeka:** Oyunlarda hamle üretimi ve çözüm arama.
- **Kısıt Tabanlı Problemler:** Ders programı, vardiya planlaması, Sudoku.
- **Optimizasyon:** Gezgin satıcı, yerleşim düzeni.
- **Biyoinformatik:** DNA dizilim analizi.
- **Güvenlik:** Şifre çözme, anahtar arama.
- **Eğitim:** Algoritma ve problem çözme eğitimi.

### Sonuç

**Geri İzleme algoritması**, N-Queens problemini çözmek için etkili bir yöntemdir. Bu yöntem, tüm olasılıkları adım adım deneyerek, geçerli olmayan bir düzenle karşılaşıldığında geri adım atarak tüm çözümleri keşfeder ve kısıtlamaları korur.



# Geri İzleme Algoritmaları

## Subset Sum Problemi ve Geri İzleme Algoritması

### Problem Tanımı

Subset Sum Problemi, bir dizi tam sayının ve bir hedef toplamın verildiği bir karar problemidir. Amaç, verilen dizinin bir alt kümesinin toplamının hedef toplamla eşit olup olmadığını belirlemektir.

### Örnek

Verilen tam sayılar kümesi {3, 34, 4, 12, 5, 2} ve hedef toplam 9 ise, {4, 5} alt kümesinin toplamı 9 olduğundan, cevap "Evet" olacaktır.

### Geri İzleme Yaklaşımı

Geri izleme, bir problemi çözmek için adayları keşfeden ve geçerli bir çözüme yol açamayacağını fark ettiğinde bu adayları terk eden bir tekniktir.

### Subset Sum Problemini Geri İzleme ile Çözme Adımları

- İlk elemandan başla:** Dizin ilk elemanını göz önünde bulundur.
- Elemanı dahil et ya da etme:** Her eleman için iki seçeneğin vardır:
  - Elemanı dahil et: Elemanı mevcut alt kümeye ekleyip, hedef toplamı o elemanın değeri kadar azalt.
  - Elemanı hariç tut: Elemanı alt kümeye eklemekten bir sonraki elemana geç.
- Toplam hedefe eşit mi kontrol et:** Mevcut alt kümenin toplamı hedef toplam ile eşitse, çözüm bulunmuştur.
- Geriye adım at:** Eğer herhangi bir noktada toplam hedefi aşarsa ya da tüm elemanlar kontrol edildikten sonra çözüm bulunmazsa, son dahil edilen elemanı çıkararak bir önceki adıma geri dön ve bir sonraki olası elemanı dene.
- İşlemi tekrarla:** Bu işlemi, tüm olası kombinasyonları göz önünde bulundurarak tekrarla.
- Çözüm bulundu:** Geçerli bir alt küme bulunduğunda "Evet" olarak dön. Eğer tüm olasılıklar kontrol edildikten sonra geçerli bir alt küme bulunamazsa, "Hayır" dön.

### Kullanım Alanları

- Kısıt Tabanlı Problemler:** Finansal planlama, bütçe sınırlamaları.
- Optimizasyon:** Kaynak dağılımı, iş programlama.
- Şifreleme:** Anahtar bulma ve güvenlik analizleri.
- Eğitim:** Algoritmaların öğretimi ve problem çözme teknikleri.

### Sonuç

**Geri izleme algoritması, Subset Sum Problemini** çözmek için güçlü bir tekniktir. Bu yöntem, dizinin elemanlarının tüm olası kombinasyonlarını keşfederek hedef toplamı sağlayan bir çözüm bulur ve geçersiz yolları etkili bir şekilde terk eder.



# Geri İzleme Algoritmaları

## Sudoku Çözücü ve Geri İzleme Algoritması

### Problem Tanımı

Sudoku, mantık tabanlı bir sayı yerleştirme bulmacasıdır. Amaç, 9x9 boyutunda bir ızgarayı 1'den 9'a kadar sayılarla doldurmak ve aşağıdaki kurallara uymaktır:

- Her satırda 1'den 9'a kadar olan her sayı yalnızca bir kez yer almalıdır.
- Her sütunda 1'den 9'a kadar olan her sayı yalnızca bir kez yer almalıdır.
- 9 adet 3x3 alt ızgaranın her birinde 1'den 9'a kadar olan her sayı yalnızca bir kez yer almalıdır.

Verilen kısmi bir ızgara ile amaç, bu bulmacayı tamamlamaktır.

### Geri İzleme Yaklaşımı

**Geri izleme**, tüm olası çözümleri deneyerek ve kısıtları ihlal edenleri reddederek bir problemi çözme tekniğidir. Sudoku durumunda, boş hücrelere sayılar yerleştirilerek ilerleriz ve eğer şu anki seçim geçersiz bir yapı oluşturursa geri adım atarız.

### Sudoku Bulmacasını Geri İzleme ile Çözme Adımları

1. **İlk boş hücreyle başla:** İlk boş hücreyi (sayı ile doldurulmamış hücre) belirleyin.
2. **Bir sayı yerleştirmeyi dene:** Boş hücreye 1'den 9'a kadar sayılardan birini yerleştirmeyi deneyin.
3. **Kısıtları kontrol et:** Bir sayı yerleştirdikten sonra, bu sayının Sudoku kurallarını ihlal edip etmediğini kontrol edin:
  - Sayı aynı satırda yer alan başka bir sayı ile çakışmamalıdır.
  - Sayı aynı sütunda yer alan başka bir sayı ile çakışmamalıdır.
  - Sayı aynı 3x3 alt ızgarada yer alan başka bir sayı ile çakışmamalıdır.
4. **Geçerliyse bir sonraki boş hücreye geç:** Eğer mevcut yerleştirme geçerliyse, bir sonraki boş hücreye geçin ve işlemi tekrarlayın.
5. **Geriye adım at:** Eğer bir sayı, geçersiz bir konfigürasyona (satır, sütun veya alt ızgara ihlali) yol açarsa, son yerleştirilen sayıyı çıkarın ve o hücre için bir sonraki olası sayıyı deneyin.
6. **İşlemi tekrarla:** Bu işlemi, tüm boş hücreler için geçerli yerleştirmeleri deneyecek şekilde tekrarlayın.
7. **Çözüm bulundu:** Eğer tüm hücreler kurallara uygun şekilde doldurulmuşsa, Sudoku bulmacası çözülmüş demektir.

### Kullanım Alanları

- **Oyun Geliştirme:** Sudoku çözme, oyun motorları ve uygulamalarda kullanılır.
- **Yapay Zeka:** Zeka oyunları çözme teknikleri, AI algoritmalarını geliştirmeye yardımcı olur.
- **Eğitim:** Zeka gelişimi için Sudoku, problem çözme becerilerini artırır.
- **Kriptografi:** Bazı şifreleme yöntemlerinde Sudoku çözme teknikleri kullanılabilir.
- **Optimizasyon:** Sudoku, benzer optimizasyon problemleri için örnek oluşturur.

### Sonuç

**Geri izleme**, Sudoku bulmacalarını çözmek için etkili bir yöntemdir. Bu yöntem, sayılar için tüm olası yerleştirmeleri sistematik olarak keşfeder, her adımda kısıtları kontrol eder ve geçersiz yolları terk ederek geçerli bir çözüm bulur.



# Geri İzleme Algoritmaları

## Graph Coloring (Graf Boyama) Algoritması

### Tanım

**Graph Coloring**, bir grafın düğümlerini (nodes) belirli kurallara göre boyama işlemidir. Temel amaç, komşu düğümlerin aynı renge sahip olmamasını sağlamaktır. Bu problem özellikle zamanlama, frekans atama, harita boyama gibi birçok alanda uygulama bulur.

### Temel Kavramlar

- **Graf (Graph):** Düğümler (nodes) ve bu düğümleri birbirine bağlayan kenarlardan (edges) oluşan yapı.
- **Renk Atama:** Her düğüme bir renk atanır. Komşu düğümler farklı renklerde olmalıdır.
- **Kromatik Sayı (Chromatic Number):** Bir grafı boyamak için gereken minimum renk sayısı.

### Geri İzleme (Backtracking) Algoritması

**Geri izleme algoritması**, Graph Coloring problemine çözüm bulmak için kullanılan bir yöntemdir. Bu yöntem, tüm olası çözümleri sistematik bir şekilde deneyerek, uygun olmayan çözümleri eleyerek ilerler.

### Algoritmanın Adımları

1. **Başlangıç:** Grafın düğümlerini sırayla dolaşmaya başla.
2. **Renk Atama:** Her düğüm için, kullanılabilir renklerden birini dene.
3. **Kontrol Et:** Seçilen renk, düğümün komşularında kullanılmamış olmalıdır.
4. **Uygunluk:** Eğer renk uygunsa, bir sonraki düğüme geç.
5. **Uygun Değilse:** Eğer renk uygun değilse, başka bir renk dene.
6. **Geri İzleme:** Eğer hiçbir renk uygun değilse, bir önceki düğüme geri dön ve farklı bir renk dene.
7. **Çözüm Bulma:** Tüm düğümler başarıyla renklendirildiğinde, çözüm bulunmuş olur.

### Örnek

Bir grafın düğümlerini renklendirmek için 3 renk (örneğin, kırmızı, mavi, yeşil) kullanıldığını varsayalım. Algoritma şu şekilde çalışır:

- İlk düğüme kırmızı renk atanır.
- İkinci düğüm, birinci düğümle komşu ise kırmızı olamaz, bu yüzden mavi atanır.
- Üçüncü düğüm, hem birinci hem de ikinci düğümle komşu ise, kırmızı ve mavi kullanılamaz, bu yüzden yeşil atanır.
- Bu şekilde tüm düğümler renklendirilene kadar devam edilir.

### Avantajları

- ✓ Tüm olası çözümleri denediği için doğru çözümü bulma garantisi vardır.
- ✓ Küçük ve orta boyutlu graflar için etkilidir.

### Dezavantajları

- ✗ Büyük graflar için zaman alıcı olabilir.
- ✗ Karmaşıklığı yüksektir, özellikle renk sayısı ve düğüm sayısı arttıkça performans düşer.

### Sonuç

**Graph Coloring problemi**, geri izleme algoritması kullanılarak etkili bir şekilde çözülebilir. Bu yöntem, özellikle küçük ve orta ölçekli problemler için uygundur. Ancak, büyük ölçekli problemlerde daha verimli algoritmaların kullanılması gerekebilir.



# Geri İzleme Algoritmaları

## Hamiltonian Path (Hamilton Yolu) Algoritması

**Hamiltonian Path**, bir grafta (graph) tüm düğümleri (nodes) tam olarak bir kez ziyaret eden bir yol bulma problemidir. Bu yol, başlangıç ve bitiş düğümleri arasında olmalı ve her düğüm yalnızca bir kez kullanılmalıdır. Hamiltonian Path, özellikle rotalama, devre tasarımı ve optimizasyon problemlerinde kullanılır.

### Temel Kavramlar

- **Graf (Graph):** Düğümler (nodes) ve bu düğümleri birbirine bağlayan kenarlardan (edges) oluşan yapı.
- **Hamilton Yolu (Hamiltonian Path):** Bir grafta, tüm düğümleri tam olarak bir kez ziyaret eden yol.
- **Hamilton Devresi (Hamiltonian Cycle):** Eğer yolun başlangıç ve bitiş düğümleri birbirine bağlıysa, bu bir devre (cycle) oluşturur.

### Geri İzleme (Backtracking) Algoritması

Geri izleme algoritması, Hamiltonian Path problemine çözüm bulmak için kullanılan bir yöntemdir. Bu yöntem, tüm olası yolları sistematik bir şekilde deneyerek, uygun olmayan yolları eleyerek ilerler.

### Algoritmanın Adımları

- **Başlangıç:** Grafın bir düğümünden başla ve bu düğümü yolun ilk adımı olarak işaretle.
- **Komşu Düğümleri Keşfet:** Mevcut düğümün komşularını incele ve henüz ziyaret edilmemiş bir düğüm seç.
- **Yola Ekle:** Seçilen düğümü yola ekle ve ziyaret edildi olarak işaretle.
- **Kontrol Et:** Eğer tüm düğümler ziyaret edildiyse, bir Hamiltonian Path bulunmuş demektir.
- **Geri İzleme:** Eğer mevcut düğümünden çıkış yolu kalmadıysa, bir önceki düğüme geri dön ve farklı bir komşu düğüm dene.
- **Çözüm Bulma:** Tüm düğümler başarıyla ziyaret edildiğinde, Hamiltonian Path bulunmuş olur.

### Örnek

Bir grafın düğümlerini A, B, C, D olarak adlandıralım. Algoritma şu şekilde çalışır:

- A düğümünden başla ve A'yı yola ekle.
- A'nın komşusu B'yi seç ve B'yi yola ekle.
- B'nin komşusu C'yi seç ve C'yi yola ekle.
- C'nin komşusu D'yi seç ve D'yi yola ekle.
- Tüm düğümler ziyaret edildiği için, yol A -> B -> C -> D şeklinde bir Hamiltonian Path'tir.

### Avantajları

- ✓ Tüm olası yolları denediği için doğru çözümü bulma garantisi vardır.
- ✓ Küçük ve orta boyutlu graflar için etkilidir.

### Dezavantajları

- ✗ Büyük graflar için zaman alıcı olabilir.
- ✗ Karmaşıklığı yüksektir, özellikle düğüm sayısı arttıkça performans düşer.

### Sonuç

Hamiltonian Path problemi, geri izleme algoritması kullanılarak etkili bir şekilde çözülebilir. Bu yöntem, özellikle küçük ve orta ölçekli problemler için uygundur. Ancak, büyük ölçekli problemlerde daha verimli algoritmaların kullanılması gerekebilir.



# Greedy Algoritmalar

## Fractional Knapsack Problemi

### Problem Tanımı

Fractional Knapsack problemi, sınırlı kapasiteye sahip bir çantaya, verilen ağırlık ve değer bilgilerinin doğrultusunda en yüksek toplam değeri elde edecek şekilde eşya yerleştirme problemidir. Ancak bu problemde eşyalar bölünebilir, yani bir eşyanın tamamını almak yerine, belirli bir kısmını da çantaya koyabiliriz.

### Greedy Yaklaşımı

**Fractional Knapsack problemi**, açgözlü (greedy) algoritmalarla çözülebilen bir problemidir. Açgözlü yaklaşım, her adımda en iyi görünen seçimi yaparak en yüksek toplam değeri elde etmeye çalışır.

### Adımlar

- Eşyaları Sıralama:** Eşyalar, değer/ağırlık oranına göre azalan şekilde sıralanır.
- Eşyaları Seçme:** Sıralanan eşyalardan en yüksek değer/ağırlık oranına sahip olanlar öncelikli olarak çantaya eklenir.
- Kapasite Kontrolü:** Çantanın kapasitesine göre eşyaların tamamı veya bir kısmı eklenir.
- Sonuç:** Çantanın kapasitesi dolana kadar bu adımlar tekrarlanır ve maksimum değer elde edilir.

### Örnek Senaryo

- Eşyalar:**
  - Eşya 1: Ağırlık = 10 kg, Değer = 60
  - Eşya 2: Ağırlık = 20 kg, Değer = 100
  - Eşya 3: Ağırlık = 30 kg, Değer = 120
- Çanta Kapasitesi:** 50 kg

### Çözüm Adımları

- Değer/ağırlık oranlarını hesaplayalım:**
  - Eşya 1:  $60/10 = 6$
  - Eşya 2:  $100/20 = 5$
  - Eşya 3:  $120/30 = 4$
- En yüksek oranlı eşyadan başlayarak çantaya ekleyelim:**
  - Eşya 1 tamamen eklenir (10 kg, 60 değer)
  - Eşya 2 tamamen eklenir (20 kg, 100 değer)
  - Eşya 3'ten 20 kg eklenir (20 kg, 80 değer)
- Toplam değer:**  $60 + 100 + 80 = 240$

### Kullanım Alanları

**Lojistik ve Depolama:** Eşyaların taşıma ve depolama planlamasında kapasite sınırlamalarına göre en verimli yerleştirmeyi sağlar.

**Yatırım ve Portföy Yönetimi:** Yatırımcılar, en yüksek getiri sağlayan varlıkları seçerek yatırım portföylerini optimize edebilirler.

**Üretim ve Dağıtım:** Üretim hatlarında malzeme kullanımını optimize etmek için kullanılabilir.

**Kargo ve Teslimat:** Kargo taşıma ve teslimat sistemlerinde sınırlı alanı en verimli şekilde kullanmak için uygundur.

### Sonuç

**Fractional Knapsack problemi**, açgözlü algoritmalar kullanılarak optimal çözümle sonuçlanan bir problemidir. Eşyaların bölünebilir olması, greedy yaklaşımın her adımda en iyi seçimi yaparak en yüksek değeri elde etmesini sağlar.



# Greedy Algoritmalar

## Huffman Kodlama Algoritması

**Huffman Kodlama**, verileri daha az yer kaplayacak şekilde temsil etmek için kullanılan bir sıkıştırma yöntemidir. Bu algorithmada, sık geçen karakterlere kısa, seyrek geçen karakterlere uzun bit dizileri atanır.

### Nasıl Çalışır?

1. Frekansları Belirle:
2. İlk olarak, verilen metindeki her karakterin kaç kez tekrar ettiğini sayarsın.
3. En Küçükleri Birleştir:
4. Frekansı en düşük iki karakter seçilir ve bir düğüm altında birleştirilir. Bu birleşim yeni bir karakter gibi düşünülür.
5. Ağaç Oluştur:
6. Aynı işlemi tekrar ederek bir Huffman ağacı kurarsın. Her birleşme, karakterlerin toplam frekansını temsil eden yeni bir düğüm oluşturur.
7. Kodları Ata:
8. Ağacın sol dalına giden her yola '0', sağ dalına '1' verilir. Kökten karaktere kadar giden yol, o karakterin Huffman kodudur.
9. Veriyi Kodla:
10. Artık her karakter için özel bir bit dizin var. Metindeki karakterleri bu kodlarla değiştirerek sıkıştırılmış veriyi oluşturursun.

### Basit Bir Örnek Üzerinden

**Metin: "BABAABB"**

- Bu metinde "B" karakteri daha sık geçiyor, "A" ise biraz daha az.
- Ağaç oluşturulurken A ve B birleştirilir, biri sola, biri sağa yerleştirilir.
- Sol tarafa 0, sağ tarafa 1 verilirse; A için kod 0, B için kod 1 olur.
- Orijinal metin bu kodlarla yazıldığında: BABAABB → 1010101

Yani 7 karakterlik bir metni, sadece 7 bit ile temsil etmiş olduk.

### Kullanım Alanları

- ZIP dosyalarının oluşturulmasında,
- JPEG ve MP3 gibi medya dosyalarının sıkıştırılmasında,
- Bellek tasarrufu gereken her yerde.

### Özetle

**Huffman Kodlama**, veriyi daha kısa kodlarla ifade ederek yer tasarrufu sağlar. Her adımda "en iyi seçimi" yaptığı için bir greedy algoritmadır. Ne kadar sık kullanılıyorsa, o kadar kısa kod verilir. Bu yüzden, sistematik ve oldukça verimli bir yöntemdir.



# Greedy Algoritmalar

## Activity Selection Problemi (Etkinlik Seçimi)

### Problem Tanımı:

Bu problem, belirli başlangıç ve bitiş zamanlarına sahip etkinlikler arasından, çakışmayan en fazla sayıda etkinliği seçmeyi amaçlayan bir optimizasyon problemidir.

### Greedy Yaklaşımı

Greedy Algoritma, bu tür problemleri verimli bir şekilde çözmek için kullanılabilir. Amaç, her adımda en erken biten etkinliği seçerek mümkün olan en fazla etkinliği seçmektir.

### Adımlar

1. Etkinlikleri Bitiş Zamanına Göre Sırala:
2. Tüm etkinlikleri, bitiş zamanlarına göre artan sırayla düzenle.
3. İlk Etkinliği Seç:
4. İlk biten etkinliği seç ve programa dahil et.
5. Çakışmayan Etkinlikleri Seçmeye Devam Et:
6. Bir önceki seçilen etkinliğin bitiş zamanından sonra başlayan ilk etkinliği seç. Bu işlemi tüm etkinlikler için tekrarla.
7. Sonuç:
8. Seçilen etkinliklerin listesi döndürülür.

### Örnek Senaryo

Diyelim ki elimizde şu etkinlikler var:

- Etkinlik A: Başlangıç = 1, Bitiş = 3
- Etkinlik B: Başlangıç = 2, Bitiş = 5
- Etkinlik C: Başlangıç = 4, Bitiş = 7
- Etkinlik D: Başlangıç = 6, Bitiş = 8
- Etkinlik E: Başlangıç = 5, Bitiş = 7
- Etkinlik F: Başlangıç = 8, Bitiş = 9

### Sıralı Hali:

- A: (1, 3)
- B: (2, 5)
- E: (5, 7)
- C: (4, 7)
- D: (6, 8)
- F: (8, 9)

### Seçim Süreci:

1. İlk olarak bitiş zamanlarına göre sıraladık. Şimdi ilk etkinlik olan A'yı seçiyoruz (1-3).
2. A'dan sonra, çakışmayan bir etkinlik seçmemiz gerekiyor. Bitiş zamanı 3 olan A'dan sonra başlayacak en erken etkinlik E (5-7).
3. E'den sonra, bitiş zamanı 7 olan etkinlik F seçilebilir çünkü E ile çakışmıyor (8-9).

### Seçilen Etkinlikler: A, E, F

### Kullanım Alanları:

- Zaman yönetimi ve program planlaması
- Proje yönetimi ve iş planlama
- Kaynak optimizasyonu

### Sonuç:

**Greedy algoritma**, her adımda optimal seçimi yaparak en fazla etkinliği seçmeyi sağlar. Bu yöntem, etkinliklerin sıralanması ve çakışmaların önlenmesi gereken planlama ve zaman yönetimi gibi problemlerde oldukça etkilidir.



# Greedy Algoritmalar

## Job Scheduling Problemi

### Problem Tanımı

**Job Scheduling (İş Zamanlama) problemi**, her bir işin belirli bir kazanç ve tamamlanması gereken son teslim süresi olduğu senaryolarda, maksimum kazanç elde edecek şekilde işlerin zamanlanmasını hedefleyen bir optimizasyon problemidir.

### Greedy Yaklaşımı

Bu problem, **Greedy (Açgözlü) Algoritma** kullanılarak çözülebilir. Amaç, işleri tamamlanma sürelerine uygun şekilde planlayarak en yüksek toplam kazancı elde etmektir.

### Adımlar

- İşleri Kazanca Göre Azalan Şekilde Sırala:** İşleri, kazançlarına göre büyükten küçüğe sıralayın.
- Maksimum Süreyi Belirle:** İşlerin tamamlanabileceği en büyük teslim süresini belirleyin.
- Zamanlama Tablosu Oluştur:**
  - Tüm işlerin tamamlanma sürelerini içeren boş bir zaman çizelgesi oluşturun.
  - En yüksek kazançlı işten başlayarak, onu en geç uygun sürede yerleştirin.
  - Eğer uygun bir zaman dilimi yoksa, o iş atlanır.
- Sonuç:** Planlanan işler ve elde edilen maksimum kazanç döndürülür.

### Örnek Senaryo

#### Girdi

| İŞ | KAZANÇ | SON TESLİM SÜRESİ |
|----|--------|-------------------|
| A  | 100    | 2                 |
| B  | 50     | 1                 |
| C  | 200    | 2                 |
| D  | 20     | 1                 |
| E  | 150    | 3                 |



# Greedy Algoritmalar

Sıralı Hali (Kazanca Göre)

| İŞ | KAZANÇ | SON TESLİM SÜRESİ |
|----|--------|-------------------|
| C  | 200    | 2                 |
| E  | 150    | 3                 |
| A  | 100    | 2                 |
| B  | 50     | 1                 |
| D  | 20     | 1                 |

## Zamanlama

- 1.C seçilir (2. süreye yerleştirilir)
- 2.E seçilir (3. süreye yerleştirilir)
- 3.A seçilir (1. süreye yerleştirilir)
- 4.B ve D uygun yer bulunamadığı için atlanır.

## Zamanlama Sonucu

- Seçilen işler: A, C, E
- Toplam kazanç: 450

## Kullanım Alanları

**Yazılım ve İşlem Zamanlama:** Sunucular veya işlemciler üzerinde görevlerin sırasıyla işlenmesi gereken durumlarda kullanılabilir.

**Üretim Planlaması:** Fabrika üretim hatlarında, makinelerin en verimli şekilde kullanılabilmesi için işlerin zamanlanması.

**Projelerin Zamanlanması:** İnşaat projeleri veya yazılım geliştirme gibi projelerde, her işin belirli bir kazanç sağladığı ve süre kısıtlamaları olduğu durumlarda.

**Ağ Yönetimi:** Veri iletimi ve ağ üzerindeki görevlerin verimli şekilde sıralanması ve zamanlanması.

**Hizmet Sektörü:** Birden fazla müşteriye hizmet veren sistemlerde, her bir işin tamamlanma süresi ve kazancı göz önüne alınarak hizmet sırasının belirlenmesi.

## Sonuç

**Greedy algoritma**, her adımda optimal seçimi yaparak maksimum kazancı elde edecek şekilde iş zamanlamasını oluşturur. Bu yöntem, üretim planlama, proje yönetimi ve CPU zamanlama gibi alanlarda yaygın olarak kullanılır.



# Greedy Algoritmalar

## Prim Algoritması (Minimum Spanning Tree - MST)

### Problem Tanımı

Prim Algoritması, bir grafın tüm düğümlerini kapsayan minimum toplam ağırlıklı ağacı (Minimum Spanning Tree - MST) bulmak için kullanılan bir Açıgözlü (Greedy) algoritmadır. Bu algoritma, bağlı, ağırlıklı ve yönsüz graflarda kullanılır.

### Temel Mantık

- Minimum Spanning Tree (MST), V düğüm varsa V-1 kenardan oluşur.
- Başlangıçta bir düğüm seçilir ve o düğümden başlayarak, komşu düğümler arasında en düşük ağırlıklı kenar seçilerek ağaç genişletilir.
- Seçilen kenar, ağaca eklenir ve yeni düğüm artık ağacın bir parçası olur.
- Bu işlem, tüm düğümler ağaca dahil edilene kadar devam eder.
- Her adımda, halihazırda ağaca dahil edilen düğümler ile dışarıda kalan düğümler arasındaki en düşük maliyetli kenar seçilir.

### Algoritmanın Adımları

1. Rastgele veya belirli bir başlangıç düğümü seçilir.
2. Bu düğümün komşuları arasından en düşük ağırlıklı kenar bulunur ve MST'ye eklenir.
3. Eklenen yeni düğümün komşuları da incelenerek, ağaca henüz dahil edilmemiş olanlar arasından yine en düşük ağırlıklı kenar seçilir.
4. Tüm düğümler ağaca dahil edilene kadar bu işlem tekrarlanır.

### Örnek Senaryo (Grafiksiz)

#### Verilen Kenarlar ve Ağırlıkları:

- A - B: 2
- A - C: 3
- B - C: 1
- B - D: 4
- C - D: 5

#### Algoritmanın İşleyişi:

1. A düğümünden başlanır.
2. A ile komşuları arasında en düşük ağırlıklı kenar A - B (ağırlık = 2) seçilir ve MST'ye eklenir.
3. B düğümünün komşuları kontrol edilir. A dışında kalan en düşük ağırlıklı kenar B - C (ağırlık = 1) seçilir.
4. C düğümünden B - D (ağırlık = 4) kenarı en düşük olduğu için eklenir.
5. Tüm düğümler (A, B, C, D) ağaçta yer aldığı için algoritma tamamlanır.

#### Sonuç:

- Seçilen Kenarlar: A - B, B - C, B - D
- Toplam Ağırlık:  $2 + 1 + 4 = 7$

### Kullanım Alanları

- Ağ ve iletişim altyapısı tasarımı
- Elektrik ve enerji dağıtım sistemleri
- Ulaşım ve yol planlamaları
- Veri madenciliği ve kümeleme algoritmaları

### Sonuç

**Prim Algoritması**, her adımda sadece en küçük ağırlıklı kenarı seçerek çalıştığı için hem verimli hem de optimal sonuçlar üretir. Bağlı ve ağırlıklı graflar için MST bulmada etkili bir çözümdür.



# İleri Seviye Algoritmalar

## Floyd's Cycle-Finding Algorithm (Tortoise and Hare)

**Floyd's Cycle-Finding Algorithm**, bir bağlantılı liste veya benzeri veri yapılarında döngü (cycle) olup olmadığını bulmak için kullanılan bir algoritmadır. Bu algoritma, iki işaretçi kullanarak çalışır: "tavşan" (hare) ve "kaplumbağa" (tortoise). Algoritma, döngü varsa, döngünün başlangıcını ve uzunluğunu bulabilir.

### Algoritmanın Çalışma Prensipleri

#### 1. Başlangıç Durumu:

- "Kaplumbağa" ve "Tavşan" işaretçileri, listenin ilk elemanına yerleştirilir.
- "Kaplumbağa" işaretçisi, her adımda bir adım ilerler.
- "Tavşan" işaretçisi, her adımda iki adım ilerler.

#### 2. Döngü Bulma:

- Her iki işaretçi de listede ilerlerken, "Tavşan" işaretçisi "Kaplumbağa"yı yakalarsa, bu, listede bir döngü olduğu anlamına gelir.
- Eğer "Tavşan" ile "Kaplumbağa" işaretçileri hiçbir zaman kesişmezse, listede döngü yoktur.

#### 3. Döngünün Başlangıcı:

- Eğer döngü bulunursa, "Kaplumbağa"yı başa, yani ilk düğüme geri getiririz.
- Ardından, hem "Kaplumbağa" hem de "Tavşan" birer adım ilerleyerek, döngünün başlangıç noktasını bulurlar.

### Algoritmanın Adımları

- **Adım 1:** Başlangıçta, "Kaplumbağa" ve "Tavşan" işaretçilerini listenin ilk elemanına yerleştirin.
- **Adım 2:** "Kaplumbağa"yı her seferinde bir adım, "Tavşan"ı ise her seferinde iki adım ilerletin.
- **Adım 3:** Eğer "Kaplumbağa" ve "Tavşan" işaretçileri birbirini bulursa, döngü olduğunu kabul edin ve döngünün başlangıcını bulmak için yukarıdaki adımlara devam edin.
- **Adım 4:** Eğer "Kaplumbağa" ve "Tavşan" birbirini bulamazsa, listenin sonuna gelmiş olursunuz ve döngü yoktur.

### Avantajları

- ✓ **Zaman Karmaşıklığı:** Algoritma,  $O(n)$  zaman karmaşıklığına sahiptir.
- ✓ **Hafıza Verimliliği:** Bu algoritma yalnızca sabit bir miktarda ek hafıza kullanır ( $O(1)$  alan karmaşıklığı).

### Örnek

Bir liste düşünün:  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 3$  (döngü başı)

- İlk başta "Kaplumbağa" 1'e, "Tavşan" 1'e yerleştirilir.
- Her iki işaretçi ilerlemeye başladığında, "Tavşan" her adımda iki düğüm ilerlerken, "Kaplumbağa" sadece bir düğüm ilerler.
- "Tavşan" ve "Kaplumbağa" sonunda 3'te buluşur, bu da döngünün başladığını gösterir.
- Döngünün başlangıcını bulmak için "Kaplumbağa"yı başa yerleştiririz ve her iki işaretçiyi birer adım ilerleterek 3'te buluşuruz.

### Sonuç

**Floyd's Cycle-Finding Algorithm**, döngüleri hızlı ve hafızada verimli bir şekilde tespit etmek için ideal bir yöntemdir. Herhangi bir liste veya bağlı veri yapısındaki döngüleri tespit etmenin temel yollarından biridir.



# İleri Seviye Algoritmalar

## Topolojik Sıralama (Topological Sort)

### Tanım

**Topolojik Sıralama**, yalnızca yönlü ve döngüsüz (DAG) graflar için geçerli bir algoritmadır. Amaç, graf içindeki düğümleri öyle bir sıraya koymaktır ki, her kenar ( $u \rightarrow v$ ) için  $u$  düğümü  $v$ 'den önce gelsin.

### Temel Özellikler

- Sadece DAG (Directed Acyclic Graph) üzerinde çalışır.
- Döngü varsa sıralama mümkün değildir.
- Her düğüm, kendisine bağlı olan düğümlerden önce gelir.
- Sıralama doğrusal bir dizidir.

### Algoritma Adımları

1. Girdi Derecesi (in-degree) 0 olan tüm düğümleri bul.
2. Bunları bir kuyruğa (veya yığına) ekle.
3. Kuyruktan düğüm çıkar  $\rightarrow$  sıralama listesine ekle.
4. Bu düğümden çıkan tüm kenarları kaldır  $\rightarrow$  bağlı düğümlerin in-degree'lerini 1 azalt.
5. Yeni in-degree'si 0 olanları kuyruğa ekle.
6. Kuyruk boşalana kadar 3-5 arası adımları tekrarla.

**Not:** Eğer işlem sonunda hala in-degree  $> 0$  olan düğümler varsa, graf döngülüdür.

### Zaman ve Alan Karmaşıklığı

- Zaman:  $O(V + E)$
- Alan:  $O(V)$
- ( $V$ : düğüm sayısı,  $E$ : kenar sayısı)

### Örnek Senaryo (Görev Sıralama)

#### Görev ilişkileri:

- $A \rightarrow B$
- $C \rightarrow E$
- $B \rightarrow D$
- $E \rightarrow D$

#### Bağımlılıklara göre sıralama:

$A \rightarrow C \rightarrow B \rightarrow E \rightarrow D$  (olası bir çözüm)

### Kullanım Alanları

- Görev yönetimi, proje planlama
- Yazılım derleme sırası (build systems)
- Ders ön koşulu çözümü
- Paket yöneticilerde kurulum sırası

### Sonuç

**Topolojik sıralama**, bir işlem yapılmadan önce hangi işlemlerin tamamlanması gerektiğini belirlemek için ideal bir yöntemdir. Bağımlılık çözümlemelerinde vazgeçilmezdir ve modern yazılım mühendisliğinde sıkça kullanılır.



# İleri Seviye Algoritmalar

## Sieve of Eratosthenes

**Sieve of Eratosthenes (Eratosthenes Eleği)**, verilen bir sayıya kadar olan asal sayıları hızlı bir şekilde bulmak için kullanılan eski ve verimli bir algoritmadır. Bu algoritma, sayılar arasındaki asal sayıları bulmak için belirli bir yöntemi takip eder.

### Algoritmanın Temel Prensibi

#### 1. Başlangıç Durumu:

- Başlangıçta, 2'den başlayarak verilen sayı kadar olan tüm sayıları asal kabul ederiz.
- Bu sayılar, başlangıçta bir dizi içerisinde işaretlenir.

#### 2. İşaretleme:

- İlk asal sayı olan 2'yi aldığımızda, 2'nin katları (4, 6, 8, ...) asal olamaz, çünkü bu sayılar 2'ye bölünebilir.
- 2'nin katları hariç, 2'yi asal olarak işaretleriz.
- Ardından 3'e geçeriz. 3'ün katları (6, 9, 12, ...) asal olamaz, çünkü bu sayılar 3'e bölünebilir.
- Bu işlem, n sayısına kadar devam eder.

#### 3. İşlem Bittiğinde:

- Algoritma, asal sayılarla işaretlenmiş tüm sayıları bırakır. Geriye kalan sayılar asal olmayanlardır (yani 2'ye, 3'e veya diğer asal sayılara bölünebilenler).
- Sonuç olarak, başlangıçta işaretlenen asal sayılar, verilen sayıya kadar olan asal sayıları verir.

### Algoritmanın Adımları

- Adım 1:** 2'den başlayarak verilen sayıya kadar olan tüm sayıları bir listeye ekleyin ve başlangıçta tümünü asal olarak işaretleyin.
- Adım 2:** İlk asal sayıyı (2) seçin.
- Adım 3:** Bu asal sayının katlarını (bu sayı hariç) işaretleyin.
- Adım 4:** Sonraki asal sayıyı seçin ve tekrar katlarını işaretleyin.
- Adım 5:** Adım 4'ü, verilen sayı kadar devam edin.
- Adım 6:** Algoritma tamamlandığında, asal sayılar listelenir.

### Zaman Karmaşıklığı

- **Zaman Karmaşıklığı:**  $O(n \log \log n)$ , burada n, verilen sayıyı temsil eder. Bu, algoritmanın çok hızlı çalışmasını sağlar.
- **Alan Karmaşıklığı:**  $O(n)$ , çünkü asal sayıları işaretlemek için n kadar alan kullanılır.

### Örnek

Verilen sayı 20 olduğunda, **Sieve of Eratosthenes algoritması** şu şekilde çalışır:

Başlangıçta, tüm sayılar asal kabul edilir:

[2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]

1. 2'yi asal olarak kabul ettikten sonra, 2'nin katları (4, 6, 8, 10, 12, 14, 16, 18, 20) işaretlenir.

2. 3'ü asal olarak kabul ettikten sonra, 3'ün katları (6, 9, 12, 15, 18) işaretlenir.

3. 5'i asal olarak kabul ettikten sonra, 5'in katları (10, 15, 20) işaretlenir.

4. 7'yi asal olarak kabul ettikten sonra, 7'nin katları (14) işaretlenir.

Sonuçta, **geriye kalan asal sayılar:** [2, 3, 5, 7, 11, 13, 17, 19]

### Sonuç

**Sieve of Eratosthenes**, verilen bir sayıya kadar olan asal sayıları bulmak için en etkili algoritmalardan biridir. Büyük sayılar için bile hızlı bir şekilde asal sayılar listesi oluşturulabilir.



# İleri Seviye Algoritmalar

## Union-Find (Disjoint Set)

**Union-Find (veya Disjoint Set)**, bir küme veya öğeler arasındaki ilişkiyi yönetmek için kullanılan bir veri yapısıdır. Bu veri yapısı, genellikle "kümeler" halinde gruplanmış öğeler arasında birleşim (union) ve bulma (find) işlemleri yapmayı sağlar. Genellikle ağlarda, birleşik bileşenlerin bulunması gibi problemlerde kullanılır.

### Algoritmanın Temel Özellikleri

- **Find:** Bir öğenin hangi kümeye ait olduğunu bulma işlemi.
- **Union:** İki kümeyi birleştirme işlemi.
- **Disjoint:** Kümeler birbirine bağlı değildir, yani her öğe sadece bir kümeyle ilişkilidir.

### Algoritmanın Adımları

#### 1. Başlangıç:

- Başlangıçta, her öğe kendi kümesinde yer alır. Yani, her öğe tek başına bir küme oluşturur.

#### 2. Find İşlemi:

- Bir öğenin hangi kümeye ait olduğunu bulmak için, öğenin kökünü buluruz. Eğer öğe bir kümeye başlı ise, o kümenin köküdür.
- Find işlemi, "path compression" tekniği ile hızlandırılabilir. Bu teknik, öğenin kökünü bulduğunda, ona bağlı tüm öğeleri doğrudan kök ile ilişkilendirir.

#### 3. Union İşlemi:

- İki küme birleştirildiğinde, iki kümeyi de dengede tutmak için "rank" (derece) veya "size" (boyut) kullanılabilir.
- Bu yöntemle daha küçük kümeyi büyük kümeye bağlamak, dengenin korunmasını sağlar.

#### 4. Bitiş:

- Tüm union ve find işlemleri tamamlandıktan sonra, her öğenin hangi kümeye ait olduğu kolayca belirlenebilir.

### Zaman Karmaşıklığı

- Find:  $O(\alpha(n))$  — Burada  $\alpha(n)$ , inverse Ackermann fonksiyonudur ve pratikte çok küçük bir değer alır.
- Union:  $O(\alpha(n))$  — Aynı şekilde çok hızlı bir işlemdir.

Toplamda, Union-Find veri yapısının bu işlemleri genellikle yaklaşık  $O(1)$  olarak kabul edilir.

### Örnek

Bir grup öğrenciyi düşünelim.

Her öğrenci başlangıçta kendi grubundadır: [1], [2], [3], [4], [5]

1. Öğrenciler 1 ve 2 aynı grupta olmalı. Union işlemi ile bu iki grup birleşir: [1, 2], [3], [4], [5]
2. Öğrenciler 3 ve 4 aynı grupta olmalı. Union işlemi ile bu iki grup birleşir: [1, 2], [3, 4], [5]
3. Öğrenciler 2 ve 4 aynı grupta olmalı. Union işlemi ile bu iki grup birleşir: [1, 2, 3, 4], [5]
4. Öğrenciler 1 ve 5 aynı grupta olmalı. Union işlemi ile bu iki grup birleşir: [1, 2, 3, 4, 5]

Sonuç olarak, tüm öğrenciler tek bir grupta birleştirilmiştir.

### Sonuç

**Union-Find (Disjoint Set) algoritması**, kümelerle yapılan birleşim ve bulma işlemlerini hızla gerçekleştiren bir veri yapısıdır. Genellikle ağ teorisi, küme işlemleri ve grafik algoritmalarında (minimum spanning tree gibi) kullanılır.



# İleri Seviye Algoritmalar

## Turing Makineleri

### Tanım

**Turing makinesi**, 1936'da Alan Turing tarafından önerilen soyut bir hesaplama modelidir. Modern bilgisayarların teorik temeli sayılır. Her türlü algoritmayı tanımlamak, çalıştırmak ve hesaplamanın sınırlarını test etmek için kullanılır.

### Temel Bileşenler

- **Bant:** Sonsuz uzunlukta, üzerinde veri (semboller) barındıran hafıza alanı.
- **Okuyucu/Yazıcı Başlık:** Bant üzerinde gezerek veri okur ve yazar.
- **Durumlar (States):** Makinenin mevcut işlem modunu tanımlar.
- **Geçiş Fonksiyonu:**
  - **Okunan sembole ve mevcut duruma göre:**
    - → Yeni sembol yaz,
    - → Durumu değiştir,
    - → Başlığı sola/sağa hareket ettir.

### Çalışma Prensibi

1. Başlangıç Durumu ile işlem başlar.
2. Banttan bir sembol okunur.
3. Geçiş fonksiyonu devreye girer:
  - Yeni sembol yazar
  - Başlığı hareket ettirir
  - Durumu değiştirir
4. Durma Durumu: Geçerli bir geçiş tanımı kalmayana kadar işlem sürer.

### Örnek Uygulama: Binary Sayı Çarpma

- Bantta "101" (ikilik sistemde 5) yazılı.
- Turing makinesi bu sayıyı 2 ile çarpmak için çalışır.
- Okuma, yazma ve durum geçişleriyle sonuç: "1010" (10).

### Hesaplama Sınırları

Turing makineleri, teorik olarak her algoritmik problemi çözebilir gibi görünse de bazı problemler çözülemez:

- Durma Problemi (Halting Problem):
- Bir Turing makinesi, başka bir makinenin durup durmayacağını karar veremez.
- Bu tür problemler "Turing tarafından çözülemez" sınıfına girer.

### Karmaşıklık ve Teorik Güç

- Turing makineleri, algoritmaların neler yapabileceğini ve nerede durduğunu gösterir.
- Gerçek bilgisayarlar, Turing makinelerinin pratik yansımalarıdır.
- Bilgisayar biliminin temel kavramları bu model üzerine inşa edilmiştir.

### Kullanım Alanları

- Teorik Bilgisayar Bilimi
- Otomata ve Dil Teorisi
- Algoritma Analizi
- Hesaplanabilirlik ve Karar Verilebilirlik Analizi

### Sonuç

**Turing makineleri**, bilgisayar biliminin felsefi ve teorik temelini oluşturur. Ne hesaplanabilir, ne hesaplanamaz sorularına cevap ararken karşımıza ilk çıkan modeldir. Basitliği sayesinde büyük bir güç taşır.



# Matematiksel Algoritmalar

## Euclidean Algorithm (Öklid Algoritması) ile En Büyük Ortak Bölen (GCD) Hesaplama

### GCD Nedir?

GCD, iki sayının En Büyük Ortak Bölünü anlamına gelir. Yani, iki sayının ortak bölünüleri arasında en büyük olan sayıdır. Örneğin, 36 ve 60 sayıları için GCD 12'dir, çünkü 36 ve 60'ı bölün en büyük sayı 12'dir.

### Euclidean Algorithm (Öklid Algoritması) Nedir?

Öklid Algoritması, GCD'yi hızlı bir şekilde hesaplamak için kullanılan matematiksel bir yöntemdir. Bu algoritma, iki sayı arasındaki GCD'yi hesaplamak için bölme işlemine dayanır. Algoritma, iki sayıyı birbirine bölerek, kalanı bulur ve bu kalan üzerinden işlem yaparak GCD'yi bulur.

### Öklid Algoritması Adımları

- İki pozitif tam sayı  $a$  ve  $b$  verilir (örneğin, 36 ve 60).
- İlk adımda  $a$  sayısını  $b$  sayısına bölün.
- Bu bölme işleminin kalanını bulun.
- Yeni adımda,  $b$ 'yi kalanı ile değiştirin ve kalan 0 olana kadar adımı tekrarlayın.
- Kalan sıfır olduğunda, son bölüme karşılık gelen bölün, GCD'yi verir.

### Örnek

Örnek olarak 36 ve 60 sayılarının GCD'sini hesaplayalım:

1. İlk olarak, 60'ı 36'ya bölüyoruz:  $60 \div 36 = 1$  (Kalan: 24)
2. Şimdi, 36'yı 24'e bölüyoruz:  $36 \div 24 = 1$  (Kalan: 12)
3. Sonra, 24'ü 12'ye bölüyoruz:  $24 \div 12 = 2$  (Kalan: 0)
4. Kalan 0 olduğunda, son bölün 12, GCD'yi verir.

**Sonuç:** 36 ve 60 sayılarının GCD'si 12'dir.

### Avantajları

✓ **Verimlilik:** Bu algoritma oldukça hızlıdır ve genellikle çok büyük sayılar için bile etkili bir şekilde çalışır.

✓ **Basitlik:** Uygulaması kolaydır ve sadece temel bölme işlemlerine dayanır.

Uygulama Alanları

✓ **Matematik:** En büyük ortak bölün hesaplamaları gerektiren çeşitli problemlerde kullanılır.

✓ **Kriptografi:** Anahtar üretimi gibi işlemlerde GCD önemli bir rol oynar.

✓ **Dijital Sistemler:** Veri analizinde ve hata düzeltme algoritmalarında kullanılabilir.



# Matematiksel Algoritmalar

## Sieve of Eratosthenes (Eratosthenes Eleği) ile Asal Sayıların Bulunması

### Asal Sayılar Nedir?

**Asal sayılar**, yalnızca 1 ve kendisiyle tam bölünebilen, 1'den büyük olan sayılardır. Örneğin, 2, 3, 5, 7, 11 gibi sayılar asal sayılardır. Asal sayılar, matematiksel ve sayısal analizlerde önemli bir yer tutar.

### Eratosthenes Eleği (Sieve of Eratosthenes) Nedir?

**Eratosthenes Eleği**, asal sayıları bulmak için kullanılan en eski ve etkili algoritmalarından biridir. Bu algoritma, belirli bir sayıya kadar olan asal sayıları hızlı ve verimli bir şekilde bulmayı amaçlar. Algoritma, bir dizi sayıyı işaretleyerek, asal olmayan sayıları (yani, asal olmayan sayıları işaretleyerek) eler.

### Eratosthenes Eleği Nasıl Çalışır?

- Başlangıç:** Bir sayı dizisi oluşturulur. Bu dizide 2'den başlayarak, belirli bir sayıya kadar olan bütün sayılar bulunur.
- 2'yi İşaretle:** 2'nin katları asal olmayan sayılardır, bu yüzden 2'nin katlarını diziden çıkarın.
- Bir Sonraki Sayıyı İşaretle:** Bir sonraki asal sayıyı (yani işaretlenmemiş sayıyı) bulun, bu sayının katlarını diziden çıkarın.
- Adımları Tekrarlayın:** 2'den sonraki her asal sayıyı bulana kadar işlemi tekrarlayın. Bu işlem, belirtilen sınırdaki tüm asal sayılar bulunduğunda sona erer.
- Sonuç:** Sonuç olarak, işaretlenmemiş sayılar asal sayılardır.

### Adım Adım Örnek

Diyelim ki 30'a kadar asal sayıları bulmak istiyoruz:

- Başlangıçta, 1'den 30'a kadar olan tüm sayılar yazılır:  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30
- İlk asal sayı olan 2'yi alır ve 2'nin katlarını (4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30) çıkarırız.  
1, 2, 3, X, 5, X, 7, X, 9, X, 11, X, 13, X, 15, X, 17, X, 19, X, 21, X, 23, X, 25, X, 27, X, 29, X
- Sonra 3'ü alırız ve 3'ün katlarını (9, 15, 21, 27) çıkarırız:  
1, 2, 3, X, 5, X, 7, X, X, X, 11, X, 13, X, X, X, 17, X, 19, X, X, 23, X, 25, X, X, 29, X
- 5'e gelene kadar devam ederiz, 5'in katlarını (25) çıkarırız.  
1, 2, 3, X, 5, X, 7, X, X, X, 11, X, 13, X, X, X, 17, X, 19, X, X, 23, X, X, X, 29, X
- Son olarak, kalan sayılar asal sayılardır: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29

### Avantajları

- ✓ **Verimlilik:** Bu yöntem, özellikle küçük sayılar için çok hızlıdır. Daha büyük sayılar için de hâlâ oldukça etkili bir yöntemdir.
- ✓ **Basitlik:** Algoritmanın temel mantığı oldukça basittir, ve sadece bir dizi işaretleme işlemine dayanır.
- ✓ **Zaman Karmaşıklığı:** Bu algoritmanın zaman karmaşıklığı  $O(n \log \log n)$ 'dir, yani oldukça hızlıdır.

### Kullanım Alanları

- **Matematiksel Hesaplamalar:** Asal sayılar üzerinde yapılan analizlerde kullanılır.
- **Kriptografi:** Şifreleme algoritmalarında asal sayılar önemli bir rol oynar, özellikle RSA şifrelemesinde.
- **Sayısal Analiz ve Veri Analizi:** Asal sayılar çeşitli algoritmalarda anahtar bir rol oynar.



# Matematiksel Algoritmalar

## Fast Exponentiation (Hızlı Üssü Alma) Algoritması

### Üssü Alma Nedir?

Üssü alma, bir sayıyı kendisiyle belirli bir sayıda çarpmaktır. Örneğin,  $a^b$  ifadesi,  $a$ 'yı  $b$  kez kendisiyle çarpmayı ifade eder. Bu işlem, özellikle büyük üslü hesaplamalar için zaman alıcı olabilir. Fast Exponentiation, bu tür hesaplamaları daha hızlı yapmak için kullanılan bir tekniktir.

### Fast Exponentiation (Hızlı Üssü Alma) Nedir?

Hızlı üssü alma algoritması, üssü alma işlemini daha hızlı hale getirmek için böl ve yönet (divide and conquer) yöntemini kullanır. Bu algoritma, üssü almak için tekrar tekrar çarpma yaptıktan sonra, üsleri bölerek hesaplama yapar. Bu şekilde, hesaplamalar çok daha hızlı bir şekilde yapılır.

### Temel İlkeler

1. Eğer üs çiftse, problemi yarıya böleriz.
2. Eğer üs tekse, üssü bir azaltıp, ardından kare alırsak.

### Adım Adım Açıklama

1. Çift Üs Durumu: Eğer üs  $b$  çiftse, o zaman  $a^b = (a^{b/2})^2$  olarak hesaplanır.
2. Tek Üs Durumu: Eğer üs  $b$  tekse, o zaman  $a^b = a * a^{(b-1)}$  olarak hesaplanır.

### Örnek

Diyelim ki  $a = 2$  ve  $b = 13$ .  $2^{13}$ 'ü hızlı bir şekilde hesaplamak için algoritmayı takip edelim:

1.  $2^{13}$  tek olduğu için  $2 \times 2^{12}$  olarak yazabiliriz.
2.  $2^{12}$  çift olduğu için  $(2^6)^2$  olarak yazabiliriz.
3.  $2^6$  çift olduğu için  $(2^3)^2$  olarak yazabiliriz.
4.  $2^3$  tek olduğu için  $2 \times 2^2$  olarak yazabiliriz.
5.  $2^2$  çift olduğu için  $(2^1)^2$  olarak yazabiliriz.
6.  $2^1$  tek olduğu için  $2 \times 2^0$  olarak yazabiliriz.
7.  $2^0 = 1$ .

Sonuç olarak, tüm bu adımların sonucunda  $2^{13} = 8192$ .

### Avantajları

- ✓ **Zaman Karmaşıklığı:** Fast Exponentiation algoritmasının zaman karmaşıklığı  $O(\log b)$ 'dir.
- ✓ **Verimlilik:** Büyük üsleri hesaplamak için geleneksel çarpma yöntemlerine göre çok daha verimlidir.

### Kullanım Alanları

- **Kriptografi:** Büyük üslerle yapılan işlemler, örneğin RSA şifrelemesi, bu algoritmayı kullanarak hızlandırılır.
- **Sayısal Hesaplamalar:** Fiziksel simülasyonlar ve mühendislik hesaplamalarında yüksek hassasiyetli üs hesaplamaları yapılırken kullanılır.
- **Algoritmalar:** Çeşitli algoritmalarda hızlı üssü alma gereklidir, özellikle büyük sayıların kullanıldığı problemlerle çalışırken.



# Matematiksel Algoritmalar

## Fermat's Little Theorem (Fermat'ın Küçük Teoremi)

### Fermat'ın Küçük Teoremi Nedir?

Fermat'ın Küçük Teoremi, asal sayılarla ilgili önemli bir teoremdir. Bu teorem, bir asal sayının modüler aritmetik üzerindeki özelliklerini açıklar. Teorem şöyle der:

Eğer  $p$  bir asal sayı ve  $a$   $p$ 'ye bölünmeyen bir tamsayı ise, o zaman:

$$a^{(p-1)} \equiv 1 \pmod{p}$$

Yani,  $a$  sayısının  $(p-1)$ . kuvveti  $p$  sayısına bölündüğünde, kalan 1'e eşit olacaktır. Bu teorem, özellikle sayı teorisi, şifreleme algoritmaları (özellikle RSA) ve modüler aritmetik işlemlerinde oldukça kullanılır.

### Teorem Açıklaması

- $p$  asal sayıdır.
- $a$  sayısı  $p$  sayısına bölünmeyen bir tamsayıdır. Yani  $a \% p \neq 0$  olmalıdır.
- Fermat'ın Küçük Teoremi diyor ki,  $a$  sayısının  $(p-1)$ . kuvvetinin  $p$  sayısına bölümünden kalan 1 olacaktır.

### Örnek

Diyelim ki  $p = 5$  ve  $a = 2$ . Fermat'ın Küçük Teoremi'ne göre:

$$2^{(5-1)} = 2^4 = 16$$

Şimdi, 16'nın 5'e bölümünden kalanı bulalım:

$$16 \div 5 = 3 \text{ (kalan 1)}$$

Bu, Fermat'ın Küçük Teoremi'nin geçerli olduğunu gösterir çünkü  $2^4 \bmod 5 = 1$ .

### Fermat'ın Küçük Teoreminin Kullanım Alanları

- **Şifreleme Sistemleri (RSA):** Fermat'ın Küçük Teoremi, özellikle RSA algoritmasında kullanılan modüler aritmetik işlemlerinde önemli bir rol oynar. RSA şifreleme algoritmasında asal sayılar ve modüler üs hesaplamaları kullanılır.
- **Hızlı Üs Hesaplamaları:** Fermat'ın Küçük Teoremi, büyük üslerin hesaplanmasında, özellikle modüler hesaplamalarla birlikte kullanıldığında işlemleri hızlandırır.
- **Sayısal Analiz ve Kriptografi:** Sayılar teorisinde çeşitli hesaplamalar yaparken ve kriptografik uygulamalarda Fermat'ın Küçük Teoremi kullanılabilir.

### Fermat'ın Küçük Teoremi ile İlgili Önemli Notlar

- **Asal Sayı Olmayan  $p$ :** Eğer  $p$  asal bir sayı değilse, Fermat'ın Küçük Teoremi geçerli olmayabilir. Bu nedenle, teorem sadece asal sayılar için geçerlidir.
- **$a$ 'nın  $p$ 'ye Bölünebilir Olması Durumu:** Eğer  $a$  sayısı  $p$  sayısına bölünebiliyorsa, teorem geçerli olmaz. Ancak, teoremdeki  $a$  sayısının  $p$ 'ye bölünmediği koşulunun sağlanması önemlidir.

### Sonuç

**Fermat'ın Küçük Teoremi**, özellikle asal sayılarla yapılan işlemler ve modüler aritmetik hesaplamaları için temel bir araçtır. Bu teorem, sayı teorisi ve kriptografi gibi alanlarda geniş bir uygulama alanına sahiptir.



# Matematiksel Algoritmalar

## Modüler Aritmetik Algoritmalarının Temelleri

### Modüler Aritmetik Nedir?

**Modüler aritmetik**, sayıları belirli bir modül ( $m$ ) değerine göre işlemenin matematiksel yöntemidir. Bir sayı başka bir sayıya bölündüğünde kalanı bulma prensibine dayanır.

### Örnek:

$a \equiv b \pmod{m} \Rightarrow (a - b), m$ 'nin bir katıdır.

Bu yöntem özellikle büyük sayıların işlendiği kriptografi, sayısal hesaplamalar ve bilgisayar bilimleri gibi alanlarda kullanılır.

### Temel İşlemler

Modüler aritmetikte dört temel işlem bulunur:

- **Toplama:**
  - $(a + b) \pmod{m} = ((a \pmod{m}) + (b \pmod{m})) \pmod{m}$
- **Çıkarma:**
  - $(a - b) \pmod{m} = ((a \pmod{m}) - (b \pmod{m})) \pmod{m}$
- **Çarpma:**
  - $(a \times b) \pmod{m} = ((a \pmod{m}) \times (b \pmod{m})) \pmod{m}$
- **Bölme:**
  - $a / b \pmod{m} = a \times b^{-1} \pmod{m}$
  - $(b^{-1}, b$ 'nin mod  $m$ 'ye göre tersi)

### Modüler Aritmetik Algoritmaları

#### Modüler Ters

Bir sayının modüler tersi, çarpıldığında sonuç 1 yapan değerdir.

$$a \times b \equiv 1 \pmod{m}$$

Bu işlem için Genişletilmiş Öklid Algoritması (Extended Euclidean Algorithm) kullanılır.

#### Modüler Üs Hesaplama

Fast Exponentiation yöntemiyle büyük üsler hızlı şekilde hesaplanır:

$$a^b \pmod{m}$$

Her adımda mod alınarak işlem basitleştirilir. Bu, kriptografide özellikle RSA algoritmasında kullanılır.

#### Çin Kalan Teoremi (CRT)

Birden fazla modül içeren denklem sistemlerini çözmek için kullanılır.

Farklı modüllerle hesaplama yaparken işlemleri birleştirerek daha büyük bir çözüm elde etmeyi sağlar.

CRT, çok büyük sayılarla yapılan işlemleri hızlandırır.

#### Modüler Bölme

Bölme işlemi doğrudan yapılamaz, bunun yerine modüler ters kullanılır:

$$a / b \pmod{m} = a \times b^{-1} \pmod{m}$$

### Kullanım Alanları

- **Kriptografi:** RSA gibi algoritmalarda temel yapıtaşıdır.
- **Sayısal Hesaplamalar:** Büyük sayılarla verimli işlem yapılmasını sağlar.
- **Bilgisayar Bilimi:** Algoritmalarda, sayı teorisinde ve optimizasyonda kullanılır.

### Sonuç

**Modüler aritmetik**, modern sayısal dünyada güçlü bir araçtır.

Hızlı ve etkili algoritmalar geliştirmek için kullanılan bu yapı, kriptografi ve algoritmalar dünyasının vazgeçilmezlerinden biridir.



# Matematiksel Algoritmalar

## Çin Kalan Teoremi (Chinese Remainder Theorem)

### Çin Kalan Teoremi Nedir?

**Çin Kalan Teoremi (CRT)**, modüler aritmetikle ilgili önemli bir teoremdir. Bu teorem, farklı modüllerle çözülebilen sistemlerin birleştirilmesini sağlar. Yani, birden fazla modüler denklem kümesinin tek bir çözümünü bulmak için kullanılır.

### Temel Koşullar

Çin Kalan Teoremi'nin uygulanabilmesi için modüllerin birbirleriyle asal (coprime) olması gerekir. Yani, her bir modül  $m_1, m_2, \dots, m_n$  için:  $\gcd(m_i, m_j) = 1$  ( $i \neq j$ ).

### Teorem Nasıl Çalışır?

Çin Kalan Teoremi'nin temel mantığı, verilen modüler denklemlerle çözümü adım adım birleştirmektir. İlk olarak, modüller arasındaki ilişkileri kullanarak her bir denklem için bir çözüm bulunur, sonra bu çözümler birleştirilir.

### Örnek

Diyelim ki aşağıdaki modüler denklemleri çözeceğiz:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 1 \pmod{7}$$

Bu adımları takip ederek çözüm bulunur ve sonuç olarak  $x = 8$  elde edilir.

### Çin Kalan Teoremi'nin Kullanım Alanları

- Kriptografi
- Sayısal Hesaplamalar
- Çeşitli Algoritmalar

### Sonuç

**Çin Kalan Teoremi**, modüler aritmetik ve sayı teorisindeki güçlü araçlardan biridir. Farklı modüllerle verilen denklemleri tek bir denklemde birleştirerek çözüm sağlamak, özellikle büyük sayılarla işlem yaparken önemli avantajlar sunar.



# String Matching Algoritmaları

## Rabin-Karp String Matching Algorithm

**Rabin-Karp algoritması**, metin içinde bir alt diziye aramak için kullanılan bir başka verimli algoritmadır. Bu algoritma, özellikle birden fazla arama yapıldığında veya büyük metinlerde birden fazla alt dizi aranırken avantaj sağlar. Rabin-Karp algoritması, "hashing" (karma fonksiyonu) kullanarak, alt dizilerin eşleşmesini hızlandırır.

### Temel Fikir

Rabin-Karp algoritmasının temel fikri, her bir alt diziye bir sayı ile temsil etmektir. Bu sayılar, "hashing" kullanılarak hesaplanır. Bir metnin her alt dizisi için bir hash değeri hesaplanır ve bu hash değerleri karşılaştırılır. Eğer iki hash değeri eşleşirse, o zaman o alt dizilerin gerçekten eşleşip eşleşmediği kontrol edilir.

### Rabin-Karp Algoritması Adımları

- 1.Hash Fonksiyonu:** Algoritma, her bir alt dizi için bir hash değeri hesaplar. Hash fonksiyonu, karakterlerin her birini bir sayı ile temsil eder ve bu sayıları birleştirerek bir hash değeri oluşturur. Bu sayede, iki alt diziye karşılaştırmak yerine, onların hash değerleri karşılaştırılır.
- 2.Arama Adımları:**
  - Aranan alt dizinin hash değeri hesaplanır.
  - Metin üzerinde, her olası alt dizinin hash değeri hesaplanır.
  - Eğer bir alt dizinin hash değeri aranan alt dizinin hash değerine eşitse, o zaman bu iki alt dizi karşılaştırılır.
  - Eğer eşleşme sağlanmazsa, diğer alt diziler için hash değerleri hesaplanır.
- 3.Çakışmalar (Collisions):** Hash fonksiyonları, farklı dizilerin aynı hash değerini üretmesi durumunu "çakışma" (collision) olarak adlandırır. Rabin-Karp algoritmasında, hash değeri eşleşse bile dizilerin gerçekten eşleşip eşleşmediğini kontrol etmek gerekir. Bu, çakışmaları çözmek için yapılan bir ek adımdır.

### Avantajları

- ✓ Rabin-Karp algoritması, çoklu aramalar yapıldığında çok hızlıdır. Bir metin içinde birden fazla alt diziye aramak için verimli bir çözümdür.
- ✓ Hash fonksiyonu kullanarak, tek bir karşılaştırma ile alt dizilerin eşleşmesi kontrol edilir.

### Zaman Karmaşıklığı

Rabin-Karp algoritmasının zaman karmaşıklığı  $O(n + m)$ 'dir, ancak bu karmaşıklık hash fonksiyonunun performansına ve çakışma olasılığına bağlı olarak değişebilir.

- n: Metnin uzunluğu
- m: Aranan alt dizinin uzunluğu

En kötü durumda, algoritmanın zaman karmaşıklığı  $O(n * m)$ 'dir, çünkü çakışmalar olduğunda her alt dizi karşılaştırılmalıdır.

### Özet

**Rabin-Karp algoritması**, hashing kullanarak alt dizileri arayan bir algoritmadır. Özellikle çoklu aramalar yapıldığında ve çakışmaların yönetilmesi gerektiğinde etkilidir. Hashing sayesinde algoritma hızlı çalışır, ancak çakışmalar durumunda ek işlemler yapılması gerekir.



# String Matching Algoritmaları

## KMP (Knuth-Morris-Pratt) String Matching Algorithm

**KMP algoritması**, bir dizinin içinde belirli bir alt diziye aramak için kullanılan verimli bir arama algoritmasıdır. Bu algoritma, arama işlemi sırasında karşılaşılan karakter eşleşmelerini geri almaz, yani karakterleri yeniden kontrol etmez. Bu sayede daha hızlı çalışır. KMP algoritması, özellikle büyük metinlerde ve veri tabanlarında etkilidir.

### Temel Fikir

KMP algoritmasının temel fikri, metnin ve aranan alt dizinin karşılaştırılmasını hızlandırmaktır. Eğer eşleşme sağlanmazsa, algoritma bu noktada tekrar başa dönmekten, aranan alt dizinin uygun kısmına kayarak devam eder. Bunu, arama yapılan dizinin “prefix” (ön ek) ve “suffix” (son ek) ilişkisini kullanarak yapar.

### KMP Algoritması Adımları

#### 1. Preprocessing (Önişleme) - Prefix Suffix Tablosu (LPS)

KMP algoritması, öncelikle bir 'LPS' (Longest Prefix Suffix) tablosu oluşturur. Bu tablo, aranan alt dizinin her karakteri için, o karakterden önceki karakterlerle olan uzun 'prefix' ve 'suffix' eşleşmelerini tutar. Bu tablo, algoritmanın çalışma süresini iyileştirir. LPS tablosu, her karakter için şu bilgiyi sağlar:

- O karaktere kadar olan en uzun prefix, aynı zamanda bir suffix (son ek) de oluyorsa, o kadar karakteri atlayabiliriz.
- Bu sayede karşılaştırmaların tekrarı engellenir.

#### 2. Arama Adımları

- Aranan alt dizinin ilk karakteri, metnin ilk karakteri ile karşılaştırılır.
- Eşleşme sağlanmazsa, LPS tablosu kullanılarak, hangi karakterden başlanacağı belirlenir.
- LPS tablosu, hangi karakterin atlanabileceğini belirler ve bu sayede karşılaştırmaların çoğu atlanır.

### Avantajları

- ✓ KMP algoritması, her karakterin yalnızca bir kez kontrol edilmesini sağlar. Yani, en kötü durumda bile zaman karmaşıklığı  $O(n)$  olur ( $n$ , metnin uzunluğu).
- ✓ Bu, geleneksel doğrusal arama algoritmalarına göre önemli bir hız avantajı sağlar.

### Zaman Karmaşıklığı

KMP algoritmasının zaman karmaşıklığı  $O(n + m)$ 'dir.

- $n$ : Metnin uzunluğu
- $m$ : Aranan alt dizinin uzunluğu

Bu karmaşıklık, LPS tablosunun oluşturulması ve arama işleminin birlikte hesaplanmasından gelir.

### Özet

**KMP algoritması**, metin içinde alt diziye ararken, geri gitmek yerine ileri doğru adımlar atarak karşılaştırmaları verimli bir şekilde gerçekleştirir. Bu yöntem sayesinde büyük metinlerde arama işlemi çok daha hızlı hale gelir.



# String Matching Algoritmaları

## Boyer-Moore String Matching Algorithm

**Boyer-Moore algoritması**, metin içinde bir alt diziye aramak için kullanılan en verimli string matching algoritmalarından biridir. Özellikle büyük metinlerde arama yaparken çok etkilidir. Boyer-Moore algoritması, "from right to left" (sağdan sola) yaklaşımını kullanarak çalışır ve eşleşme sağlamadığında metni ileri doğru kaydırır. Bu sayede birçok karşılaştırmayı atlayabilir.

### Temel Fikir

Boyer-Moore algoritması, metni ve aranan alt diziye karşılaştırırken yalnızca önemli karşılaştırmaları yapar. Algoritma, karşılaşılan her hatayı (eşleşmeme durumunu) kullanarak alt diziye verimli bir şekilde kaydırır. Bu kaydırma işlemi, önceden belirlenen iki tablonun yardımıyla yapılır: bad character rule ve good suffix rule.

### Boyer-Moore Algoritması Adımları

- 1. Bad Character Rule:** Bu kural, metin üzerinde karşılaşılan hatalı karakterlerin (eşleşmeyen karakterlerin) nasıl kullanılacağını belirler. Eğer metnin bir karakteri aranan alt diziyle eşleşmezse, o zaman o karakterin bulunduğu konumdan sağa kaydırma yapılır. Kaydırma miktarı, aranan alt dizisindeki karakterin metinde bulunup bulunmadığına göre belirlenir. Eğer karakter bulunuyorsa, alt diziye o karakterin bulunduğu yere kaydırmak mantıklıdır.
- 2. Good Suffix Rule:** Bu kural, metin üzerinde eşleşme sağlanmış bir kısım varsa ve geri kalan kısmı eşleşmemişse, ne kadar kaydırma yapılacağını belirler. Eğer alt dizinin bir kısmı metinde eşleştiyse, bu kısmın sonrasındaki kısımların da kontrol edilmesine gerek yoktur. Bu sayede daha hızlı bir kaydırma yapılır. Eğer eşleşmeyen kısım varsa, alt dizi bu eşleşmeyen kısmın öncesine kaydırılır.
- 3. Arama Adımları:**
  - Arama yapılacak metin ile aranan alt dizi, sağdan sola karşılaştırılır.
  - Eğer bir eşleşme sağlanmazsa, bad character ve good suffix kuralları kullanılarak kaydırma yapılır.
  - Kaydırma işlemi her eşleşmesiz durumda tekrarlanır.

### Avantajları

- ✓ Boyer-Moore algoritması, büyük metinlerde arama yaparken verimlidir çünkü birçok karşılaştırma atlanır.
- ✓ Algoritma, sağdan sola yaklaşım sayesinde daha az karşılaştırma yaparak daha hızlı çalışır.

### Zaman Karmaşıklığı

Boyer-Moore algoritmasının zaman karmaşıklığı  $O(n/m)$ 'dir.

- n: Metnin uzunluğu
- m: Aranan alt dizinin uzunluğu

En kötü durumda, zaman karmaşıklığı  $O(n * m)$  olabilir, ancak genellikle çok daha hızlıdır.

### Özet

**Boyer-Moore algoritması**, sağdan sola karşılaştırma yaparak metin içinde alt diziye hızlı bir şekilde arar. Bad character ve good suffix kuralları sayesinde verimli bir şekilde kaydırma yapılır, bu da algoritmanın büyük metinlerde hızlı çalışmasını sağlar.



# String Matching Algoritmaları

## Z-Algorithm

**Z-Algorithm**, bir dizi (string) üzerinde yapılan bir algoritmadır ve genellikle string matching ve string manipülasyonu problemlerinde kullanılır. Bu algoritma, bir dizinin her bir indeksine kadar olan kendi kendine benzerlik uzunluğunu hesaplar.

### Z-Array

Z-Algorithm'ı anlamak için Z-array kavramını bilmemiz gerekir. Z-array, bir stringin her pozisyonu için, o pozisyondan başlayarak stringin en uzun eşleşmesini (prefix ile) bulur.

### Örneğin:

Bir string verelim:  $s = \text{"abac"}$

#### Z-array'i şöyle hesaplarız:

1. İlk karakterden başla: "abac" ile "abac" tam eşleşir,  $Z[0] = 4$ .
2. İkinci karakterden başla: "b" ile "abac" en başta 0 eşleşir,  $Z[1] = 0$ .
3. Üçüncü karakterden başla: "a" ile "abac" en başta "a" eşleşir,  $Z[2] = 1$ .
4. Dördüncü karakterden başla: "c" ile "abac" en başta 0 eşleşir,  $Z[3] = 0$ .

Sonuç olarak, Z-array şu olur:  $[4, 0, 1, 0]$

### Nasıl Çalışır?

**Z-Algorithm**, Z-array'in verimli bir şekilde hesaplanmasını sağlar. Bu işlemde, her bir indeksi sırasıyla kontrol ederiz, ancak her seferinde diziyi yeniden taramak yerine, daha önce hesapladığımız sonuçları kullanarak işlemi hızlandırırız.

### Adımlar:

1. İlk pozisyonda  $Z[0]$  her zaman stringin uzunluğudur.
2. Diğer indekslerde, önceki hesaplamalardan faydalanarak benzerlikleri buluruz.
3. Eğer bir eşleşme bulunursa, bunu Z-array'e yazarız.
4. Bu işlem bir döngü ile gerçekleştirilir ve genellikle  $O(n)$  zaman karmaşıklığına sahiptir.

### Kullanım Alanları

**Z-Algorithm**, string matching (örneğin, bir desenin ana dizide aranması) gibi problemlerde çok kullanışlıdır. Özellikle, bir dizi içindeki belirli alt dizileri hızlıca bulmak için kullanılır.

### Özet

**Z-Algorithm**, bir stringin her pozisyonundaki kendi kendine benzerlik uzunluklarını hesaplayarak, string manipülasyonları ve arama işlemleri için verimli bir çözüm sunar. Bu algoritma, genellikle daha karmaşık işlemleri optimize etmek için kullanılır.



# Çok Değişkenli Algoritmalar

## Simulated Annealing (Simüle Edilen Tavlama) Algoritması

**Simulated Annealing (SA)**, çok değişkenli optimizasyon problemleri için kullanılan bir küresel arama algoritmasıdır. Temelde fiziksel tavlama sürecine dayanır ve global optimuma ulaşmayı amaçlar.

### Temel Kavramlar

- **Tavlama:** Fizikte, bir maddeyi yüksek sıcaklıkta eritip sonra yavaşça soğutma işlemine "tavlama" denir. Bu süreçte madde düzensiz bir yapıdan düzenli bir yapıya (yani minimum enerji durumuna) geçer. SA algoritması, bu süreci matematiksel bir modelle benzetir.
- **Ekran Değişimi (State Transition):** SA algoritmasında her bir çözüm bir "durum" olarak kabul edilir. Algoritma, çözüm alanındaki bir durumdan başka bir duruma geçer. Bu geçişin enerjisi, çözümün kalitesine göre belirlenir.
- **Sıcaklık (Temperature):** Sıcaklık, algoritmanın çözüm alanındaki yeni durumlara "açıklığı"nı kontrol eder. Yüksek sıcaklıkta, algoritma kötü çözüm tekliflerine de "açık"tır, bu da daha geniş bir arama alanı sağlar.

### Algoritma Adımları

1. **Başlangıç Durumu:** Başlangıçta bir çözüm seçilir ve bir sıcaklık değeri belirlenir.
2. **Yeni Çözüm Üretimi:** Mevcut çözümden küçük değişiklikler ile yeni bir çözüm üretilir.
3. **Değişim Kararı:** Eğer yeni çözüm daha iyi ise, hemen kabul edilir. Eğer daha kötü ise, kötü çözüm belirli bir olasılıkla kabul edilir. Bu olasılık, sıcaklık değerine bağlıdır.
4. **Sıcaklık Güncellemesi:** Sıcaklık yavaşça düşürülür. Sıcaklık düştükçe, daha kötü çözümler kabul edilme olasılığı azalır.
5. **Sonuç:** Sıcaklık yeterince düşük olduğunda algoritma durur ve en iyi bulunan çözüm geri döndürülür.

### Avantajları

- ✓ Küresel optimizasyon yapma yeteneği, yerel minimumlara takılmaktan kaçınmayı sağlar.
- ✓ Karmaşık ve çok değişkenli problemlerde etkin bir şekilde çalışır.

### Dezavantajları

- ✗ Çözüm arama süresi, sıcaklık düşüş hızına bağlıdır. Hızlı bir düşüş, erken bir durdurma noktasına neden olabilir.
- ✗ Parametre ayarları (başlangıç sıcaklığı, sıcaklık düşüş oranı) algoritmanın başarısını etkileyebilir.

### Kullanım Alanları

Simulated Annealing, özellikle aşağıdaki alanlarda kullanılır:

- **Makine öğrenimi:** Model optimizasyonu ve parametre ayarları.
- **Yapay zeka:** Yüksek boyutlu arama alanlarında çözüm bulma.
- **Endüstriyel optimizasyon:** Üretim süreçlerinde en iyi tasarımı ve kaynak tahsisini bulma.



# Çok Değişkenli Algoritmalar

## Genetic Algorithms (Genetik Algoritmalar)

**Genetik Algoritmalar (GA)**, biyolojik evrim sürecine dayalı bir optimizasyon yöntemidir. Doğal seleksiyon, genetik çeşitlilik ve genetik miras gibi biyolojik kavramları kullanarak çözüm uzayında en iyi çözümü bulmayı hedefler.

### Temel Kavramlar

- **Genetik Seçim:** Biyolojik evrimde en güçlü bireylerin hayatta kalıp genetik materyali bir sonraki nesle aktardığına dayanır. GA'da da en iyi çözümler seçilir ve "ebeveyn" olarak kullanılır.
- **Krossover (Crossover):** İki ebeveynin genetik bilgilerini birbirine aktararak yeni bireyler üretmesidir. Bu işlem, ebeveynlerin genetik özelliklerini birleştirir ve yeni çözüm adayları oluşturur.
- **Mutasyon:** Mutasyon, bir bireyin genetik bilgisinde rastgele değişiklikler yaparak çeşitlilik sağlar. Bu, evrim sürecinin çeşitliliği korumasına benzer.
- **Popülasyon:** Çözüm uzayında bir dizi çözüm oluşturulur. Bu çözüm kümesi, "popülasyon" olarak adlandırılır ve her birey, problem için bir çözümü temsil eder.
- **Fitness (Uygunluk) Fonksiyonu:** Her bireyin çözüm kalitesini değerlendiren bir fonksiyondur. Fitness fonksiyonu, bir çözümün ne kadar iyi olduğunu belirler.

### Algoritma Adımları

1. **Başlangıç Popülasyonu:** İlk popülasyon rastgele oluşturulur. Bu popülasyon, çözümün genetik temsilini içeren bireylerden oluşur.
2. **Fitness Değerlendirmesi:** Popülasyondaki her birey için fitness fonksiyonu hesaplanır.
3. **Seçim (Selection):** Yüksek fitness değerine sahip bireyler, ebeveyn olarak seçilir. Bu seçim, genellikle turnuva veya rulet tekerleği gibi yöntemlerle yapılır.
4. **Krossover (Crossover):** Seçilen ebeveynlerden yeni bireyler oluşturulmak üzere krossover işlemi yapılır.
5. **Mutasyon:** Yeni bireyler arasında çeşitlilik oluşturmak için mutasyon işlemi uygulanır.
6. **Yeni Popülasyon:** Yeni nesil popülasyon, seleksiyon, krossover ve mutasyon işlemlerinin ardından oluşturulur.
7. **Durma Kriteri:** Algoritma, belirli bir iterasyon sayısına ulaştığında veya yeterince iyi bir çözüm bulunduğunda durur.

### Avantajları

- ✓ Karmaşık ve büyük çözüm uzaylarında etkili sonuçlar elde edebilir.
- ✓ Yerel minimumlara takılmadan küresel optimuma yakın çözümler bulabilir.
- ✓ Paralel işlem yapma imkânı sağlar, bu da hızlanma sağlayabilir.

### Dezavantajları

- ✗ Optimal çözümü bulma garantisi yoktur, bazen yerel optimumda kalabilir.
- ✗ Çözümün kalitesini etkileyen parametrelerin (seçim, krossover oranı, mutasyon oranı) iyi ayarlanması gerekir.
- ✗ Hesaplama maliyetleri yüksek olabilir, özellikle büyük popülasyonlarla çalışırken.

### Kullanım Alanları

Genetik Algoritmalar, aşağıdaki alanlarda yaygın olarak kullanılır:

- **Makine öğrenimi:** Özellik seçimi, model optimizasyonu.
- **Yapay zeka:** Stratejik oyunlar, robotik planlama.
- **Endüstriyel optimizasyon:** Tasarım optimizasyonu, üretim süreçleri.



# Çok Değişkenli Algoritmalar

## Ant Colony Optimization (Karıncı Kolonisi Optimizasyonu)

**Ant Colony Optimization (ACO)**, karıncaların doğal davranışlarından ilham alarak geliştirilmiş bir küresel optimizasyon algoritmasıdır. Bu algoritma, özellikle kombinatoriyal optimizasyon problemlerinde kullanılır ve çözüm uzayında en iyi çözüme ulaşmayı hedefler.

### Temel Kavramlar

- **Karıncı Davranışı:** Doğal karıncalar, yiyecek kaynaklarını bulmak için birbirleriyle feromon salarak iletişim kurarlar. Bu feromonlar, diğer karıncaların yolunu yönlendirir ve zamanla daha kısa yollar daha fazla feromon biriktirir.
- **Feromon:** Karıncaların yol üzerinde bıraktığı kimyasal izlerdir. Algoritmada feromon, çözümün kalitesini temsil eder. Yüksek feromon yoğunluğuna sahip yollar, diğer karıncalar tarafından daha fazla tercih edilir.
- **Evaporizasyon:** Feromonlar zamanla buharlaşarak kaybolur. Bu, algoritmanın zaman içinde kötü yolların etkisini azaltmasına yardımcı olur.
- **Seçim ve Güncelleme:** Karıncalar, çözüm bulmak için feromon yoğunluğuna göre yolları seçerler. Zamanla, daha iyi çözümler daha fazla feromon biriktirir ve bu yollar daha fazla tercih edilir.

### Algoritma Adımları

1. **Başlangıç Popülasyonu:** İlk olarak, her karınca çözüm alanına rastgele yerleştirilir.
2. **Feromon Bırakma:** Karıncalar, çözüm bulduktan sonra yol üzerinde feromon bırakırlar. Bu feromon miktarı, çözümün kalitesine göre değişir.
3. **Feromon Evaporizasyonu:** Feromonlar zamanla buharlaşır ve eski çözümler daha az tercih edilir.
4. **Karıncı Hareketi:** Karıncalar, çözüm alanında hareket ederken feromon yoğunluğuna göre yolları seçerler.
5. **Sonuç Güncelleme:** En iyi çözüm bulunduğunda, bu çözüm üzerine feromon bırakılır ve popülasyondaki diğer karıncalar bu yolu tercih etmeye başlar.

### Avantajları

- ✓ Büyük çözüm uzaylarında küresel optimizasyon yapabilme yeteneği vardır.
- ✓ Yerel minimuma takılmadan daha iyi sonuçlar elde edebilir.
- ✓ Dağıtık yapıda çalıştığı için paralel işlem yapılabilir.

### Dezavantajları

- ✗ Hesaplama maliyeti yüksek olabilir, özellikle büyük popülasyonlarla çalışırken.
- ✗ Parametre ayarları (feromon yoğunluğu, buharlaşma oranı) algoritmanın başarısını etkiler.
- ✗ Optimum çözüm bulma garantisi yoktur, bazen yerel optimumda kalabilir.

### Kullanım Alanları

Ant Colony Optimization, aşağıdaki alanlarda yaygın olarak kullanılır:

- **TSP (Traveling Salesman Problem):** En kısa yolu bulma problemleri.
- **Robotik:** Yolların ve rotaların optimizasyonu.
- **Ağ Tasarımı:** Veri iletim yollarının ve ağ yapılarının optimize edilmesi.



# Benzerlik ve Kümeleme Algoritmaları

## K-Means Clustering Algoritması

### K-Means Clustering Nedir?

K-Means Clustering, verileri belirli sayıda kümeye ayırmak için kullanılan bir kümeleme algoritmasıdır. Veriler, önceden belirlenen k sayıda küme merkezine (centroid) en yakın olacak şekilde gruplandırılır.

### K-Means Algoritmasının Çalışma Mantığı

- Başlangıç Noktaları Belirleme:** Kullanıcı tarafından belirlenen k sayısı kadar rastgele küme merkezi seçilir.
- Veri Noktalarının Küme Merkezlerine Atanması:** Her veri noktası, en yakın küme merkezine atanır.
- Yeni Küme Merkezlerini Hesaplama:** Her kümenin ortalaması alınarak yeni küme merkezleri belirlenir.
- Tekrar Etme:** Veri noktalarının yeni küme merkezlerine atanması ve merkezlerin güncellenmesi işlemi, küme merkezleri sabit kalana veya belirli bir iterasyon tamamlanana kadar devam eder.

### Avantajları

- ✓ Basit ve hızlıdır.
- ✓ Büyük veri setlerinde etkili çalışır.
- ✓ Küme sayısı önceden belirlendiğinde iyi sonuçlar verebilir.

### Dezavantajları

- ✗ Küme sayısının (k) önceden belirlenmesi gerekir.
- ✗ Başlangıç noktaları farklı sonuçlara yol açabilir.
- ✗ Küresel olmayan kümeler için iyi çalışmaz.

### Kullanım Alanları

- Müşteri segmentasyonu
- Görüntü sıkıştırma
- Anomali tespiti
- Öneri sistemleri

**K-Means algoritması**, özellikle büyük ve anlamlı veri kümelerini ayrıştırmak için sıkça kullanılan etkili bir yöntemdir.



# Benzerlik ve Kümeleme Algoritmaları

## Hierarchical Clustering Algoritması

### Hierarchical Clustering Nedir?

**Hierarchical Clustering (Hiyerarşik Kümeleme)**, veri noktalarını hiyerarşik bir yapı içinde kümeleştiren bir kümeleme algoritmasıdır. Bu yöntem, verileri iç içe geçmiş bir ağaç yapısında (dendrogram) gruplandırarak kümeler oluşturur.

### Hierarchical Clustering Algoritmasının Çalışma Mantığı

Hiyerarşik kümeleme, iki temel yaklaşıma sahiptir:

#### 1. Aşağıdan Yukarıya (Agglomerative) Yaklaşım

- Her veri noktası başlangıçta kendi başına bir küme olarak kabul edilir.
- En yakın iki küme birleştirilerek yeni bir küme oluşturulur.
- Bu işlem, tek bir küme kalana kadar devam eder.

#### 2. Yukarıdan Aşağıya (Divisive) Yaklaşım

- Tüm veri noktaları tek bir küme olarak başlar.
- Küme, en farklı veri noktalarına göre alt kümelere ayrılır.
- Bu işlem, her veri noktası kendi başına bir küme olana kadar devam eder.

### Avantajları

- ✓ Küme sayısını önceden belirlemek gerekmez.
- ✓ Kümeleme sonuçları dendrogram ile görselleştirilebilir.
- ✓ Kümeleme yapısı, farklı seviyelerde analiz edilebilir.

### Dezavantajları

- ✗ Büyük veri setlerinde hesaplama maliyeti yüksektir.
- ✗ Hatalı kümeleme yapıldığında geri dönüşü zordur.
- ✗ Veri noktaları arasındaki mesafe ölçümüne çok duyarlıdır.

### Kullanım Alanları

- Genetik analizler
- Müşteri segmentasyonu
- Belge kümeleme
- Görüntü işleme

**Hierarchical Clustering algoritması**, özellikle kümeleme yapısını görselleştirmek ve veri gruplarını farklı seviyelerde analiz etmek için etkili bir yöntemdir.



# Benzerlik ve Kümeleme Algoritmaları

## Nearest Neighbors Algoritması

### Nearest Neighbors Nedir?

**Nearest Neighbors (En Yakın Komşular)**, veri noktaları arasındaki benzerliği ölçerek sınıflandırma ve kümeleme yapan bir algoritmadır. En çok bilinen versiyonu K-Nearest Neighbors (KNN) olarak adlandırılır ve bir veri noktasının sınıfını, en yakın komşularının sınıfına göre belirler.

### Nearest Neighbors Algoritmasının Çalışma Mantığı

- 1. Mesafe Ölçümü:** Yeni bir veri noktası ile veri kümesindeki diğer noktalar arasındaki mesafe hesaplanır. Genellikle Öklid mesafesi kullanılır.
- 2. En Yakın Komşuların Seçilmesi:** Belirlenen k değeri kadar en yakın komşu seçilir.
- 3. Sonuç Belirleme:**
  - Sınıflandırma (Classification): Komşuların çoğunluk sınıfı, yeni veri noktasının sınıfı olarak atanır.
  - Regresyon (Regression): Komşuların ortalaması alınarak tahmin yapılır.

### Avantajları

- ✓ Kolay anlaşılır ve uygulanabilir.
- ✓ Önceden bir eğitim aşaması gerektirmez, anlık kararlar alabilir.
- ✓ Küçük ve orta ölçekli veri setlerinde etkili çalışır.

### Dezavantajları

- ✗ Büyük veri setlerinde hesaplama maliyeti yüksektir.
- ✗ Gürültülü verilerden etkilenebilir.
- ✗ K değeri yanlış seçildiğinde doğruluk düşebilir.

### Kullanım Alanları

- Öneri sistemleri
- Sahtecilik tespiti
- Tıbbi teşhis
- Görüntü tanıma

**Nearest Neighbors algoritması**, özellikle benzer veri noktalarını gruplamak ve yeni gelen verileri doğru şekilde sınıflandırmak için yaygın olarak kullanılan bir yöntemdir.



# Benzerlik ve Kümeleme Algoritmaları

## DBSCAN Algoritması

### DBSCAN Nedir?

**DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**, yoğunluk tabanlı bir kümeleme algoritmasıdır. Kümeleri yoğunluk bölgelerine göre belirler ve gürültülü (aykırı) verileri ayırt edebilir.

### DBSCAN Algoritmasının Çalışma Mantığı

1. **Başlangıç:** Rastgele bir veri noktası seçilir.
2. **Yoğunluk Kontrolü:** Seçilen noktanın belirlenen mesafe (epsilon,  $\epsilon$ ) içindeki komşu sayısı incelenir.
3. **Kümeleme:**
  - Eğer yeterli sayıda komşuya sahipse, bu nokta bir kümenin çekirdeği olarak belirlenir ve komşuları kümeye dahil edilir.
  - Eğer yeterli komşuya sahip değilse, bu nokta gürültü olarak işaretlenir.
4. **Genleşme:** Yeni eklenen noktalar da aynı şekilde kontrol edilir ve kümeler genişletilir.
5. **Tekrar:** Tüm noktalar ziyaret edilene kadar işlem devam eder.

### Avantajları

- ✓ Küme sayısını önceden belirlemeye gerek yoktur.
- ✓ Aykırı (gürültü) verileri tespit edebilir.
- ✓ Farklı şekillerdeki kümeleri başarılı bir şekilde yakalayabilir.

### Dezavantajları

- ✗ Küme yoğunlukları çok farklı olduğunda iyi çalışmaz.
- ✗ Büyük veri setlerinde hesaplama maliyeti yüksektir.
- ✗ Parametrelerin ( $\epsilon$  ve minimum komşu sayısı) doğru seçilmesi gerekir.

### Kullanım Alanları

- Anomali tespiti
- Coğrafi veri analizi
- Görüntü işleme
- Müşteri segmentasyonu

**DBSCAN algoritması**, özellikle yoğunluk bazlı kümeleme gerektiren veri setlerinde etkili bir yöntemdir ve klasik kümeleme algoritmalarına göre daha esnektir.



# Diğer Önemli Algoritmalar

## Bloom Filter Algoritması

### Bloom Filter Nedir?

**Bloom Filter**, bir öğenin bir kümede olup olmadığını belirlemek için kullanılan, az yer kaplayan ve hızlı sorgulama yapabilen olasılıksal bir veri yapısıdır. Kesin sonuç vermemekle birlikte, yanlış pozitif sonuçlara izin verir ancak yanlış negatif sonuç vermez.

### Çalışma Mantığı

- Başlangıç:** Boş bir bit dizisi oluşturulur.
- Öğe Ekleme:**
  - Öğe, birden fazla hash fonksiyonundan geçirilerek belirli bit dizisi konumları belirlenir.
  - Bu konumlardaki bitler 1 olarak işaretlenir.
- Öğe Sorgulama:**
  - Öğe tekrar hash fonksiyonlarından geçirilir ve ilgili bit konumları kontrol edilir.
  - Eğer tüm konumlar 1 ise, öğe büyük ihtimalle kümededir (ama kesin değildir).
  - Eğer en az bir bit 0 ise, öğe kesinlikle kümede değildir.

### Avantajları

- ✓ Bellek verimli bir veri yapısıdır.
- ✓ Sabit süre içinde arama yapılabilir.
- ✓ Hash tablolarına göre daha az yer kaplar.

### Dezavantajları

- ✗ Yanlış pozitif sonuçlar verebilir (olmayan bir öğeyi varmış gibi gösterebilir).
- ✗ Eklenen öğeler silinemez (gelişmiş varyantlar hariç).
- ✗ Veri yapısı doldukça yanlış pozitif oranı artar.

### Kullanım Alanları

- Web tarayıcılarında kötü amaçlı URL filtreleme
- Veritabanı sorgularında önbellekleme
- Büyük veri setlerinde hızlı üyelik sorgulama
- Ağ güvenliği ve izleme sistemleri

**Bloom Filter algoritması**, özellikle büyük ölçekli veri kümelerinde hızlı ve hafif bir üyelik testi yapmak için kullanışlı bir yöntemdir.



# Diğer Önemli Algoritmalar

## Reservoir Sampling Algoritması

**Reservoir Sampling**, belirli bir sayıda öge seçmek için kullanılan bir algoritmadır. Bu algoritma, bilinmeyen büyüklükteki bir veri akışından (stream) veya çok büyük veri kümesinden rastgele örnekler seçmek için kullanılır. Bu, verinin tamamını belleğe yükleyemediğimiz durumlar için oldukça faydalıdır.

### Temel Kavram

Bir veri akışından, her bir ögenin seçilme olasılığını eşit tutmak amacıyla, sürekli bir şekilde yeni verilerle rastgele örnekler seçilir. Bu, özellikle verilerin büyüklüğü önceden bilinmeyen ya da çok büyük olduğu durumlarda etkili bir yöntemdir.

### Algoritmanın Adımları

#### 1. Başlangıç Durumu:

- İlk adımda, ilk  $k$  öge (örnek sayısı kadar) doğrudan rezervuara eklenir.

#### 2. Daha Fazla Veri Geldikçe:

- Verinin  $k+1$ . ögesi geldiğinde, bir rastgele sayı  $r$  ( $0$  ile  $i$  arasında, burada  $i$  o anki öge numarası) üretilir.
- Eğer  $r$  değeri  $k$ 'den küçükse, mevcut rezervuardaki bir öge rastgele seçilir ve yerine bu yeni öge eklenir.

#### 3. Sonuç:

- Algoritma, tüm veri akışı tamamlandığında, rezervuar içindeki  $k$  ögeyi, veri akışındaki rastgele seçilmiş örnekler olarak sunar.

### Örnek

- Varsayalım ki, veri akışından 3 öge ( $k=3$ ) seçmek istiyoruz.
- İlk 3 öge doğrudan rezervuara alınır:  $[X1, X2, X3]$ .
- 4.öge  $X4$  geldiğinde:
  - Rastgele bir sayı üretilir, örneğin  $r = 2$ .
  - Eğer  $r < k$  ise, rezervuardan bir öge çıkarılır ve yerine  $X4$  eklenir. Bu adımlar veri akışı sonlanana kadar devam eder.

### Avantajları

✓ **Bellek Verimliliği:** Yalnızca  $k$  kadar veri saklanır, yani çok büyük veri kümeleri ile çalışırken bellek tasarrufu sağlar.

✓ **Eşit Seçim Olasılığı:** Her veri ögesinin seçilme olasılığı eşittir.

✓ **Sürekli Veri Akışı:** Veri akışının başlangıçta uzunluğu bilinmediğinde, dinamik bir şekilde örnekleme yapabilirsiniz.

### Kullanım Alanları

- Büyük veri kümesi örnekleme (veri analitiği).
- Akış verisi analizi.
- Online algoritmalar.



# Diğer Önemli Algoritmalar

## Karnaugh Haritası (Boolean Minimization)

### Basitçe: Karmaşık ifadeleri sadeleştirme sanatı

**Karnaugh Haritası**, dijital devre tasarımında kullanılan etkili ve görsel bir sadeleştirme tekniğidir. Mantık ifadelerinde yer alan gereksiz tekrarları ortadan kaldırarak, çok daha az işlemle aynı sonucu almayı mümkün kılar. Bu sayede daha küçük, daha hızlı ve daha verimli dijital sistemler tasarlanabilir.

### Nasıl Çalışır?

1. İfadeyi Tanı
2. Önce sadeleştirmek istediğin Boolean fonksiyonuna bakarsın. Hangi girişlerde “1” çıktısı verdiğini bilmek işin başlangıcıdır.
3. Görselleştir
4. Giriş kombinasyonlarını zihninde veya bir düzende yerleştir. Örneğin iki girişli bir sistemde dört olasılık vardır:
  - Her iki giriş sıfır
  - Birinci giriş bir, ikinci giriş sıfır
  - Birinci giriş sıfır, ikinci giriş bir
  - İkisi de bir
5. Bu olasılıkların hangilerinde çıkış “1” ise onları işaretlersin.
6. Benzerleri Grupla
7. Ardından aynı sonucu veren, birbirine yakın olan durumları gruplayarak ortak özellikleri yakalarsın. Mesela iki farklı durumda sadece bir değişkenin değiştiğini görüyorsan, o değişkenin etkisiz olduğunu fark edebilirsin. Bu da sadeleştirme fırsatıdır.
8. İfadeyi Yaz
9. Gruplar üzerinden, ortak olan giriş değişkenlerini kullanarak yeni bir sade ifade yazarsın. Bu yeni ifade, öncekiyle aynı sonucu verir ama çok daha az karmaşıktır.

### Neden Önemli?

- Görsel düşünmeyi kolaylaştırır. Rakamlar ve semboller yerine, yapıyı görüp sadeleştirirsin.
- Devre tasarımını kolaylaştırır. Daha az kapı (gate) ile aynı işi yapabilirsin.
- Zamandan ve maliyetten tasarruf sağlar.

### Kullanım Alanları

- Dijital mantık devrelerinde
- Mikrodenetleyici tasarımlarında
- Otomasyon sistemlerinde
- Akış kontrol devrelerinde
- Durum makineleri kurulumunda

### Özetle

**Karnaugh Haritası**, matematiksel karmaşayı azaltıp görsel çözümleme yapmamızı sağlar. Az sayıda giriş için son derece etkilidir ve en sade halini hızlıca bulmamıza yardımcı olur. İki veya dört değişkenli sistemlerde özellikle kullanışlıdır.



# Diğer Önemli Algoritmalar

## Consensus Algoritmaları (Paxos, Raft, vb.)

**Consensus algoritmaları**, dağıtık sistemlerde birden fazla düğümün (node) uyum içinde çalışmasını sağlayan algoritmalarlardır. Bu algoritmalar, sistemdeki tüm düğümlerin ortak bir karar almasını ve veriye tutarlı bir şekilde erişilmesini sağlar.

Dağıtık sistemlerde, düğümler arasında iletişim gecikmeleri, ağ hataları veya düğüm arızaları olabilir. Bu durumlar, verinin tutarsız hale gelmesine yol açabilir. Consensus algoritmaları, bu tür durumlarla başa çıkmak için kullanılır.

### Temel Kavram

- **Dağıtık Sistemler:** Birden fazla bilgisayarın, birbirleriyle iletişim kurarak bir sistemi oluşturduğu yapılar.
- **Consensus (Mutabakat):** Dağıtık sistemlerde, tüm düğümlerin bir konuda anlaşması gerektiğinde kullanılan bir kavramdır. Her düğüm, diğer düğümlerle uyum içinde kararlar alır.

### Consensus Algoritmalarının Temel Amacı

- **Veri Tutarlılığı:** Farklı düğümlerin aynı veriye aynı anda ve tutarlı bir şekilde erişebilmesini sağlamak.
- **Hata Toleransı:** Düğümler arızalandığında veya iletişimde aksama olduğunda bile sistemin tutarlı kalmasını sağlamak.
- **Ağ Kesintileri ve Gecikmeleri:** Ağda bir kesinti veya gecikme olduğunda bile kararların alınmasını sağlamak.

### Yaygın Consensus Algoritmaları

#### 1. Paxos

**Paxos**, dağıtık sistemlerde consensus sağlamak için kullanılan klasik bir algoritmadır. Amaç, birden fazla düğüm arasında tutarlı bir karar almayı sağlamaktır.

#### Temel Adımlar

- **Önerici (Proposer):** Bir değer önerir.
- **Kabul Edici (Acceptor):** Bu değeri kabul eder veya reddeder.
- **Öğrenici (Learner):** Kabul edilen değeri öğrenir.

**Paxos algoritması**, karmaşık olabilir ve genellikle çok sayıda ağ mesajı gerektirir. Ancak, güvenilir ve sağlamdır.

#### 2. Raft

Raft, Paxos algoritmasından daha anlaşılır bir alternatif olarak tasarlanmıştır. Genellikle daha pratik ve uygulanabilir olduğu için, modern dağıtık sistemlerde daha yaygın kullanılır.

#### Temel Adımları

- **Lider Seçimi:** İlk olarak, bir lider seçilir (leader election). Bu lider, tüm kararları koordine eder.
- **Kayıt Güncelleme:** Lider, diğer düğümlere (followers) kararlarını ve güncellemelerini iletir.
- **Çoğunluk Kararı:** Karar, çoğunluk tarafından kabul edilirse, işlem uygulanır.

Raft, daha az mesajlaşma gerektiren ve daha kolay anlaşılır bir algoritma olduğu için genellikle tercih edilir.

#### 3. Diğer Consensus Algoritmaları

- **Zab:** Raft'a benzer şekilde, bir lider seçim algoritmasıdır, ancak daha büyük sistemlerde daha verimli olabilir.
- **Viewstamped Replication:** Raft'a benzer bir algoritmadır ve özellikle yüksek kullanılabilirlik gereksinimlerinde kullanılır.



# Diğer Önemli Algoritmalar

## Consensus Algoritmalarının Kullanım Alanları

- **Dağıtık Veritabanları:** Veritabanlarında tutarlılığı sağlamak için kullanılır.
- **Blockchain Teknolojisi:** Konsensüs, blokların onaylanması ve doğrulanması için gereklidir.
- **Mikroservis Mimarileri:** Farklı servislerin ortak bir karar alması gerektiğinde, consensus algoritmaları devreye girer.
- **Hata Toleransı Gereksinimleri:** Yüksek erişilebilirlik ve hata toleransı olan sistemlerde yaygın olarak kullanılır.

## Avantajları

- ✓ **Tutarlılık:** Dağıtık sistemlerde veri tutarlılığını sağlar.
- ✓ **Yüksek Erişilebilirlik:** Düğümler arasında hata olsa bile sistemin çalışmaya devam etmesini sağlar.
- ✓ **Güvenilirlik:** Ağ hatalarına ve düğüm arızalarına karşı güvenilir bir sistem sağlar.

## Dezavantajları

- ✗ **Bürokratik Olabilir:** Özellikle Paxos gibi algoritmalar çok fazla iletişim gerektirebilir.
- ✗ **Performans Sorunları:** Ağda gecikmeler olduğunda performans sorunları oluşabilir.
- ✗ **Karmaşıklık:** Özellikle büyük sistemlerde, konsensüs algoritmalarının yönetilmesi zor olabilir.

## Kullanım Alanları:

- **Dağıtık Veritabanları:** Dağıtık veritabanlarında verilerin tutarlı ve güncel kalması için consensus algoritmaları kullanılır. Bu algoritmalar, veritabanı düğümleri arasında veri senkronizasyonunu sağlayarak, sistemin farklı düğümlerinde aynı veriye tutarlı bir şekilde erişilmesini garanti eder.
- **Blockchain Teknolojisi:** Blockchain sistemlerinde, her yeni blok eklenmeden önce ağdaki tüm katılımcıların onayı gereklidir. Consensus algoritmaları, blokların doğruluğunu onaylamak ve şüpheli durumları engellemek için kritik rol oynar. Bu, blockchain'in merkeziyetsiz yapısının güvenliğini ve sağlamlığını sağlar.
- **Mikroservis Mimarileri:** Mikroservis mimarilerinde, farklı servislerin birbirleriyle tutarlı bir şekilde çalışabilmesi için ortak kararlar alınması gerekebilir. Consensus algoritmaları, bu tür kararların doğru ve güvenli bir şekilde alınmasını sağlayarak, sistemin bütünsel çalışmasını mümkün kılar.
- **Yüksek Erişilebilirlik ve Hata Toleransı Gereksinimleri:** Büyük ve karmaşık sistemlerde, hata toleransı çok önemlidir. Dağıtık sistemler, ağdaki kesintiler, düğüm arızaları veya gecikmeler nedeniyle veri tutarsızlığına yol açabilir. Consensus algoritmaları, ağ hataları veya düğüm çökmeleri gibi durumlar yaşandığında bile sistemin tutarlı ve erişilebilir olmasını sağlar.



# Diğer Önemli Algoritmalar

## MapReduce Algoritması

**MapReduce**, büyük veri kümelerini parçalayıp işleyerek anlamlı sonuçlar üretmeyi amaçlayan güçlü bir algoritmadır. Bu algoritma, veriyi birden çok işlemciye dağıtarak paralel bir şekilde işler ve sonrasında bu parçaların sonuçlarını birleştirir.

### Nasıl Çalışır?

MapReduce iki ana aşamaya dayanır: Map (Haritalama) ve Reduce (Azaltma).

#### 1. Map Aşaması – Veriyi Parçala ve İşle

- Veriyi küçük parçalara böleriz.
- Her parça anahtar-değer (key-value) çiftine dönüştürülür.
- Örnek veri:
- ["a", "b", "a", "c", "b"]
- Map sonucu:
- a -> 1, b -> 1, a -> 1, c -> 1, b -> 1

#### 2. Shuffle and Sort – Aynı Anahtarları Gruplandır

- Aynı anahtara sahip değerler gruplanır.
- Gruplama sonucu:
- a: [1, 1]
- b: [1, 1]
- c: [1]

#### 3. Reduce Aşaması – Sonuçları Birleştir

- Gruplanan verilerdeki değerler birleştirilir (örneğin, sayılar toplanır).
- Reduce sonucu:
- a: 2
- b: 2
- c: 1

#### 4. Sonuç

Tüm bu adımların sonunda, işlenmiş ve anlamlı hale gelmiş bir sonuç elde edilir.

**Sonuç: {"a": 2, "b": 2, "c": 1}**

### Neden MapReduce Kullanılır?

- **Paralel Çalışır:** Verileri birden fazla işlemciye dağıtarak hız kazandırır.
- **Dağıtık Sistem Dostu:** Veriler birçok bilgisayarda işlenebilir.
- **Hata Toleransı:** Her işlem tekrar edilebilir, bu da veri kaybını önler.
- **Esneklik:** Metin, log dosyaları, görseller gibi farklı türde veriler işlenebilir.

### Kullanım Alanları

- **Kelime Sayma:** Metinlerdeki kelimelerin kaç kez geçtiğini bulmak.
- **Log Analizi:** Web sunucularının kayıtlarını inceleyerek istatistikler çıkarmak.
- **Veri Sıralama:** Büyük veri kümelerini sıralamak.
- **Arama Motorları:** Web sitelerini indeksleyip anahtar kelimelere göre sıralama yapmak.

### Özet

**MapReduce**, devasa veri setlerini etkili bir şekilde işlemek için kullanılan bir yöntemdir. Paralel işlem yapabilmesi ve hata toleransına sahip olması, onu büyük veriyi hızlı ve verimli bir şekilde işlemek için vazgeçilmez bir yaklaşım yapar.



# Diğer Önemli Algoritmalar

## Monte Carlo Yöntemleri

**Monte Carlo yöntemleri**, karmaşık problemlerin çözülmesinde kullanılan olasılık teorisi ve rastgele örnekleme tekniklerine dayalı algoritmalar. Bu yöntemler, analitik çözümler için uygun olmayan problemleri çözmek amacıyla kullanılır.

### Temel Kavram

Monte Carlo yöntemleri, belirli bir sorunun çözümünü rastgele örnekler kullanarak yaklaşık olarak elde etmeyi amaçlar. Genellikle karmaşık veya analitik çözümü zor olan problemlerde kullanılır.

### Adımlar

1. Rastgele Sayılar Üretme:
2. Problemi temsil eden rastgele sayılar üretilir.
3. Simülasyon:
4. Üretilen sayılarla simülasyon yapılır.
5. Sonuçları Hesaplama:
6. Simülasyon sonuçları kullanılarak bir hesaplama yapılır.
7. Yaklaşık Sonuç:
8. Elde edilen veriler ortalanarak yaklaşık bir sonuç tahmin edilir.

Örnek: Pi Sayısının Hesaplanması

1. Çeyrek çembere rastgele noktalar atılır.
2. Noktalar çeyrek çemberin içinde mi dışında mı olduğuna göre belirlenir.
3. Pi sayısı, çeyrek çembere düşen noktaların oranı ile yaklaşık olarak hesaplanır:

$$\pi \approx 4 \times \left( \frac{\text{Çeyrek çember içine düşen noktalar}}{\text{Toplam noktalar}} \right)$$

### Avantajları

- ✓ Basitlik ve esneklik
- ✓ Paralel işleme uygunluk

### Dezavantajları

- ✗ Yavaş işlem süreleri (yüksek doğruluk için çok sayıda örnek gerekir)
- ✗ Yaklaşık sonuçlar

### Kullanım Alanları

- **Fizik:** Parçacık fiziği, atom altı simülasyonlar
- **Finans:** Opsiyon fiyatlandırma, risk analizi
- **İstatistik:** Parametre tahminleri
- **Makine Öğrenmesi:** Optimizasyon problemleri

**Monte Carlo yöntemleri**, çeşitli alanlarda karmaşık problemlerin çözümüne yardımcı olan esnek bir araçtır. Ancak, doğruluk örnekleme sayısına bağlı olarak değişir.



# Uygulamalı Örnekler ve Mülakat Soruları

Sevgili Okuyucu,

Bu noktaya kadar geldiysen, algoritmaların yalnızca teorik kavramlar değil, gerçek **dünya problemlerini çözme gücüne sahip** stratejik araçlar olduğunu artık biliyorsun.

Artık sıra, öğrendiklerini pratiğe dökme ve algoritmik düşünme becerini bir üst seviyeye taşıma zamanı.

 **Temel algoritmaların uygulamalı örnekleri için geliştirdiğim açık kaynak GitHub projesine göz atabilirsin:**

 <https://github.com/enestoy/PHP-Algorithms-Basics-to-Business>

Bu repo, sıralama, arama, veri yapıları ve daha fazlasını PHP diliyle uygulamalı olarak açıklıyor.

 **Teknik mülakatlarda karşılaşılabileceğin algoritmik sorular ve çözümleri için:**

 <https://github.com/enestoy/Top-Interview-Algorithms>

Burada, kitap boyunca işlenen algoritmalara dayalı gerçek mülakat sorularını ve detaylı çözümlerini bulacaksın.

Unutma, iyi bir yazılımcı olmak sadece kod yazmakla değil, doğru çözümü en verimli şekilde üretmekle mümkündür. Bu kaynaklarla birlikte hem teknik bilgi seviyeni hem de problem **çözme hızını geliştirebilir, mülakatlara ve gerçek iş senaryolarına** daha donanımlı bir şekilde hazırlanabilirsin.

**Yeni projelerde, yeni başarı hikâyelerinde** görüşmek üzere.

İyi çalışmalar ve bol kodlamalar! 



# Teşekkürler!

Sevgili Okuyucu,

Bu e-kitabı okuduğunuz için teşekkür ederim. Algoritmalar dünyasına adım atarken sizlere rehberlik edebilmek benim için büyük bir keyifti. Bu kitapta anlatılan konular, yazılım geliştirme ve algoritmalarla ilgili temel ve ileri düzey bilgilerinizi pekiştirmek için bir araç olmayı hedefledi.

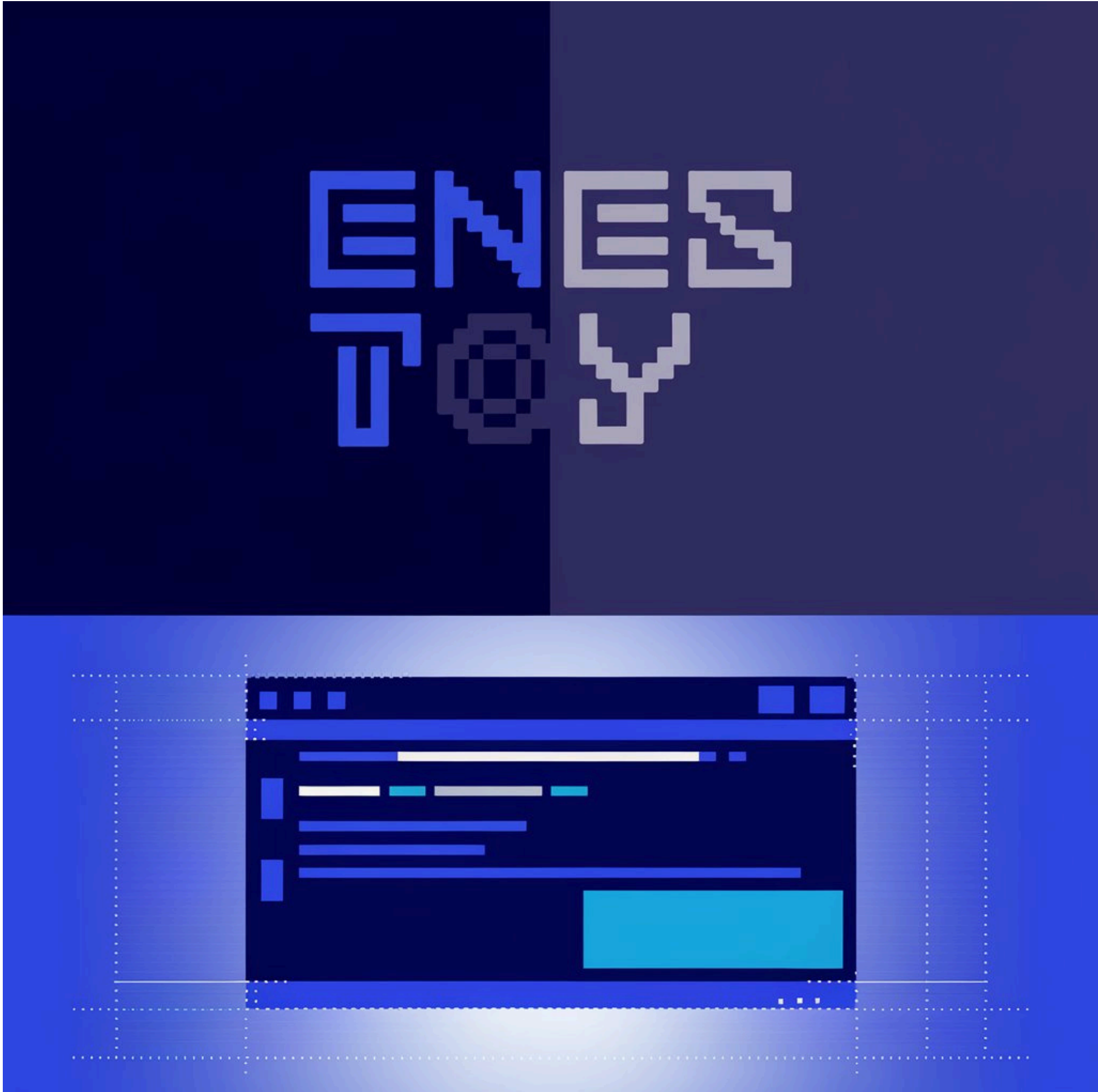
Sizden alacağım geribildirimler, kitabın daha iyi hale gelmesi için çok değerli olacaktır. Yorumlarınızı, önerilerinizi veya sorularınızı benimle paylaşmaktan çekinmeyin. Ayrıca, ilerleyen zamanlarda bu alandaki başka projelerde de birlikte olmayı umuyorum. Eğer daha fazla bilgi edinmek isterseniz, GitHub hesabımdan projelerimi inceleyebilir ve diğer kaynaklara ulaşabilirsiniz.

## İletişim ve Bağlantılar:

- **E-posta:** [enestoy@protonmail.com](mailto:enestoy@protonmail.com)
- **LinkedIn:** [linkedin.com/in/enestoy](https://www.linkedin.com/in/enestoy)
- **GitHub:** [github.com/enestoy](https://github.com/enestoy)
- **Web Site:** [www.enestoy.com](http://www.enestoy.com)

Başarılar dilerim ve umarım öğrendikleriniz hayatınızda size yeni fırsatlar sunar!

İyi çalışmalar!



# ENES TTY

